忍

SHINOBI AI (NARUTO)

Complete system documentation: every file, every module, how it works

Real-time Naruto hand-sign recognition and jutsu effects
FastAPI + WebSocket backend · React + TypeScript frontend · MediaPipe, YOLOX and a landmark classifier

01 November 2026

Contents

Part 1: The Recognition Core (the shinobi package)

- Architecture overview
- shinobi/__init__.py
- shinobi/config.py
- shinobi/camera.py
- shinobi/hands.py
- shinobi/features.py
- shinobi/classifier.py
- shinobi/seal_detector.py
- shinobi/fusion.py
- shinobi/controller.py
- shinobi/engine.py
- shinobi/events.py
- shinobi/filters.py
- shinobi/tracking.py
- shinobi/segmentation.py
- shinobi/hud.py
- shinobi/main.py
- shinobi/catalog.py
- Summary: how a sign becomes a jutsu (worked example)

Part 2 — Server, protocol and effects

- shinobi/server.py
- The effects package: overview
- shinobi/effects/__init__.py
- shinobi/effects/base.py
- shinobi/effects/fx.py
- shinobi/effects/fire.py
- shinobi/effects/chidori.py
- shinobi/effects/rasengan.py
- shinobi/effects/shadow_clone.py
- assets/audio/
- gesture_references/

Part 3 — Frontend (React)

- Overview: pages and data flow
- Project files at the root of frontend/
- Entry point and routing
- Types: the contract with the backend
- Services: talking to the server
- Hooks: camera and session
- Contexts (app-wide state)
- The Konoha UI: frame and shared pieces
- The dojo (Home and Live)
- Live page
- Jutsu page (library)
- Carousel
- Practice page

Settings page
Background music
Audio engine (sound effects, voices, jutsu music)
Live overlay components (src/components/live)
Art components (src/components/art)
Older layout and UI kit (src/components/layout, src/components/ui)
Older pages (src/pages)
Styles and assets
Unit tests: how to run them
End-to-end tests (frontend/e2e)

Part 4 — Data, training, evaluation, tools and tests

The big picture in one page
data/ — the data layout
models/ — every file
requirements.txt
README.md
collect_data.py
record_sequences.py
splits.py
train.py
evaluate.py
tools/recording_review.py
tools/record_training_session.py
tools/record_test_session.py
tools/assign_recordings.py
tools/make_test_set.py
tools/grouped_cv.py
tools/evaluate_pipeline.py
tools/check_voices.py
tools/build_bold_signs.py
tools/build_gesture_references.py
reports/ — what each folder is
tests/ — what each test module checks
tests/test_core.py
tests/test_splits.py
tests/test_evaluate.py
tests/test_recording_tools.py
tests/test_sequences.py
tests/test_fusion.py
tests/test_engine.py
tests/test_events.py
tests/test_activation.py
tests/test_shadow_clone.py
tests/test_catalog.py
tests/test_server.py
Current state of the model and evaluation
How to run everything

Part 1: The Recognition Core (the shinobi package)

This part explains every Python file in the `shinobi/` package except `shinobi/server.py` (the web server) and the `shinobi/effects/` folder (the visual effects). Those two are covered in later parts. Everything described here is taken from the code as it is today, so if a number in this text and a number in the code ever differ, the code wins.

The files covered in this part are:

- `shinobi/__init__.py`
- `shinobi/config.py`
- `shinobi/camera.py`
- `shinobi/hands.py`
- `shinobi/features.py`
- `shinobi/classifier.py`
- `shinobi/seal_detector.py`
- `shinobi/fusion.py`
- `shinobi/controller.py`
- `shinobi/engine.py`
- `shinobi/events.py`
- `shinobi/filters.py`
- `shinobi/tracking.py`
- `shinobi/segmentation.py`
- `shinobi/hud.py`
- `shinobi/main.py`
- `shinobi/catalog.py`

Architecture overview

What the program does, in one paragraph

SHINOBI AI watches you through a webcam. When you perform the four hand seals of a jutsu in the right order (for example Monkey, Boar, Horse, Tiger for the Fire Style), it plays that jutsu's visual effect on top of the live video: a fireball from your hand, lightning (Chidori), a spinning sphere on your palm (Rasengan), or ten copies of you (Shadow Clone). The program can run in two ways: as a desktop window (`python -m shinobi.main`, an OpenCV window) or as a web app (`python -m shinobi.server`, a FastAPI/WebSocket server that a React page in the browser talks to). Both ways use exactly the same recognition core, the `Engine` class in `shinobi/engine.py`.

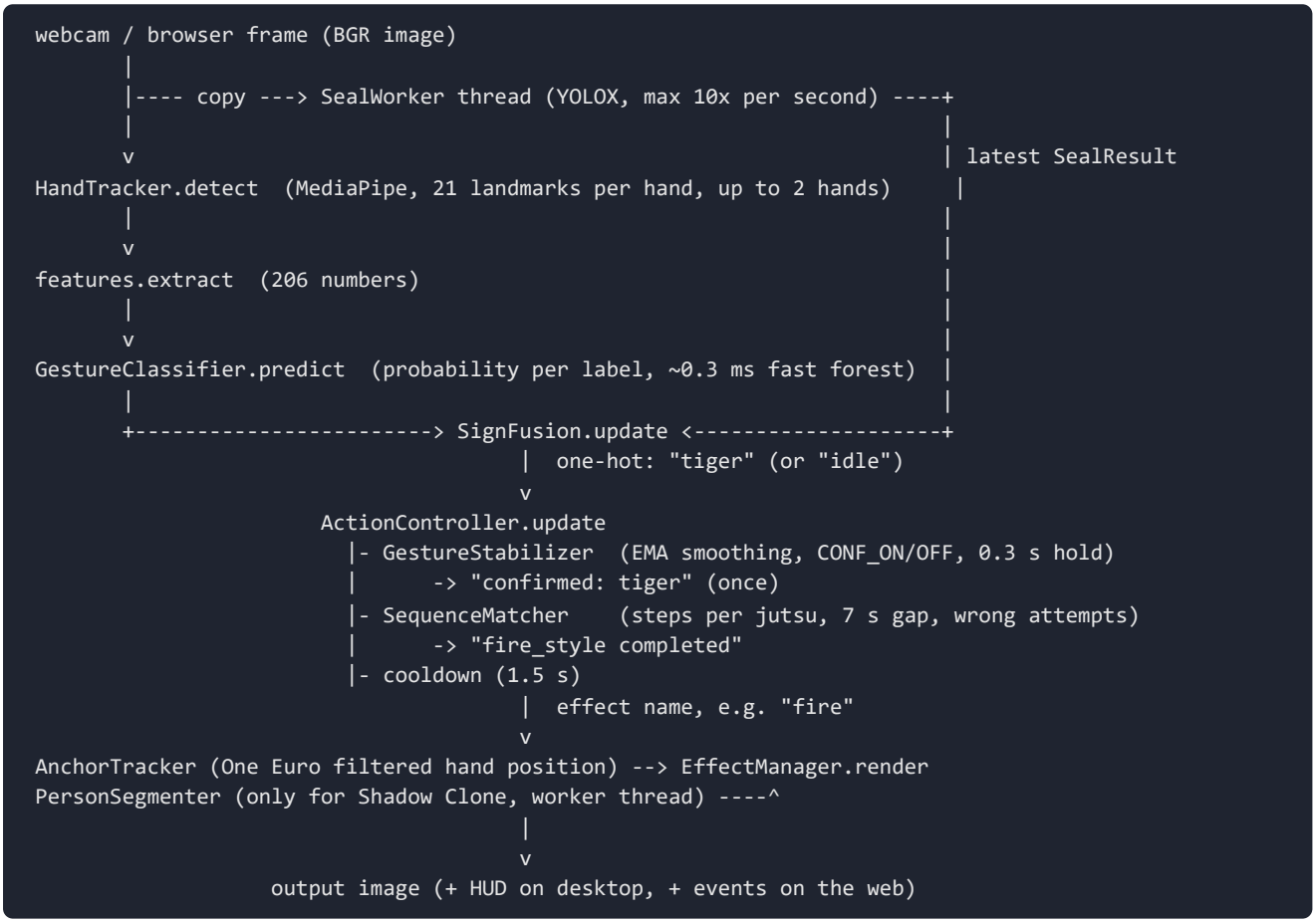
The two recognisers

The project uses two different AI models to recognise hand signs, because neither one alone can recognise all ten signs:

- **YOLOX-Nano seal detector** (`seal_detector.py`). A pretrained object detector that looks at the whole camera image and draws a box around a hand seal. It knows the 12 official Naruto seals plus a few extra classes, and the project uses 7 of them: tiger, snake, horse, ram, monkey, dog, boar. It does not know the two custom signs.
- **Landmark classifier** (`hands.py` + `features.py` + `classifier.py`). MediaPipe finds the 21 joints (landmarks) of each hand, `features.py` turns those joints into 206 numbers, and a model trained by the project owner (a random forest or a small neural network, saved in `models/gesture_model.joblib`) turns the 206 numbers into a probability for each of the 10 labels. This model is responsible for the two custom signs, `clone_seal` and `rasengan_palm`.

`fusion.py` combines both opinions into ONE decision per frame, so a sign can never be counted twice.

The full pipeline, from camera frame to jutsu



Per-frame order inside Engine.process

1. The newest frame is handed to the YOLOX worker thread (it never makes the main loop wait).
2. If the active effect needs a person mask (only Shadow Clone), segmentation starts on a second worker thread.
3. MediaPipe finds the hands in the frame.
4. If at least one hand is visible and a landmark model is loaded, the 206 features are built and the classifier gives probabilities.
5. `SignFusion` combines the classifier probabilities with the latest YOLOX result into a single sign (or idle).
6. `ActionController` smooths that decision over time, confirms a sign after it is held, follows all jutsu sequences, and fires a jutsu when one is complete (unless still cooling down).

7. `AnchorTracker` picks the hand that effects should follow and smooths its position.
8. The effect manager draws the active effect on the frame.
9. The result (image, timings, sign, progress, time left, effect stage) is returned in an `EngineResult`.

The desktop app (`main.py`) then draws a HUD and shows the image; the web server (`server.py`) passes a summary to `EventTracker` (`events.py`) to produce protocol events, encodes the image as JPEG and sends both to the browser.

Design ideas that run through the whole core

- **The AI is separate from the drawing.** The recognisers never know which effect will play, and the effects never know which model was used. They only meet through an effect name and a hand position.
- **Models look at one frame; time is handled by rules.** Instead of a model that watches video (like an LSTM), each frame is classified on its own, and the controller adds the time rules (hold a sign for 0.3 s, follow the order, 7 s limit between steps). This is easier to debug and needs far less training data.
- **All tunable numbers live in `config.py`.** Thresholds, times and the definition of every jutsu.
- **Measured, not guessed.** Many comments in the code mention measurements on the development laptop (an Intel i7-8550U), for example camera opening time or classifier speed.

shinobi/__init__.py

What it is for

This file makes `shinobi` a Python package (so you can write `from shinobi import config`). It also contains one important start-up setting for Windows.

What it does

```
import os

# Must be set before OpenCV opens a camera: without it the Windows Media Foundation
# backend takes ~15 s to open the webcam. (Measured on this laptop: 14.4 s -> 0.6 s.)
os.environ.setdefault("OPENCV_VIDEOIO_MSMF_ENABLE_HW_TRANSFORMS", "0")
```

Python runs a package's `__init__.py` the first time anything from the package is imported. So this environment variable is set before any other module (in particular `camera.py`) can open a camera. The variable turns off hardware transforms in Windows Media Foundation, the camera backend the project uses; with them on, opening the webcam took about 14.4 seconds on the development laptop, without them about 0.6 seconds. `setdefault` means: only set it if the user has not already set it themselves.

Connections

Every other module imports something from `shinobi`, so this always runs first.

shinobi/config.py

What it is for

The single place for every path, threshold, time limit and jutsu definition. No other file in the core hard-codes these values; they all read them as `config.NAME`. Changing behaviour (for example making the sign hold time longer) means changing one number here. The file contains no logic, only constants.

Paths

Name	Value	Meaning
`ROOT`	the project folder (parent of `shinobi/`)	Computed from this file's own location, so the project works from any folder.
`MODELS_DIR`	`ROOT / "models"`	Where the model files live.
`DATA_DIR`	`ROOT / "data"`	Recorded training data.
`LANDMARKS_DIR`	`DATA_DIR / "landmarks"`	Recorded hand landmarks (training data).
`VIDEOS_DIR`	`DATA_DIR / "videos"`	Recorded videos.
`HAND_MODEL_PATH`	`models/hand_landmarker.task`	MediaPipe hand landmark model.
`SEGMENTER_MODEL_PATH`	`models/selfie_segmenter.tflite`	MediaPipe person segmentation model (Shadow Clone).
`GESTURE_MODEL_PATH`	`models/gesture_model.joblib`	The trained landmark classifier (created by `train.py`).
`REFERENCES_DIR`	`ROOT / "gesture_references"`	Pictures and explanations of each sign.
`SEAL_MODEL_PATH`	`models/yolox_nano.onnx`	The YOLOX-Nano seal detector.

Camera and hand tracking

Name	Value	Meaning
`CAMERA_INDEX`	`None`	Which webcam to use. `None` = try cameras 0..3 and use the first that shows a real (not black) picture.
`CAMERA_WIDTH`	`1280`	Requested camera width in pixels.
`CAMERA_HEIGHT`	`720`	Requested camera height in pixels.
`MIRROR`	`True`	Flip the image left-right, so the screen behaves like a mirror (selfie view). Also used by the web server.
`MAX_HANDS`	`2`	MediaPipe looks for at most 2 hands.
`MIN_DETECTION_CONFIDENCE`	`0.5`	MediaPipe: minimum score to detect a new hand.
`MIN_PRESENCE_CONFIDENCE`	`0.5`	MediaPipe: minimum score that a tracked hand is still there.
`MIN_TRACKING_CONFIDENCE`	`0.5`	MediaPipe: minimum score to keep tracking instead of detecting again.

Signs (gestures)

`GESTURES` is the list of the 10 labels the system knows:

```
GESTURES = ["idle", "ram", "snake", "tiger", "dog", "monkey", "boar", "horse", "clone_seal",
"rasengan_palm"]
```

Index 0 must be `"idle"` (meaning "no sign": hands down, waving, half-formed signs). The order matters because it is stored together with the trained model. `GESTURE_HINTS` gives a one-line description of each sign (used by the recording tools), for example `"rasengan_palm": "one hand: open palm toward camera, fingers spread"`.

`LEGACY_LABELS = {"rasengan": "rasengan_palm"}` translates the label of old recordings, made before the sign set changed, when loading data. Old labels that are in neither place (the former "chidori" claw and "fire" OK-ring) stay on disk but are not used for training.

YOLOX hand-seal detector

Name	Value	Meaning
`SEAL_INPUT_SIZE`	`416`	The ONNX model only accepts 416 x 416 images.
`SEAL_THREADS`	`2`	CPU threads for onnxruntime (measured: best speed per core used).
`SEAL_RATE_HZ`	`10`	At most 10 YOLOX detections per second, in its own thread.
`SEAL_CLASSES`	`{2: "tiger", 5: "snake", 6: "horse", 7: "ram", 8: "monkey", 10: "dog", 11: "boar"}`	Model class id to project sign name. All other classes (rat, ox, hare, dragon, bird, and extra classes such as gassho, unknown, mizunoe) are ignored.

A comment notes the model's origin (Kazuhito00/NARUTO-HandSignDetection): the code is MIT licensed, but the weights and their training data are not clearly licensed, so the model is for personal and educational use only.

Recognition fusion

Name	Value	Meaning
`LANDMARK_SIGNS`	`["clone_seal", "rasengan_palm"]`	Signs owned by the landmark classifier (YOLOX has no class for them).
`SEAL_MIN_SCORE`	`0.70`	YOLOX score needed when its box lies on a hand MediaPipe sees.
`SEAL_NO_HAND_SCORE`	`0.85`	YOLOX score needed when MediaPipe sees no hand at all (some seals hide the fingers so well that MediaPipe loses the hands).
`SEAL_CONFIRM`	`2`	The same seal must win this many YOLOX results in a row.
`SEAL_MAX_AGE`	`0.35`	Seconds a YOLOX result stays valid.
`LANDMARK_MIN_PROB`	`0.60`	Classifier probability needed for clone_seal / rasengan_palm.
`LANDMARK_BACKUP_PROB`	`0.90`	The classifier may report an official seal only if YOLOX is not running, and only at this probability.
`LANDMARK_MIN_MARGIN`	`0.30`	clone_seal / rasengan_palm also need top1 minus top2 probability of at least this. The comment records that on held-out rasengan_palm recordings the 5th percentile of the margin was 0.84 (measured 2026-11-01), so real palms clear 0.30 easily.

Name	Value	Meaning
`PALM_MIN_THUMB_GAP`	`0.60`	rasengan_palm needs the thumb tip to be away from the index tip by at least 0.60 x the hand size. The OK sign (thumb touching index) has a gap of about 0.3-0.5, real open palms 0.8-1.2.

Recognition to action (stabilizer, sequences, readiness)

Name	Value	Meaning
`PROB_SMOOTHING`	`0.4`	EMA weight of the newest frame at the reference frame rate.
`PROB_SMOOTHING_FPS`	`15`	The frame rate that weight was tuned at; at other frame rates the weight is rescaled so the smoothing per second stays the same.
`CONF_ON`	`0.75`	Smoothed probability needed to start confirming a sign.
`CONF_OFF`	`0.45`	Below this smoothed probability a confirmed sign is released.
`MIN_MARGIN`	`0.25`	Smoothed top1 minus top2 must be at least this.
`HOLD_SECONDS`	`0.30`	The sign must stay strong for this long before it is confirmed.
`COOLDOWN_SECONDS`	`1.5`	After a jutsu fires, new jutsu are ignored for this long.
`SEQUENCE_MAX_GAP`	`7.0`	Maximum seconds from one confirmed step (or wrong attempt) to the next step of the same sequence. It is a limit, not a wait. Raised from 3.5 s because changing hands between two seals took longer in real use.
`SEQUENCE_CLOCK_MAX_STEP`	`0.4`	One processed frame adds at most 0.4 s to the sequence clock, so a stall does not eat into the 7 s.
`READY_FRAMES`	`5`	Signs count only after this many frames in a row ran fast enough.
`READY_MAX_MS`	`500`	"Fast enough" = a frame processed in under 500 ms.

The jutsu table

```
JUTSU = {
    "fire_style": {"sequence": ["monkey", "boar", "horse", "tiger"], "effect": "fire", "official":
True},
    "chidori": {"sequence": ["snake", "dog", "tiger", "monkey"], "effect": "chidori", "official":
True},
    "shadow_clone": {"sequence": ["ram", "snake", "tiger", "clone_seal"], "effect": "shadow_clone",
                     "official": False},
    "rasengan": {"sequence": ["clone_seal", "dog", "horse", "rasengan_palm"], "effect": "rasengan",
                 "official": False},
}
```

Each jutsu has an ordered `sequence` of four signs, the name of the `effect` it plays, and `official` (True when the four signs are consecutive seals from the anime; False for a custom SHINOBI AI sequence). A rule enforced at start-up by `SequenceMatcher._check_conflicts`: no jutsu may appear (in order) inside another one before that one's last sign. Adding a new jutsu is one new line in this table.

Effects, tracking and segmentation (used by the effects part)

Name	Value	Meaning
`EFFECT_DURATION`	shadow_clone 10.0, rasengan 8.0, chidori 8.0, fire 6.0	Seconds each main effect plays.

Name	Value	Meaning
`EFFECT_KEYS`	keys 1, 2, 3, 4	Desktop keys that start shadow_clone, rasengan, chidori, fire directly.
`HAND_LOST_GRACE`	`0.6`	Seconds the hand anchor still counts as visible after MediaPipe lost the hand.
`HAND_EFFECT_MIN_SECONDS`	`None`	Hand effects keep running at the last hand position when the hand is lost (None = never ended by hand loss).
`ACTIVATION_CHARGE`	1.2 s per jutsu	Length of the "charge" build-up stage.
`ACTIVATION_CHARGE_DEFAULT`	`1.2`	Charge length for a jutsu missing from the table.
`VOICE_DEFAULT`	`0.4`	Length of the "voice" and "name" stages when no voice clip is installed.
`VOICE_GAP`	`0.05`	Added after a voice clip's own length.
`CLONE_COUNT`	`10`	Number of shadow clones (11 people on screen).
`CLONE_OFFSETS`, `CLONE_SCALES`, `CLONE_LIFTS`, `CLONE_DELAYS`	10 values each	Place, size, height lift and time delay of each clone (three rows: 2, 4, 4).
`CLONE_PARALLEL`	`True`	Build the clone layer on a worker thread.
`SEGMENT_WIDTH`	`640`	Segmentation runs on a frame scaled to 640 px width (speed).
`MASK_SMOOTHING`	`0.5`	EMA weight of the newest person mask.
`FILTER_MIN_CUTOFF`	`1.5`	One Euro filter: smaller = smoother but more lag.
`FILTER_BETA`	`0.02`	One Euro filter: how quickly smoothing relaxes when the hand moves fast.

The effects part of this documentation explains how the clone and activation values are used.

Data collection and web server

Name	Value	Meaning
`RECORD_SECONDS`	`20`	Length of one recording session (`collect_data.py`).
`COUNTDOWN_SECONDS`	`3`	Countdown before recording.
`SEQUENCES_DIR`, `SEQUENCE_VIDEOS_DIR`	`data/sequences`, `data/videos/sequences`	Guided sequence recordings.
`SEQ_HOLD_SECONDS`, `SEQ_TRANSITION_SECONDS`, `SEQ_LEAD_SECONDS`	1.5 each	Hold, change-hands and lead-in times of guided recordings.
`APP_VERSION`, `PROTOCOL_VERSION`	`"0.1.0"`, `1`	Reported to the React app.
`SERVER_HOST`, `SERVER_PORT`	`"127.0.0.1"`, `8000`	Where the server listens.
`SERVER_MAX_SESSIONS`	`2`	Browser sessions at the same time (each has its own Engine).

Name	Value	Meaning
`FRAME_MAX_WIDTH`, `FRAME_MAX_HEIGHT`	1280, 1280	Larger browser frames are scaled down.
`FRAME_MIN_SIZE`	`32`	Smaller frames are refused.
`FRAME_JPEG_QUALITY`, `OUTPUT_JPEG_QUALITY`	80, 90	JPEG quality of incoming and outgoing frames.
`OUTPUT_JPEG_444`	`True`	Full colour resolution in outgoing JPEGs.
`FRAME_MAX_IN_FLIGHT`	`2`	Frames a client may send before receiving an answer.
`FRAME_MAX_BYTES`	4 MiB	Larger messages are refused.
`FRONTEND_DIST`, `AUDIO_DIR`	`frontend/dist`, `assets/audio`	Built React app and sound files.

Connections

Imported by almost every file. `catalog.py` reads `EFFECT_DURATION`, `CLONE_COUNT` and `AUDIO_DIR`; the server reads the server block; `train.py` and the recording scripts read paths and `GESTURES`.

shinobi/camera.py

What it is for

Reads the webcam on a background thread and always hands out the newest frame. Used only by the desktop app (`main.py`); in the web app the browser owns the camera.

Why a thread

`cap.read()` blocks until the camera delivers the next frame (about 33 ms at 30 FPS). If the main loop called it directly, it would spend that time waiting instead of running the AI. With a thread, the camera keeps filling a "latest frame" slot, and the main loop takes whatever is newest without waiting. Frames the main loop was too slow for are simply overwritten, which keeps the display current rather than lagging behind.

Functions

- `_open(index, width, height)`: opens camera `index` with the Media Foundation backend (`cv2.CAP_MSMF`) and requests the resolution. The comment explains the choice: MSMF delivers 30 FPS at 720p on the development laptop, DirectShow only about 10.
- `_is_real(cap, tries=10)`: reads up to 10 frames and returns True if one of them has an average brightness above 5. Virtual cameras (for example Camo with no phone attached) open without error but send black frames; this filters them out.
- `find_camera(width, height, max_index=4)`: tries camera indices 0 to 3 and returns `(index, capture)` for the first real one. Raises `RuntimeError("No working camera found ...")` if none works.

class Camera

- `__init__(index, width, height, mirror)`: defaults come from config. With `index=None` it calls `find_camera`; otherwise it opens that index and raises an error if it cannot. It prints the actual resolution, then starts the reader thread (a daemon thread, so it does not keep Python alive at exit).
- `_loop()`: the thread body. Reads a frame; if reading failed, sleeps 10 ms and tries again. Flips the frame horizontally when `mirror` is on (`cv2.flip(frame, 1)`). Stores the frame and increments `_frame_id`, under a lock.
- `read()`: returns `(frame_id, frame)` under the lock. The id lets the caller skip a frame it has already processed (the main loop compares it to the last id it handled).
- `release()`: stops the loop, waits up to 1 s for the thread, and releases the camera.

The lock (`threading.Lock`) guarantees that `read()` never returns a frame and an id that belong to different moments.

Connections

Used by `main.py`. Reads `CAMERA_INDEX`, `CAMERA_WIDTH`, `CAMERA_HEIGHT`, `MIRROR` from config.

shinobi/hands.py

What it is for

A small wrapper around MediaPipe's Hand Landmarker (Tasks API). It turns a camera frame into a list of `Hand` objects, each with 21 landmarks (joints) in three coordinate systems. It also has a helper to draw the hand skeleton.

Landmark indices

MediaPipe numbers the 21 hand points: 0 is the wrist; 1-4 the thumb (4 = thumb tip); 5-8 the index finger (5 = knuckle, 8 = tip); 9-12 the middle finger; 13-16 the ring finger; 17-20 the little finger. The file names the ones it needs:

```
WRIST, THUMB_TIP, INDEX_MCP, INDEX_TIP, MIDDLE_MCP, MIDDLE_TIP = 0, 4, 5, 8, 9, 12
RING_MCP, PINKY_MCP = 13, 17
FINGERTIPS = [4, 8, 12, 16, 20]
PALM = [0, 5, 9, 13, 17]
```

`CONNECTIONS` lists the pairs of points joined by a line when drawing the skeleton.

dataclass Hand

Field	Shape	Meaning
`landmarks`	(21, 3)	Normalized image coordinates: x and y from 0 to 1 across the image, z a relative depth.
`world`	(21, 3)	Real-world 3D coordinates in metres, origin near the hand centre. Not affected by how far the hand is from the camera.
`handedness`	str	"Left" or "Right" (as MediaPipe reports it for the mirrored image).
`score`	float	Handedness confidence.

Field	Shape	Meaning
`px`	(21, 2)	Pixel coordinates in the frame (normalized x, y multiplied by width, height).

Two properties are used by the effects: `palm_center` (mean pixel position of the 5 palm points) and `size_px` (pixel distance from wrist to middle knuckle, a scale reference so effects grow with the hand).

class HandTracker

- `__init__()`: creates the landmarker in `VIDEO` running mode with `num_hands=MAX_HANDS` and the three confidence thresholds from config. VIDEO mode means MediaPipe tracks hands from frame to frame instead of detecting from scratch every time, which is faster and steadier.
- `_timestamp()`: VIDEO mode requires strictly increasing timestamps in milliseconds. It uses `time.monotonic()` and, if two calls land in the same millisecond, adds 1 to the previous value.
- `detect(frame_bgr)`: converts BGR (OpenCV's colour order) to RGB (MediaPipe's), runs `detect_for_video`, and builds one `Hand` per detected hand. It then sorts the hands by the wrist's x coordinate, left to right on screen.

```
# Order by wrist x, not by handedness: handedness flips when hands overlap.
hands.sort(key=lambda hand: hand.landmarks[WRIST, 0])
```

This sorting is important for the features: slot 0 is always the left hand on screen and slot 1 the right. MediaPipe's "Left/Right" label is not reliable when hands overlap (as in most seals), so position is used instead.

- `close()`: releases the MediaPipe object.

draw_hands(frame, hands)

Draws white lines for `CONNECTIONS`, orange-yellow dots for fingertips, orange dots for other joints, and the handedness label under the wrist. Used when "draw landmarks" is on (desktop key L).

Connections

`engine.py` creates a `HandTracker` and calls `detect` every frame. The `Hand` objects go to `features.hands_to_arrays`, `fusion.py` (hand boxes and open-palm gate), `tracking.py` (anchor) and the effects.

shinobi/features.py

What it is for

Turns raw landmarks into a fixed-length vector of 206 numbers, the input of the landmark classifier. The same function is used by training (`train.py`) and by the live app, so the model always sees features computed in exactly the same way. `FEATURE_VERSION = 1` is saved with the trained model; if the feature code changes, the version must change, and `classifier.py` refuses a model trained with another version.

Why features instead of raw coordinates

Raw pixel positions change when you move your hand across the image, come closer or step back, but the sign is the same. The features remove these differences: positions are measured from the wrist, divided by

the hand's size, and complemented by finger angles and fingertip distances, which describe the shape of the hand directly.

Layout of the 206 numbers

Part	Count	Content
Hand slot 0 (left on screen)	89	1 presence flag + 63 normalized 3D coordinates + 15 joint angles + 10 fingertip distances
Hand slot 1 (right on screen)	89	same
Between the two hands	28	1 "both hands" flag + 2 wrist offset (x, y) + 25 fingertip-to-fingertip distances (5 x 5)
Total	206	`FEATURE_SIZE = 2 * PER_HAND_SIZE + INTER_HAND_SIZE`

A missing hand gives 89 zeros (its presence flag 0); without two hands the 28 inter-hand numbers are zeros.

hands_to_arrays(hands)

Converts a list of `Hand` objects (at most 2 used) into the raw storage format: `image_lm` (2, 21, 3), `world_lm` (2, 21, 3) and `present` (2,) with 1.0 for each slot that has a hand. This is also the format the recording tools save to disk, so training can recompute features later.

_hand_features(world): one hand

Works on the 3D world landmarks of one hand:

```
rel = world - world[0]                                  # wrist becomes the origin
scale = np.linalg.norm(rel[9]) + 1e-6                   # wrist -> middle knuckle
rel = rel / scale                                       # same size for near/far hands
```

1. **Coordinates (63 numbers).** Every point minus the wrist, divided by the wrist-to-middle-knuckle distance. So position and hand size no longer matter, only shape.
2. **Angles (15 numbers).** For each of the 15 triplets in `ANGLE_TRIPLETS` (three per finger, for example `(5, 6, 7)` = the angle at index joint 6), the angle between vectors b->a and b->c is computed with the dot product formula `cos = (v1 . v2) / (|v1| |v2|)`, then `arccos`, then divided by pi. The result is 0..1, where 1 means a straight finger and small values a bent one. `np.clip` keeps the cosine in -1..1 so rounding errors cannot make `arccos` fail.
3. **Fingertip distances (10 numbers).** The distance between every pair of the 5 fingertips (`TIP_PAIRS`, 10 pairs), divided by the same scale. These capture things like "thumb touching index".

The vector starts with `1.0`, the presence flag.

extract(image_lm, world_lm, present, aspect=16/9): the full vector

Builds the 89 numbers for each slot (or zeros), then the inter-hand part when both hands are present. The inter-hand part uses the 2D image coordinates, because world coordinates of two hands are each centred on their own hand and cannot tell where the hands are relative to each other:

```
pts = image_lm[:, :, :2] * np.array([aspect, 1.0])
palm = np.mean([np.linalg.norm(pts[s, 9] - pts[s, 0]) for s in range(2)]) + 1e-6
offset = (pts[1, 0] - pts[0, 0]) / palm
cross = np.linalg.norm(pts[0, TIPS][:, None] - pts[1, TIPS][None], axis=2) / palm
```

Image x and y both go from 0 to 1, but the image is wider than tall, so x is multiplied by the aspect ratio (width / height) to make distances real proportions. `palm` is the average wrist-to-middle-knuckle distance of the two hands, used as the scale. `offset` is the vector from the left wrist to the right wrist. `cross` is a 5 x 5 table of distances between every fingertip of hand 0 and every fingertip of hand 1 (computed in one step with numpy broadcasting). The result is returned as `float32`.

mirror(...) and order_slots(...)

Used for data augmentation during training. `mirror` creates the left-right mirror image of a recorded frame: image x becomes `1 - x` (only for present hands), world x changes sign. Because mirroring swaps which hand is on the left, `order_slots` re-sorts the slots: present hands first, then by wrist x, the same rule as `HandTracker.detect`. This lets a sign recorded with one hand arrangement also teach the model the mirrored arrangement.

Connections

`engine.py` calls `extract(*hands_to_arrays(hands), aspect=w / h)`. `classifier.py` imports `FEATURE_VERSION`. `train.py` uses `extract`, `mirror` and `order_slots`.

shinobi/classifier.py

What it is for

Loads the trained landmark model from `models/gesture_model.joblib` and turns a 206-number feature vector into one probability per label. It also contains a speed optimization: a hand-written, vectorized evaluator for random forests that gives exactly the same numbers as scikit-learn, about 100 times faster.

The saved model bundle

`train.py` saves a dictionary with `joblib`: `"model"` (a scikit-learn model; training compares a random forest with 300 trees and an MLP neural network and keeps the best), `"model_name"`, `"labels"` (label names in the model's output order) and `"feature_version"`.

class GestureClassifier

- `__init__(path=None)`: loads the bundle. If `feature_version` differs from `features.FEATURE_VERSION` it raises an error telling you to re-run `train.py`, because a model trained on different features would give meaningless answers. It stores `model`, `labels` and `name`. It sets every `n_jobs` parameter to 1: during training `n_jobs=-1` uses all CPU cores, but for a single frame starting a thread pool on every call only costs time. Finally `self.fast = fast_forest(self.model)`.
- `predict(features)`: returns a float32 probability array aligned with `self.labels`. If the fast forest is available and the features contain no NaN or infinity, it uses it; otherwise it calls scikit-learn's `predict_proba` (scikit-learn treats NaN specially, so those cases are left to it).

- `exists(path=None)`: static method, True if the model file exists. `engine.py` uses it to decide whether to load a model at all.

The fast random-forest path: ForestOneFrame

The problem. A random forest is many decision trees. Each tree asks a chain of questions like "is feature 37 <= 0.42? then go left, else right" until it reaches a leaf, which holds class proportions. The forest's answer is the average of all trees' leaf proportions. scikit-learn walks the 300 trees one by one, checks the input on every call, and (with `n_jobs=-1`) starts a thread pool: 40-50 ms per frame (measured 2026-10-31), while the whole frame budget is only 70-100 ms.

The idea. Pack all 300 trees into flat numpy arrays and walk all of them at the same time, one tree level per numpy step. Instead of 300 separate walks, there are only `depth` steps, each processing 300 nodes at once.

How the arrays are built in `__init__(forest)`:

1. `counts` = number of nodes of each tree. `roots` = where each tree starts in the flat arrays (0, then running totals).
2. `shift` repeats each tree's start position for all its nodes, so a tree's own child ids (which start at 0 in each tree) can be converted to flat ids by adding the shift.
3. `left` / `right` = children of every node. In scikit-learn a leaf has child -1. Here a leaf points to itself:

```
leaf = left < 0
here = np.arange(len(left))
self.left = np.where(leaf, here, left + shift)           # a leaf points to itself, so walking on
keeps it
self.right = np.where(leaf, here, right + shift)
self.feature = np.where(leaf, 0, np.concatenate([t.feature for t in trees]))
```

This trick means that a tree that reaches a leaf early just stays there while deeper trees keep walking, so every tree can be stepped exactly `depth` times without special cases. A leaf's feature is set to 0 (any valid index) because the comparison result does not matter there.

1. `threshold` = the split value of each node. `proba` = the leaf values divided by their sum, which is each tree's `predict_proba` at that leaf. `depth` = the deepest tree's depth.

Evaluating one frame:

```
def __call__(self, features):
    x = np.asarray(features, np.float32).ravel()            # sklearn also compares float32 features
    node = self.roots
    for _ in range(self.depth):
        node = np.where(x[self.feature[node]] <= self.threshold[node], self.left[node],
self.right[node])
    return self.proba[node].mean(axis=0)                     # the forest = mean of its trees
```

`node` is an array of 300 current positions (one per tree). Each loop iteration moves all of them one level down. After `depth` steps every tree is at a leaf, and the mean of their leaf probabilities is the forest's answer. Features are converted to float32 because scikit-learn also compares in float32; this keeps the left/right decisions identical. Measured time: about 0.3 ms.

Safety check: fast_forest(model). Returns a `ForestOneFrame` only if the model is a single-output `RandomForestClassifier` and its answers match scikit-learn: 16 random test vectors (from a fixed seed, the first set to all zeros) are evaluated both ways and compared with `np.allclose(..., atol=1e-9)`. If they

ever differ, it prints a message and returns None, and `GestureClassifier` falls back to scikit-learn. For an MLP model it returns None directly (the MLP is fast enough already).

Connections

`engine.py` creates a `GestureClassifier` (when the model file exists) and calls `predict` every frame with hands; `fusion.py` receives the probabilities and `classifier.labels`. The server can share one loaded classifier between sessions because it is only read.

shinobi/seal_detector.py

What it is for

Runs the pretrained YOLOX-Nano hand-seal detector (from the Kazuhito00/NARUTO-HandSignDetection project, as an ONNX file) with onnxruntime on the CPU. It has two layers: `SealDetector` (one detection on one image) and `SealWorker` (runs the detector continuously in its own thread on the newest frame, at most 10 times per second). The pre- and post-processing re-implement the original repository's `yolox_onnx.py`.

Data classes

- `Detection`: `sign` (project sign name, or None for a class no jutsu uses), `class_id`, `score`, `box` = (x0, y0, x1, y1) in pixels.
- `SealResult`: `id` (increases by one per detection run, so fusion can tell a new result from one it already counted), `time` (capture time of the frame it was computed on), `latency_ms`, `detections` (best first).

nms(boxes, scores, iou)

Greedy non-maximum suppression. A detector often outputs several overlapping boxes for the same object. NMS keeps the best-scoring box, removes every remaining box that overlaps it by more than `iou` (intersection over union), and repeats with the next best. Returns the kept indices, best first. It is class-agnostic: overlapping boxes of different classes also suppress each other, so one pair of hands yields one seal.

class SealDetector

- `__init__(path, size=416, threads=2)`: creates an onnxruntime session on the CPU with 2 intra-op threads, 1 inter-op thread, and "allow_spinning" off (idle threads sleep instead of busy-waiting and wasting CPU cores the rest of the app needs). It precomputes the anchor grid. YOLOX predicts on three grids with strides 8, 16 and 32 pixels: 52 x 52, 26 x 26 and 13 x 13 cells, 3549 cells in total. `grids` holds each cell's (x, y) and `strides` its stride.
- `detect(frame, min_score=0.1, iou=0.45)`: the full detection.

Steps of `detect`:

1. **Letterbox.** Scale factor `r = min(416 / h, 416 / w)` keeps the aspect ratio. The scaled image is placed in the top-left corner of a 416 x 416 canvas filled with grey 114 (as in the original code). The pixels stay BGR 0..255 with no normalization, because that is what this model was trained with.

2. **Run.** The canvas is transposed to channels-first (3, 416, 416), given a batch dimension and run. Output: 3549 rows of 21 numbers: dx, dy, log w, log h, objectness, and 16 class scores (already passed through a sigmoid).
3. **Score.** `score = objectness x class score` for each class; the best class per cell is kept, and cells below `min_score` are dropped.
4. **Decode boxes.** Centre = (dx, dy + cell position) x stride; size = exp(log w, log h) x stride. Converted to corner coordinates and divided by `r` to get back to the input image's pixels.
5. **NMS** at IoU 0.45, then a `Detection` per kept box, with its class mapped through `config.SEAL_CLASSES` (None for unused classes) and the box clipped to the image.

```
xy = (out[keep, :2] + self.grids[keep]) * self.strides[keep]
wh = np.exp(out[keep, 2:4]) * self.strides[keep]
boxes = np.concatenate([xy - wh / 2, xy + wh / 2], 1) / r
```

class SealWorker: the background thread

The main loop must never wait for YOLOX (it takes tens of milliseconds). `SealWorker` runs it in a separate thread:

- `submit(frame, now)`: called by the engine for every frame. It immediately shrinks the frame to 416 px width (which also makes a private copy, so the main thread may reuse its frame) and stores `(small, w / 416, now)` as the pending job, replacing any older job that was not processed yet. Then it wakes the thread with `notify()`.
- `_run()`: the thread loop. It first sleeps until `next_start` (rate limit: at most `SEAL_RATE_HZ` = 10 runs per second, period 0.1 s), then waits on the condition variable until a job is pending, takes the newest job, runs `detector.detect(small)`, scales the boxes back to full-frame pixels with the stored scale, and publishes `self.result = SealResult(run_id, stamp, latency, detections)`. The stored `stamp` is the capture time of the frame, not the time the detection finished; fusion uses it to judge freshness.
- `close()`: sets the stop flag, wakes the thread and joins it (max 2 s).

Because `submit` shrinks to exactly 416 px width, a landscape frame has height below 416, so inside `detect` the factor `r` is 1 and no second resize happens (the code checks `(nw, nh) == (w, h)` and copies directly).

A `threading.Condition` is used so the thread sleeps without using CPU while there is no new frame, and wakes the instant one arrives.

Connections

`engine.start_seal_worker()` creates `SealWorker(SealDetector())` if `models/yolox_nano.onnx` exists and onnxruntime can be imported; otherwise YOLOX is off and the app still runs. `engine.py` calls `submit` every frame and passes `seal_worker.result` to `SignFusion.update`.

shinobi/fusion.py

What it is for

Combines the two recognisers into one decision per frame. Its output is "one-hot": a probability vector with 1.0 at the chosen sign (or at idle) and 0 elsewhere. All confidence thresholds are applied here, so the

stabilizer that follows only adds time-based rules, and a sign can never be reported twice by two models.

Who owns which signs

- YOLOX owns the 7 official seals (tiger, snake, horse, ram, monkey, dog, boar).
- The landmark classifier owns `clone_seal` and `rasengan_palm` (`LANDMARK_SIGNS`), for which YOLOX has no classes.
- Backup: if YOLOX is not running at all, the classifier may report an official seal, but only at probability 0.90 or more (`LANDMARK_BACKUP_PROB`).

Helper functions

- `hand_boxes(hands, margin=0.25)`: a pixel box around each hand MediaPipe found (min/max of its 21 points), grown by 25% of its size on every side.
- `overlaps(a, b)`: True if two boxes intersect.
- `open_palm(hands, min_gap=0.60)`: the open-palm gate (explained below).

FusionResult

`probs` (one-hot over the labels), `sign` (the chosen sign or "idle"), `source` ("yolox", "landmarks", "landmarks-backup" or "none"), `score` (the winning model's own score, 0 for idle), `conflict` (both models claimed a sign this frame).

YOLOX seal confirmation

A YOLOX seal is accepted only if all three rules hold:

1. **Fresh.** The result was computed on a frame at most `SEAL_MAX_AGE` = 0.35 s old (`now - result.time`). A stale result resets the confirmation streak. This protects against showing a seal after the user has already moved on, for example when the YOLOX thread falls behind.
2. **On a hand.** `_seal_from` goes through the detections best first and skips classes no jutsu uses. A detection counts if its box overlaps one of MediaPipe's hand boxes and its score is at least `SEAL_MIN_SCORE` = 0.70, or, when MediaPipe sees no hand at all, if its score is at least `SEAL_NO_HAND_SCORE` = 0.85. The second case exists because some seals (Snake, for example) hide the fingers so well that MediaPipe loses the hands; then a stricter score is required since there is no hand to confirm the location. A box far from any visible hand (a poster on the wall, a face) is never accepted while hands are visible.
3. **Twice in a row.** The same seal must win `SEAL_CONFIRM` = 2 consecutive YOLOX results.

The streak logic in `_seal` only updates when a NEW YOLOX result arrives (its `id` differs from `_last_id`), because the main loop runs faster than YOLOX and would otherwise count the same result several times:

```
if result.id != self._last_id:                    # a new YOLOX result: update the streak
    self._last_id = result.id
    seal = self._seal_from(result, hands)
    if seal is None:
        self._streak = (None, 0, 0.0)
    elif seal[0] == self._streak[0]:
        self._streak = (seal[0], self._streak[1] + 1, seal[1])
    else:
        self._streak = (seal[0], 1, seal[1])
sign, count, score = self._streak
return (sign, score) if sign is not None and count >= self.confirm else None
```

Once confirmed, the seal is reported on every frame until the streak breaks (a result without the seal, a different seal, or staleness).

Landmark sign rules: LANDMARK_MIN_PROB, LANDMARK_MIN_MARGIN and the open-palm gate

The classifier's top label `sign` with probability `p` is examined:

- If `sign` is clone_seal or rasengan_palm and `p >= LANDMARK_MIN_PROB` (0.60), it must also pass `_sure`:
- **Margin.** The difference between the highest and second-highest probability must be at least `LANDMARK_MIN_MARGIN` (0.30). For example 0.62 vs 0.35 (margin 0.27) is rejected: the model is torn between two answers.
- **Open-palm gate (rasengan_palm only).** At least one visible hand must show an open palm: in 3D world landmarks, the distance from thumb tip (4) to index tip (8) divided by the hand size (wrist 0 to middle knuckle 9) must be at least `PALM_MIN_THUMB_GAP` (0.60). This was added because the classifier had never seen the OK sign (thumb touching index) and called it a palm. OK signs measure about 0.3-0.5, real palms 0.8-1.2. A hand without usable 3D landmarks (size near zero) fails.
- A landmark sign that fails `_sure` gives NO sign for this frame from the landmarks (and increments `self.rejected`, a counter for evaluation). This matters because an unsure frame then cannot be held, confirmed or advance a jutsu.
- If the probability is below 0.60, the landmark sign is simply ignored.

```
def _sure(self, landmark_probs, sign, hands):
    top2 = np.sort(np.asarray(landmark_probs, np.float32))[-2:]
    if float(top2[1] - top2[0]) < self.landmark_margin:
        return False
    return sign != "rasengan_palm" or open_palm(hands, self.palm_gap)
```

The decision in update()

`update(landmark_probs, landmark_labels, seal_result, hands, now, yolox_running=True)`:

1. If YOLOX is running, the confirmed YOLOX seal (if any) becomes the candidate with source "yolox".
2. If the classifier produced probabilities and its top label is a sure landmark sign: if there is no candidate yet it becomes the candidate ("landmarks"). If YOLOX also has a candidate, this is a **conflict**: `conflict = True`, `self.conflicts` is incremented (shown in the HUD status), and the higher score wins.
3. Backup: if there is still no candidate, YOLOX is NOT running, and the classifier's top label is an official seal (not idle, not a landmark sign) with probability at least 0.90, it becomes the candidate ("landmarks-backup"). Note that when YOLOX is running but simply sees nothing, the classifier can never report an official seal.

4. The one-hot output is built. If there is no candidate (or its name is not in the label list), the output is idle with source "none".

Connections

Created by `engine.py` (only when at least one recogniser exists) and called once per frame. Its `probs` go to `ActionController.update`. Its `sign`, `source`, `score` and `conflicts` form the status text.

shinobi/controller.py

What it is for

Turns the noisy per-frame decisions into confirmed signs and jutsu activations:

```
probabilities -> GestureStabilizer -> confirmed sign events
              -> SequenceMatcher -> jutsu name -> ActionController (cooldown) -> effect name
```

It contains three classes and one data class. It does not know about cameras, models or drawing, which makes it easy to test (`tests/test_sequences.py` and others test it with made-up inputs).

class GestureStabilizer

Goal. Even after fusion, single frames can flicker (one frame "tiger", one frame idle, one frame "tiger"). The stabilizer only confirms a sign after it has been strong for a while, reports it ONCE, and does not report it again until it has clearly faded.

State. `smoothed` (smoothed probability per label, starts as 100% idle), `candidate` (the sign currently being held) and `candidate_since` (when it became candidate), `latched` (the sign already confirmed and not yet released).

Time-correct EMA smoothing. An exponential moving average (EMA) mixes the new frame into the running value: `smoothed = w x new + (1 - w) x smoothed`. With a fixed weight per frame, the speed of smoothing would depend on the frame rate: at 15 FPS a sign needs 3 frames (0.2 s) to rise above 0.75, but at 6 FPS the same 3 frames take 0.5 s, and short signs (about 1 s) ended before they were confirmed. So the weight is computed from the real time between frames:

```
def weight(self, now):
    dt = self.ref_dt if self.last_time is None else min(max(now - self.last_time, 1e-3), 1.0)
    self.last_time = now
    return 1.0 - (1.0 - self.alpha) ** (dt / self.ref_dt)
```

`alpha` = 0.4 is the weight at the reference rate of 15 FPS (`ref_dt` = 1/15 s). If two frames are twice as far apart, the "keep" part (1 - 0.4) is applied twice: (0.6)^2, so the new weight is 1 - 0.36 = 0.64. At exactly 15 FPS the formula gives exactly 0.4, the old per-frame behaviour. `dt` is clamped between 1 ms and 1 s.

Example at 15 FPS, fusion switching from idle to "tiger" (one-hot): the smoothed tiger value goes 0.40, 0.64, 0.784; at the third frame it passes `CONF_ON` (0.75) while idle has dropped to 0.216.

update(probs, now), step by step:

1. `probs is None` (no hands, or the engine is not ready) is treated as 100% idle.
2. Apply the EMA with the time-based weight.

3. **Hysteresis release.** If a sign is latched and its smoothed value fell below `CONF_OFF` (0.45), the latch is released. Using a lower "off" threshold than the "on" threshold avoids flicker around a single threshold.
4. Find the top and second labels. The top is **strong** if it is not idle, its smoothed value is at least `CONF_ON` (0.75), and it leads the second by at least `MIN_MARGIN` (0.25).
5. Not strong: the candidate is cleared, return None.
6. Strong but a different sign than the current candidate: it becomes the new candidate with `candidate_since = now`, return None.
7. Strong, same candidate, not already latched, and held for `HOLD_SECONDS` (0.30 s): latch it and return its name. This is the only moment a confirmed sign is reported.

While the user keeps holding the sign, it stays latched and nothing more is returned. After it fades below 0.45 and comes back strong for 0.3 s, it is confirmed again; `SequenceMatcher` treats that as a repeat (see below).

`hold_progress(now)` returns 0..1, how far the current candidate is towards confirmation (0 if none or already latched). The desktop HUD draws it as a progress bar.

_in_order(small, big)

Returns True if every item of `small` appears in `big` in the same order, with gaps allowed. It uses a neat Python trick: `item in rest` on an iterator consumes the iterator up to the found item, so each next search continues after the previous match.

class SequenceMatcher

Goal. Follow all jutsu sequences at once and say when one is complete.

State, per jutsu. `steps[name]` (steps done, 0..3), `last_step[name]` (time of its last step, or of the leader's last wrong attempt, see below), `first_step[name]` (time of the first step of its current run). Shared: `history` (recent confirmed signs for the desktop HUD, at most 4), `last_sign` (for repeats), `repeated` (the last push was a repeat), `advanced` (jutsu that moved forward on the last push).

Start-up check: _check_conflicts(). Refuses configurations that could never work, by raising `ValueError`:

- A sequence with the same sign twice in a row (a held sign counts once, so it could never be performed).
- Two jutsu with exactly the same sequence.
- A jutsu whose whole sequence appears in order inside another jutsu's sequence before that one's last sign (checked with `_in_order(inner, seq[:-1])`). The inner jutsu would fire on the way and reset everything, so the outer one could never be completed.

The core rules of push(gesture, now). `push` receives each confirmed sign:

1. **Expire first.** `expire(now)` resets every jutsu whose last step is older than `max_gap` (7 s in free mode). It returns `{jutsu: steps it had}` sorted furthest-first, and clears the history when nothing is in progress any more.
2. **Who advances.** `nexts` = every jutsu whose NEXT sign is this gesture. A sign advances every such jutsu at the same time, so several jutsu can be in progress at once (for example "tiger" is a step of three sequences).
3. **Repeat.** If the gesture equals `last_sign` and it is no jutsu's next sign, it is a repeat: nothing changes (no step, and the clock is NOT refreshed); only the history entry's time is updated. Repeats happen naturally: a held seal is re-confirmed whenever the recogniser drops out for a moment (one missed

YOLOX result is enough), and "ram, snake, snake, tiger" must still count as "ram, snake, tiger". The same sign does count as a new step if it is some jutsu's next sign (for example after a timeout or a retry, when that jutsu starts over).

4. **Wrong signs never reset.** A sign that is not a jutsu's next sign leaves that jutsu exactly where it is. The user simply tries the step again. Only time (or completing a jutsu) resets progress.
5. **Advance.** For every jutsu in `nexts`: record `first_step` if it was at 0, add one step, set `last_step = now`. If it reaches its full length, it is "done".
6. **Wrong attempt restarts the leaders' clock** (rule added 2026-11-01, see below).
7. **Fire.** If one or more jutsu are done, the longest sequence wins (ties go by the order in `config.JUTSU`; `max` keeps the first of equal values). Then ALL progress is reset, so one performance can never fire twice or count as the start of another jutsu. `last_sign` is kept after the reset, so continuing to hold the final sign is a repeat, not the first step of something new.

The 2026-11-01 rule: a wrong attempt restarts the leaders' 7 s clock. Before this rule, a user who made a mistake at step 3 and needed a few seconds to find the right sign could run out of time even though they were actively trying. Now:

```
prog = self.progress(now)
top = max(prog.values())
leaders = [name for name in self.jutsu if top and prog[name] == top]   # furthest along, before this
sign
...
if leaders and not set(leaders) & set(nexts) and all(self.steps[n] < top for n in nexts):
    for name in leaders:        # a wrong attempt at their next step: max_gap again to make it
        self.last_step[name] = now
```

- The **leaders** are all jutsu furthest along before this sign (the most steps, at least 1).
- If the new sign advances none of the leaders, it was a wrong attempt at their next step, and every leader gets the full `max_gap` again from now.
- All leaders are refreshed, not just one, so a leftover run that happens to be level with the real one cannot take the restart away from it.
- No exception (updated 2026-11-01 at the owner's request): an earlier version did not refresh the clock when the wrong sign brought another jutsu level with the leaders. That caused resets when a wrong sign happened to be the first sign of another jutsu, so the condition was removed. Now every wrong attempt gives the step the full time again (the code is `if leaders and not set(leaders) & set(nexts):`).
- A repeat never counts as an attempt (it returns before reaching this code), so holding a wrong sign does not keep extending the time. Only stopping (7 s with no new sign) costs the steps done.

Example (free mode): at t = 0 the user makes Ram, so shadow_clone has 1 step. At t = 5 they make Dog by mistake. Dog is nobody's next sign, so nothing advances, but shadow_clone is the leader and was not advanced: its clock restarts, and the user now has until t = 12 (not t = 7) to make Snake. If instead at t = 5 they had made Monkey, fire_style would go to 1 step, and shadow_clone (the leader, not advanced) also gets a new clock until t = 12; the card stays on shadow_clone because a tie goes to the jutsu started first.

progress(now=None) returns `{jutsu: steps}`, counting a jutsu whose time has run out at `now` as 0. It never shows the full length, because a complete sequence fires and resets.

leader(now=None) returns the jutsu furthest along: most steps; on a tie, the one in progress the longest (oldest `first_step`), then config order. The tie rule means a wrong sign that happens to start another jutsu never takes the lead from the sequence the user is actually performing.

time_left(now) returns the seconds the leader has left for its next sign (`max_gap - (now - last_step)`, at least 0), or None if nothing is in progress or in practice mode. **time_left_all(now)** returns the same for every jutsu in progress, each on its own clock (empty in practice mode).

Practice mode. `max_gap=None` disables every time limit (`_expired` is always False). The web app uses this when the user practises one chosen jutsu.

reset() forgets every sequence in progress (Retry / Reset buttons, mode change).

ControllerState

A data class with what happened in one frame: `confirmed` (sign confirmed this frame), `repeat`, `jutsu` (jutsu fired), `effect` (effect to start), `history`, `progress`, `timed_out` (`{jutsu: steps}` that ran out of time this frame), `timeout` (the jutsu that was furthest along, if it is one of those that timed out; reported to the user), `leader` (furthest along before this frame's sign; wrong signs are reported for it).

class ActionController

Ties the stabilizer and the matcher together and adds the cooldown.

- `__init__(labels, cooldown=1.5, jutsu=config.JUTSU, max_gap=7.0)`: creates a `GestureStabilizer` and a `SequenceMatcher`. `max_gap=None` = practice mode.
- `update(probs, now, seq_now=None)`: one frame. `now` is the wall clock, used for the stabilizer's hold time and the cooldown. `seq_now` is the clock for the sequence time limit (the engine passes its "sequence clock", see engine.py); by default the same as `now`.

```
furthest = self.matcher.leader()
state.timed_out = self.matcher.expire(t)
state.timeout = furthest if furthest in state.timed_out else None
state.leader = self.matcher.leader()
state.confirmed = self.stabilizer.update(probs, now)
if state.confirmed is not None:
    state.jutsu = self.matcher.push(state.confirmed, t)
    state.repeat = self.matcher.repeated
    if state.jutsu is not None:
        if now - self.last_fire >= self.cooldown:
            self.last_fire = now
            state.effect = self.jutsu[state.jutsu]["effect"]
        else:
            state.jutsu = None   # still cooling down: ignore
```

The order matters: the leader is noted BEFORE expiring, so a timeout can be reported for the sequence the user was performing. Expiry runs every frame (not only when a sign arrives), so the "sequence failed" message appears as soon as the time is up. If a jutsu completes during the 1.5 s cooldown, the matcher has already reset its progress but no effect starts.

- `cooldown_left(now)`: seconds until another jutsu may fire.

Connections

`engine.py` creates the `ActionController` and calls `update` every frame. `main.py` reads the stabilizer for the HUD. The `ControllerState` fields go into `events.py` through the server.

shinobi/engine.py

What it is for

The complete SHINOBI AI pipeline for one camera stream, without window, keyboard or HUD. The desktop app creates one `Engine`; the web server creates one per browser session, so two users never share recognition state. An Engine is not thread-safe: all its methods must be called from one thread (MediaPipe objects must not be used by two threads at once; the segmenter only runs on its worker thread and that job is always finished before `process` returns).

start_seal_worker()

Returns a running `SealWorker(SealDetector())`, or None with a printed reason when `models/yolox_nano.onnx` is missing or onnxruntime cannot be imported. The import is inside the function so the app still runs without onnxruntime installed.

EngineResult

Everything one processed frame produced: `frame` (output image with the effect, plus landmarks if enabled), `now`, `dt` (seconds since the previous frame, clamped to 0.1), `fps` (smoothed), `timings` (ms per stage: hands, model, yolox, seg, fx), `hands`, `tracking`, `probs` (classifier probabilities), `fused` (FusionResult), `state` (ControllerState), `stabilizer`, `status` (a text like "sign: tiger [yolox 0.93] conflicts 0"), `history`, `progress`, `time_left`, `time_left_per_jutsu`, `ready`, `cooldown`, and about the effect: `effect`, `effect_id` (activation number), `effect_jutsu`, `effect_phase` ("voice", "charge", "name" or "release") and `effect_phase_left`.

Engine.__init__(classifier=AUTO, yolox=True, jutsu=None, draw_landmarks=False)

Creates the `HandTracker`, `PersonSegmenter`, a one-thread `ThreadPoolExecutor` for segmentation, an `AnchorTracker` and the `EffectManager`. Then the recognisers:

- `classifier=AUTO` loads `GestureClassifier()` if the model file exists, else None. The server can pass an already loaded classifier to share it, or None to turn it off.
- `yolox=True` starts this engine's own YOLOX worker.
- `SignFusion` and `ActionController` are created only if at least one recogniser exists. Without any, the app runs in "keyboard mode" (effects only by keys).

It also initializes the readiness state (`ready = False`), the sequence clock (`seq_clock = 0.0`), and bookkeeping for activations and frame size.

warm_up(timeout=15.0)

The first run of MediaPipe, YOLOX and the segmenter is slow (measured 1.8 s for the first frame of a session), and signs made meanwhile would be missed. `warm_up` runs every model once on a grey 640 x 480 image before the first camera frame: hand detection twice, one YOLOX run (waiting up to 15 s for its result, then clearing it), one segmentation (then resetting the segmenter's memory). Then it creates a fresh `SignFusion` and `ActionController` (keeping the current practice / free time limit) and a fresh `AnchorTracker`, so no state from the blank frames remains. The engine is still not `ready` after this; readiness needs real frames.

Control methods

- `start_effect(name, now, jutsu=None, staged=False)`: starts an effect, replacing the running one. `staged=True` (a jutsu fired by its sequence) adds the build-up stages from `activation_stages(jutsu)`; by key press the main effect starts immediately. Increments the activation number.
- `stop_effect(now)`: fades out the running effect (desktop key 0).
- `set_jutsu(jutsu, practice=False)`: chooses which jutsu can fire (all = free mode; one = practice). Creates a new `ActionController` with `max_gap=None` in practice mode, `SEQUENCE_MAX_GAP` otherwise. The running effect keeps running.
- `reset_sequence()`: forgets the sequences in progress; cooldown and effect are kept.
- `_new_frame_size(size)`: called when the frame size changes (a browser switched camera or turned to portrait). Old pixel positions are meaningless, so the anchor tracker and segmenter are reset, and a running Shadow Clone ends (its layer has the old size); other effects continue.

process(frame, now): one frame

```
dt = min(now - self.last_time, 0.1)                # clamp: a hiccup must not explode particles
if self._seq_last is not None:
    self.seq_clock += min(max(now - self._seq_last, 0.0), config.SEQUENCE_CLOCK_MAX_STEP)
self._seq_last = now
self.last_time = now
self.fps = 0.9 * self.fps + 0.1 / max(dt, 1e-3)
```

The sequence clock. The 7 s limit should count only time the AI could actually see the user. If the server or browser stalls for 3 seconds, that time should not be lost. So the engine keeps its own clock, `seq_clock`, which advances by the real time between processed frames but never more than `SEQUENCE_CLOCK_MAX_STEP` (0.4 s) per frame. At 2.5 or more processed frames per second it equals the wall clock; during a stall it nearly stops. This clock is passed as `seq_now` to the controller and used for `time_left`. The stabilizer's hold time and the cooldown still use the real time `now`.

`dt` is clamped to 0.1 s for the effects, so a hiccup does not make particles jump. The FPS is an EMA (90% old, 10% new).

Then the four stages:

1. **Perception.** Submit the frame to YOLOX. If the active effect needs segmentation, submit it to the segmentation worker (so it runs in parallel with hand tracking on another CPU core). Run `hands_tracker.detect(frame)`.
2. **Recognition.** If a classifier exists and hands are visible: `probs = classifier.predict(extract(*hands_to_arrays(hands), aspect=w / h))`. Then `fusion.update(...)` with the latest YOLOX result. Then the controller:

```
# not ready yet: the frame is processed (models, smoothing) but counts as no sign
state = self.controller.update(fused.probs if self.ready else None, now, seq_now=self.seq_clock)
if state.effect:
    self.start_effect(state.effect, now, jutsu=state.jutsu, staged=True)
```

1. **Effects.** `anchor.update(hands, now)` gives the smoothed hand position; the segmentation result is collected (`seg_job.result()` waits if it is not done yet); `effects.render(frame, tracking, dt, segmentation)` draws the effect. When no effect is active, the segmenter's smoothing memory is reset.
2. **Overlay.** Hand landmarks are drawn if `draw_landmarks` is on.

Finally the `EngineResult` is filled: controller values (time left using `seq_clock`, history, progress, cooldown) and effect values. `effect_id` and `effect_jutsu` are only filled if the active effect is the same object that the last `start_effect` created.

The readiness gate. At the end of each frame, while not yet ready:

```
took = (time.perf_counter() - frame_start) * 1000
self._ready_run = self._ready_run + 1 if took < config.READY_MAX_MS else 0
self.ready = self._ready_run >= config.READY_FRAMES
```

The engine becomes `ready` after `READY_FRAMES` (5) frames in a row each processed in under `READY_MAX_MS` (500 ms); a slow frame restarts the count. Until then the controller receives `None` (treated as idle), so no sign can be confirmed, no step can start and no time limit can run while the camera and models are still getting up to speed. Once ready, the engine stays ready. The web app shows the `ready` flag to the user.

close()

Stops the YOLOX worker and the segmentation thread pool, closes MediaPipe objects, and drops the active effect (a Shadow Clone's worker thread ends with it). Safe to call twice.

Connections

Imports from `classifier`, `controller`, `effects`, `features`, `fusion`, `hands`, `segmentation`, `tracking`. Used by `main.py` (desktop) and `server.py` (web).

shinobi/events.py

What it is for

Turns what the engine reports frame after frame into the events of the web protocol (`docs/web_protocol.md`, section 3.6). It is pure logic: no models, no images. The server gives an `EventTracker` one `FrameView` per processed frame and sends the returned list of events to the browser, which plays sounds and animations only on these events. The sequence rules themselves live in `SequenceMatcher`; this class only reports what happened.

Data classes

- `EffectView` (frozen): `key` (activation number; a new jutsu gets a new key even for the same effect), `name`, `stopping` (fading out), `clones` (Shadow Clone: clones visible), `jutsu`, `phase`, `staged` (has a build-up).
- `FrameView` (frozen): `now`, `confirmed`, `repeat`, `fired`, `fired_effect`, `progress` (after this frame), `effect` (an EffectView or None), `timed_out`, `timeout`, `leader`, `fired_stages` (stage lengths of the activation that fired).

`PHASE_ORDER = ("voice", "charge", "name", "release")`.

The events, in the order they are listed within one frame

Event	When
`sequence_failed` (reason "timeout")	Free mode only: the jutsu furthest along ran out of time and went back to step 0. Contains the jutsu, the step reached and the expected sign.
`sign`	The stabilizer confirmed a sign. A repeat gets only this event.
`wrong_sign`	The sign is not the next one of the sequence in progress. Free mode: only for the leader (steps >= 1). Practice mode: for the practised jutsu whenever the sign is not its next one, also at step 0. Not sent if the sign fired a jutsu.
`step`	One per jutsu whose steps went up with that sign, except the jutsu that fired (the full sequence resets everything). Contains step number, total and sign.
`effect_end`	The effect of the previous frame is gone (replaced or faded out).
`jutsu`	A full sequence fired; includes `stages` (voice, charge, name seconds) when there is a build-up.
`effect_phase`	The activation entered "charge", "name" or "release" (each once, in order; a stage that a slow frame skipped is still reported).
`effect_stopping`	The effect started fading out (always sent before its `effect_end`).
`clones`	Shadow Clone: more clones are visible than ever before in this activation.

class EventTracker

- `__init__(jutsu, practice=None)` and `set_mode(jutsu, practice)`: store which jutsu can fire and the practised jutsu id (None = free mode), and reset progress.
- `reset()`: forgets the remembered progress (Retry / Reset).
- `_leader(view, before)`: uses the controller's leader if it had at least one step; without it (older callers) the jutsu with most steps, config order on a tie.
- `_timeout(view)`: builds the `sequence_failed` event from `view.timeout` and `view.timed_out` (never in practice mode, never for 0 steps).
- `_wrong_sign(view, before)`: compares the confirmed sign with the expected next sign of the leader (or the practised jutsu).
- `_effect_events(view)`: compares the previous frame's `EffectView` with the new one (by `key`) to produce effect_end, jutsu, effect_phase, effect_stopping and clones. If an effect disappears without having been seen fading, it sends effect_stopping first so the browser always gets both.
- `update(view)`: the main function. It computes `before` (steps before this frame's sign: the previous progress, minus jutsu that just timed out), then appends events in the order of the table. A repeat produces only `sign`.

Connections

Imported only by `server.py`, which builds `FrameView` objects from `EngineResult` and `ControllerState`.

shinobi/filters.py

What it is for

The One Euro filter (Casiez et al., CHI 2012, https://gery.casiez.net/1euro/). It smooths jittery positions without adding visible lag. MediaPipe's hand positions shake by a few pixels even when the hand is still; effects drawn at the raw position would tremble.

The idea

A simple low-pass filter has one problem: strong smoothing kills jitter but makes the effect lag behind fast movements; light smoothing follows fast movements but lets jitter through. The One Euro filter changes the amount of smoothing with the speed: slow movement means strong smoothing, fast movement means light smoothing.

Code

`_alpha(cutoff, dt)` converts a cutoff frequency (Hz) and the time step into a smoothing weight: `tau = 1 / (2 pi cutoff)`, `alpha = 1 / (1 + tau / dt)`. A higher cutoff gives a higher alpha, meaning more weight on the new value (less smoothing).

`OneEuroFilter(min_cutoff=1.5, beta=0.02, d_cutoff=1.0)`; calling it with `(value, now)`:

```
speed = (value - self.x) / dt
a_d = _alpha(self.d_cutoff, dt)
self.dx = a_d * speed + (1 - a_d) * self.dx

cutoff = self.min_cutoff + self.beta * float(np.linalg.norm(self.dx))
a = _alpha(cutoff, dt)
self.x = a * value + (1 - a) * self.x
```

1. Estimate the speed and smooth it (with a fixed 1 Hz cutoff).
2. The cutoff grows with speed: `min_cutoff + beta x |speed|`.
3. Smooth the value with that cutoff.

The first call just stores the value. Works on any numpy shape (a 2D point, a size, a direction). `reset()` forgets everything, so the next value starts fresh.

Connections

Used by `tracking.py` (three filters per tracker). Reads `FILTER_MIN_CUTOFF` and `FILTER_BETA`.

shinobi/tracking.py

What it is for

Picks the hand that the effects should follow (the "anchor") and smooths its position, size and pointing direction. Effects like the Rasengan, Chidori and Fire Style are drawn at this anchor.

Tracking (data class)

`hands` (all hands this frame), `now`, `visible` (anchor hand seen now, or lost for less than the grace period), `anchor` (smoothed palm centre in pixels), `size` (smoothed hand size in pixels), `direction` (unit vector from wrist to middle knuckle: where the hand points).

class AnchorTracker

- `__init__(grace=0.6)`: three One Euro filters: position (config defaults), size (`min_cutoff=0.8, beta=0`: steady), direction (`min_cutoff=1.0, beta=0`).
- `_pick(hands)`: stay on the hand closest to the previous anchor, if it is within 3 x the previous hand size (at least 40 px), so the effect does not jump between hands. Otherwise take the biggest hand (`size_px`), which is usually the closest to the camera.
- `update(hands, now)`: with hands, it picks one, computes the wrist-to-middle-knuckle direction, normalizes it, filters it (and normalizes again because filtering changes the length), filters the palm centre and the size, and records `last_seen`. Without hands, it keeps the previous anchor, size and direction, and `visible` stays True for `HAND_LOST_GRACE` (0.6 s) seconds; after that all filters are reset so the next hand starts fresh instead of sliding from the old place.

Connections

`engine.py` calls `anchor.update` every frame and passes the `Tracking` to the effects.

shinobi/segmentation.py

What it is for

Separates the person from the background with MediaPipe's selfie segmenter. Only the Shadow Clone effect needs it (to cut the person out of the image and paste copies). The engine runs it only while that effect is active, on a worker thread.

class PersonSegmenter

- `__init__(work_width=640, smoothing=0.5)`: creates the MediaPipe `ImageSegmenter` in VIDEO mode, asking for a confidence mask (a value 0..1 per pixel).
- `_timestamp()`: strictly increasing millisecond timestamps, same trick as `HandTracker`.
- `segment(frame_bgr)`: returns `(small_frame, mask)`, both at 640 px width.

The steps of `segment`:

1. Resize the frame to 640 px width (speed) and convert to RGB; run the segmenter; copy the mask.
2. **Temporal EMA.** `mask = 0.5 x new + 0.5 x previous` (only if the previous mask has the same shape). Removes flicker between frames.
3. **Guided filter.** `cv2.ximgproc.guidedFilter(gray, mask, 6, 1e-3)` snaps the mask edges to real edges in the image (hair, shoulders). A grey guide is about twice as fast as a colour one. The guide must be scaled to 0..1: with 0..255 values, the local variance of flat areas lost precision in float32 and the mask filled with NaN (measured on 82 of 180 recorded frames; 0 with the 0..1 guide).
4. **Soft contrast curve.** `clip((mask - 0.3) / 0.4, 0, 1)`: below 0.3 becomes 0, above 0.7 becomes 1, with a soft edge in between.

- `reset()`: forgets the previous mask (called when no effect is active or the frame size changes).
- `close()`: releases MediaPipe.

Connections

Created by `engine.py`, used by `effects/shadow_clone.py` through the engine.

shinobi/hud.py

What it is for

Draws debug information on the desktop window: FPS, timings, the smoothed probability of every label, the hold progress, the seal history, the active effect, the cooldown and messages. Not used by the web app (the React page draws its own interface).

text(frame, msg, org, scale, color, thick)

Draws text with a dark outline so it is readable on any background. In OpenCV 5 the `thickness` of `putText` only chooses regular or bold letters and no longer widens the stroke, so the old trick (black thick text under coloured text) stopped making an outline. The new method:

- `_rendered(msg, scale, thick)` (cached with `lru_cache`, because most HUD strings repeat every frame) draws the text into a mask, dilates the mask by 2-3 pixels (the outline), and returns the text mask, an inverted "keep" mask, padding and height.
- `text` clips the area at the frame border, darkens the image under text and outline (`multiply` by the keep mask), then adds the colour through the anti-aliased text mask, all with 8-bit OpenCV operations.

draw_hud(frame, fps, timings, labels, probs, candidate, latched, hold, history, effect, cooldown, message)

- Top-left: FPS (green) and per-stage timings in ms.
- A bar chart of the smoothed probabilities: the latched sign in yellow, the candidate in green, others grey; under it the hold progress bar.
- Bottom: "seals: ram > snake > ..." (the history), a message line, and the key help ("1-4 effects 0 stop L landmarks H hud F fullscreen Q quit").
- Top centre: the active effect's name in capitals; top right: the cooldown.

Connections

Only `main.py` calls `draw_hud`.

shinobi/main.py

What it is for

The desktop app: camera, window, keyboard and HUD around the `Engine`. Run with `python -m shinobi.main`. (The project's final target is the web app; the OpenCV window is kept as a development and debugging tool.)

main()

1. Creates the `Camera` and an `Engine(draw_landmarks=True)`. Sets a start message depending on what loaded: nothing if the landmark model exists; "No landmark model yet: only the 7 official seals (YOLOX) + keys 1-4" if only YOLOX; "No trained model yet: keyboard mode (press 1-4)" if neither.
2. Opens a resizable window of 1280 x 720.
3. The loop:

- **Keyboard on every pass.** `cv2.waitKey(1)` is checked even when no new frame has arrived, otherwise key presses in between would be lost. Q or Esc quits; 1-4 start an effect directly (`EFFECT_KEYS`); 0 stops it; L toggles landmarks; H toggles the HUD; F toggles fullscreen. Closing the window with the X button also ends the loop.
- **New frame only.** `camera.read()` gives `(frame_id, frame)` ; if the id equals the last processed one, the loop continues without processing, so the same frame is never processed twice.
- **Process.** `r = engine.process(frame, time.perf_counter())` .
- **HUD.** If shown, `draw_hud` gets the stabilizer's labels, smoothed probabilities, candidate, latched sign and hold progress, the history, the effect name (with its build-up stage in brackets, for example "rasengan (charge)"), the cooldown, and a message combining the start message, the engine status and, for Shadow Clone, a "not enough room ... step back" warning when the effect reports it is crowded.
- `cv2.imshow` shows the frame.

1. `finally` : always closes the engine, releases the camera and destroys the windows, even after an error.

Connections

Uses `camera.Camera` , `engine.Engine` , `hud.draw_hud` and config. It is the only file that uses the camera and the HUD.

shinobi/catalog.py

What it is for

Display texts for signs and jutsu, the credits, and the voice-clip helpers. Pure data, no images or models. One copy used by two programs: `tools/build_gesture_references.py` (draws the reference cards in `gesture_references/`) and `server.py` (sends them to the web app through `/api/jutsu` and `/api/credits`). Changing a text here and re-running the tool keeps the cards and the web app consistent. The ORDER of the steps is not stored here; it comes from `config.JUTSU` .

Contents

- `COMMONS_CREDIT` , `PALM_CREDIT` : credit lines printed on the cards.
- `SIGNS` : for each of the 9 signs used by the sequences: display `name` , Japanese name `jp` , number of `hands` (2, except `rasengan_palm` with 1), `official` , and `cues` (short instructions shown on the step card). For example `rasengan_palm` : "ONE hand, palm facing the camera", "Fingers straight and spread apart", "In front of your chest / shoulder", "Other hand down, out of the frame".
- `JUTSU` : for each jutsu: card `title` , header `color` , `tag` , `status` (where the sequence comes from; for example Fire Style uses the last 4 of the official 6 seals Snake, Ram, Monkey, Boar, Horse, Tiger, and

Chidori the last 4 of the 9-seal anime version), and for the web app `name`, `kanji` and `description`. The descriptions insert durations from `config.EFFECT_DURATION` and the clone count from `config.CLONE_COUNT`, so they stay correct if those change.

- `CREDITS`: a list of sections (hand-seal illustrations, sound effects, music, the YOLOX model, disclaimer), each with items built by `_item(text, url, license, license_url)`. The constants `CC_BY_SA_4`, `CC_BY_4`, `CC_BY_3`, `CC0` hold licence URLs. The YOLOX item notes that the weights have no explicit licence, so the model is used for personal and educational purposes only.

Voice clips

No voice clips are bundled; the user may place licensed recordings at `assets/audio/voice/<jutsu>/activation.wav` and `name.wav` (PCM WAV only).

- `VOICE_PARTS = ("activation", "name")`, `VOICE_FORMATS = (".wav",)`, `VOICE_SPEAKER` (Naruto for fire_style, shadow_clone, rasengan; Sasuke for chidori).
- `wav_seconds(path)`: length of a PCM WAV file in seconds using Python's `wave` module, or None if missing or unreadable.
- `voice_seconds(jutsu, audio_dir=None)`: `{part: seconds or None}` for one jutsu. `effects/__init__.py` (`activation_stages`) uses it so that the "voice" and "name" build-up stages last exactly as long as the clip plus `VOICE_GAP`, or `VOICE_DEFAULT` without a clip.
- `voice_clips(audio_dir=None)`: for every jutsu, the speaker, the URL of each installed clip (or None) and its length; sent to the web app so it plays only clips that exist.

Connections

Read by `server.py`, `effects/__init__.py` and `tools/build_gesture_references.py`. Reads `config.EFFECT_DURATION`, `config.CLONE_COUNT` and `config.AUDIO_DIR`.

Summary: how a sign becomes a jutsu (worked example)

Suppose the user performs Fire Style (Monkey, Boar, Horse, Tiger) in free mode, at about 15 processed frames per second, after the engine has become ready.

1. The user forms Monkey. MediaPipe finds two hands; YOLOX (10 times per second) returns "monkey" with score 0.91, its box overlapping a hand box. The first such result starts a streak of 1; the second consecutive one confirms it. From then on, fusion outputs one-hot "monkey" (source "yolox") every frame.
2. The stabilizer's smoothed monkey value climbs 0.40, 0.64, 0.78. At 0.78 it is strong (above 0.75, margin above 0.25) and becomes the candidate. After 0.30 s it is latched and "monkey" is confirmed once.
3. `SequenceMatcher.push("monkey")`: monkey is the next sign of fire_style (step 0) only, so fire_style goes to 1 step and its 7 s clock starts. The web app receives `sign` and `step` events.
4. The user changes hands. While switching, fusion outputs idle; the monkey value drops below 0.45 and the latch is released.
5. Boar and Horse follow the same way: fire_style goes to 2, then 3 steps, each step restarting its 7 s clock. If the user had made Dog by mistake at step 3, nothing would be reset, a `wrong_sign` event would be sent, and fire_style's clock would restart, giving 7 more seconds to make Horse.
6. Tiger is confirmed: fire_style reaches 4 steps and is done. Since it is the only completed jutsu, it fires; all progress resets. The cooldown has passed, so `state.effect = "fire"`.

7. The engine calls `start_effect("fire", now, jutsu="fire_style", staged=True)`: the effect builds up (voice, charge, name) and then releases the flames at the smoothed hand anchor for 6 seconds.

Part 2 — Server, protocol and effects

This part explains the code that sits between the browser and the AI, and the code that draws the jutsu effects on the camera picture. It covers three areas:

- `shinobi/server.py`: the FastAPI web server, its REST endpoints, the WebSocket session that processes camera frames, and the binary message format used in both directions.
- `shinobi/effects/`: the five Python files that draw Fire Style, Chidori, Rasengan and Shadow Clone onto each frame, plus the shared base class and drawing helpers.
- The asset folders `assets/audio/` (all sounds, their licences and when the web app plays them) and `gesture_references/` (the hand-sign pictures shown in the app).

A short reminder of the big picture before we start. The browser (React app) opens the camera, compresses each camera frame to JPEG and sends it to the Python server over a WebSocket. The server runs the AI pipeline (`shinobi/engine.py`: MediaPipe hands, the YOLOX seal detector, the landmark classifier, sign fusion, the sequence controller) and the effect code on the frame, compresses the result back to JPEG and returns it together with a JSON description of what happened (the `FrameState`). The browser only displays the returned picture and plays sounds when the server reports events. The browser never decides on its own that a sign or a jutsu happened.

shinobi/server.py

Purpose

`server.py` is the entry point of the web application. You start it with:

```
python -m shinobi.server [--host 127.0.0.1] [--port 8000] [--max-sessions 2]
                         [--ssl-certfile cert.pem --ssl-keyfile key.pem]
```

It does four jobs:

1. Serves a small REST API (`/api/info`, `/api/jutsu`, `/api/credits`, `/api/voices`) that tells the web app what this server can do and gives it the texts and image URLs for the jutsu cards.
2. Accepts WebSocket connections on `/ws`. Each connection is one independent "session" with its own AI engine.
3. Serves read-only static files: the hand-sign pictures under `/references/...` and the sounds under `/audio/...`.
4. Serves the built React app (`frontend/dist`) for every other URL.

The contract between this file and the frontend is written down in `docs/web_protocol.md` (protocol version 1). The Arabic guide `docs/13_web_server.md` explains how to run it, HTTPS for phones, and common problems. Everything described below is what the code really does.

Imports and module-level setup

The file imports FastAPI (`FastAPI`, `WebSocket`, `HTTPException`, `FileResponse`, `HTMLResponse`, `StaticFiles`), OpenCV (`cv2`) and NumPy for image decoding and encoding, and four project modules:

- `shinobi.config`: all constants (ports, frame limits, JPEG qualities, folder paths, jutsu definitions).

- `shinobi.catalog`: the texts of the sign and jutsu cards, the credits, and the voice-clip helper.
- `shinobi.classifier.GestureClassifier`: the trained landmark model (optional).
- `shinobi.engine.Engine`: the complete AI pipeline for one session.
- `shinobi.events`: `EventTracker`, `FrameView`, `EffectView`, which turn the engine's state into the list of events sent to the browser.

Right after the imports there is a small but important Windows fix. Windows reads MIME types from the registry, and on some machines `.js` is registered as `text/plain`. Browsers refuse to run JavaScript modules served with that type, so the web app would be blank. The server therefore registers the correct types itself:

```
for _type, _ext in (("text/javascript", ".js"), ("text/javascript", ".mjs"), ("text/css", ".css"),
                    ("image/svg+xml", ".svg"), ("application/wasm", ".wasm"), ("application/json",
".json"),
                    ("application/manifest+json", ".webmanifest"), ("audio/wav", ".wav"),
("audio/flac", ".flac")):
    mimetypes.add_type(_type, _ext)
```

Other module-level constants:

name	value / meaning
`YOLOX_SIGNS`	the signs in `config.GESTURES` that the YOLOX seal detector can recognise (those that appear in `config.SEAL_CLASSES`)
`HEADER_LEN`	`struct.Struct(">I")`: an unsigned 32-bit big-endian integer, the length prefix of every binary message
`OUTPUT_JPEG`	the OpenCV encode options for frames sent back: quality `config.OUTPUT_JPEG_QUALITY` (90) and, when `config.OUTPUT_JPEG_444` is true, full colour resolution (4:4:4 chroma sampling) so thin coloured lines such as Chidori bolts stay sharp
`BUILD_HELP`	a small HTML page shown at `/` when `frontend/dist` does not exist yet; it explains `npm install` / `npm run build` or `npm run dev`

Binary framing: pack() and unpack()

Both directions of the WebSocket use the same binary layout for image messages:

```
[ uint32 big-endian N ][ N bytes: UTF-8 JSON header ][ JPEG bytes ]
```

The first 4 bytes say how long the JSON header is. Then comes the header (a JSON object), then the rest of the message is the JPEG picture. This lets one WebSocket message carry both structured data and an image without base64 encoding (which would make images about 33 % bigger).

```
def pack(header, payload=b""):
    head = json.dumps(header, separators=(",", ":")).encode("utf-8")
    return HEADER_LEN.pack(len(head)) + head + payload
```

`unpack(data)` does the reverse and raises `ValueError` for every malformed case: the message is shorter than 4 bytes, the announced header length is bigger than the message, the header is not valid UTF-8 JSON, or the JSON is not an object. It returns `(header_dict, payload_bytes)`.

Two tiny helpers: `_number(value)` returns a JSON number as `float` (and rejects booleans, because in Python `True` is also an `int`), else `None`. `_clean(text)` collapses runs of spaces to one; the card texts in

`catalog.py` use double spaces as visual gaps on the printed cards, but the web app wants single spaces.

class Capabilities

`Capabilities` decides, once at server start, what this server can recognise. It depends on which model files are present.

- `self.yolox` is true when `config.SEAL_MODEL_PATH` (the YOLOX-Nano ONNX file) exists and the `onnxruntime` package can be imported.
- `self.classifier`: with the default argument `"auto"`, it loads `GestureClassifier()` if the trained landmark model file exists. If loading fails (for example the model was trained with an older feature version) it prints `landmark model off: ...` and continues without it. The loaded classifier is shared read-only by all sessions, so it is only loaded once.
- `self.landmark_model` is true when a classifier was loaded.
- `self.recognizable` is the list of recognisable signs, in `config.GESTURES` order. The rule mirrors `SignFusion`: with YOLOX, the YOLOX signs plus `config.LANDMARK_SIGNS` (only if the landmark model exists); without YOLOX, every label of the landmark model except `idle`; with neither, nothing.
- `self.jutsu_available[j]` is true when every sign of jutsu `j` is recognisable.

`recognizer(sign)` returns `"yolox"` for a YOLOX sign when YOLOX is on (or when there is no landmark model either), otherwise `"landmarks"`. This value is shown on the sign cards in the app.

build_catalog() and _sign_art()

`build_catalog(caps)` builds the JSON returned by `GET /api/jutsu` once at start-up. For each sign in `catalog.SIGNS` it creates an object with `id`, `name`, `jp`, `hands`, `official`, `cues` (cleaned), `art` (image URL), `recognizer` and `recognizable`. For each jutsu in `config.JUTSU` (config order: fire_style, chidori, shadow_clone, rasengan) it creates an object with title, name, kanji, official flag, tag, provenance, description, the effect id, the list of `steps` (step number, sign id and the card URL `/references/<jutsu>/step_<n>.png`), the `sequence_image` URL, `available` and `missing_signs`.

`_sign_art(sid)` returns `/references/_sources/signs_bold/<sid>.png` if the bold version of the drawing exists (made by `tools/build_bold_signs.py` for the web cards), otherwise the original thin line art in `signs/`.

class Slots

`Slots` counts open sessions with a thread lock, because a slot is released from a session's worker thread, not from the event loop. `acquire()` returns `False` when `active >= max`; `release()` decrements but never below zero. The maximum comes from `--max-sessions` (default `config.SERVER_MAX_SESSIONS` = 2). Each session runs its own copies of MediaPipe, YOLOX and the segmenter, so this limit protects the CPU.

class Session: one browser connection

A `Session` object exists for each open WebSocket. Its fields:

field	meaning
`ws`	the FastAPI `WebSocket`
`server`	the shared `Server` object (capabilities, slots, catalog)
`id`	6 hex characters from `secrets.token_hex(3)`, shown in logs and sent in `hello`

field	meaning
`mode`	`{"mode": "free", "jutsu": None}` or `{"mode": "practice", "jutsu": "<id>"}`
`engine`	the session's own `Engine` (created on the session thread)
`tracker`	an `EventTracker(config.JUTSU)` that turns engine state into events
`executor`	a `ThreadPoolExecutor` with exactly one worker thread named `session-<id>`
`queue`	a `deque` of waiting work items, in arrival order
`wakeup`	an `asyncio.Event` that wakes the worker when the queue gets an item
`send_lock`	an `asyncio.Lock` so two coroutines never write to the socket at the same time
`seq`, `fps`, `_last_done`	the frame counter and smoothed processing rate

Why one dedicated thread per session? MediaPipe objects must not be used by two threads at once, and AI processing is slow (tens of milliseconds). Running it in the session's own thread keeps the asyncio event loop free (so other sessions and REST calls are not blocked) and guarantees that one session's models are only ever touched by one thread.

Sending

`_send(text=None, data=None)` sends text or bytes under `send_lock`. Any exception (the socket closed) is converted into the private exception `_Gone`, which means "the client is gone, end quietly". `send_json(message)` serialises compactly with `ensure_ascii=False` (so Japanese text stays readable). `send_error(code, message, fatal=False)` sends `{"type": "error", "code", "message", "fatal"}`.

Lifecycle: run()

`run()` is the whole life of a session:

1. Create the engine on the session thread (`_create_engine`): `Engine(classifier=..., yolox=...)` followed by `engine.warm_up()`. If this fails, send a fatal `internal` error, close with code 1011 and return.
2. Send the `hello` message: protocol version, session id, the full `ServerInfo` and the current mode.
3. Start two asyncio tasks: `_reader()` (receives messages) and `_worker()` (processes the queue). Wait until either one finishes.
4. If a task ended with an unexpected exception (not `_Gone`), log it, send a fatal `internal` error and close with 1011.
5. In `finally` (always runs, even if the server cancels the task): cancel both tasks, submit `_close_engine` to the session thread (so it runs after the frame in progress), release the slot when that closing job is done, register the closing job with `server.track_closing`, and shut the executor down without waiting.

The comment in the code explains why there is no `await` in the `finally` block: if the server cancels the task, an `await` there could be interrupted and the cleanup would be lost.

Warm-up. The very first run of MediaPipe, YOLOX and the person segmenter can take seconds (the engine docstring mentions 1.8 s measured on the first frame of a session). `Engine.warm_up()` runs every model once on a grey 640x480 picture before `hello` is sent: hands twice, one YOLOX inference (waiting up to 15 s for its result), and one segmentation (so Shadow Clone does not stall when it first appears). It then resets the recognition state so nothing from the blank picture remains. Even after warm-up the engine reports `ready: false` until `config.READY_FRAMES` (5) frames in a row were processed in under `config.READY_MAX_MS` (500 ms); before that no sign is confirmed.

Incoming messages: _reader(), _on_frame(), _on_text()

`_reader()` loops on `ws.receive()`. A disconnect ends it. Binary messages go to `_on_frame`, text messages to `_on_text`.

`_on_frame(data)` validates a camera frame quickly, on the event loop, without decoding the image:

- longer than `config.FRAME_MAX_BYTES` (4 MiB): bad;
- `unpack` fails: bad;
- the payload does not start with the JPEG signature `FF D8 FF`: bad.

A bad frame is queued as `("bad", t, reason)`; a good one as `("frame", t, jpeg)`, where `t` is the client's timestamp from the header (or `None`).

`_enqueue(item)` appends the item and then enforces the in-flight limit: if more than `config.FRAME_MAX_IN_FLIGHT` (2) frames are waiting, the oldest waiting ones are replaced by `("skip", t)`. They will be answered with `skipped` instead of being processed, so the server never builds up a backlog and always works on recent frames.

```
def _enqueue(self, item):
    self.queue.append(item)
    frames = [i for i, it in enumerate(self.queue) if it[0] == "frame"]
    for i in frames[:max(0, len(frames) - config.FRAME_MAX_IN_FLIGHT)]:
        self.queue[i] = ("skip", self.queue[i][1])
    self.wakeup.set()
```

`_on_text(text)` parses a JSON control message:

- `ping`: answered at once with `pong` (not queued behind frames), so the client can measure network latency.
- `reset`: queues a call to `_reset` (engine `reset_sequence()` and tracker `reset()`). No reply.
- `mode` with `free`: queues `_set_mode(None)`.
- `mode` with `practice` and a known jutsu id: queues `_set_mode(jutsu)`.
- `options` with a boolean `landmarks`: queues `_set_landmarks(show)`, which sets `engine.draw_landmarks`. No reply.
- anything else (not JSON, unknown type, practice with an unknown jutsu, `landmarks` not a boolean): a non-fatal `bad_message` error.

Because `mode`, `reset` and `options` go through the same queue as frames, they are applied in order: frames sent before a mode change are still answered with the old mode.

`_set_mode(jutsu)` chooses either all jutsu (free mode) or only one (practice), calls `engine.set_jutsu(chosen, practice=...)` and `tracker.set_mode(...)`, and returns `{"type": "mode", ...}` which the worker sends as the confirmation.

Processing: _worker() and _process()

`_worker()` waits on `wakeup` while the queue is empty, then pops one item:

- `skip`: send `{"type": "skipped", "client_t": t}`.
- `bad`: send a `bad_frame` error followed by `skipped` (so the client's count of unanswered frames stays correct).
- `frame`: run `_process` on the session thread; if it reports a problem, handle it like a bad frame, otherwise send the packed binary answer.
- `call`: run the function on the session thread and send its reply if it returned one.

`_process(client_t, jpeg)` runs on the session thread and does the real work for one frame:

1. Decode the JPEG with `cv2.imdecode`. Failure means "could not be decoded".
2. Scale down (never up) so the frame fits in `FRAME_MAX_WIDTH` x `FRAME_MAX_HEIGHT` (1280 x 1280), using `INTER_AREA` (the best filter for shrinking).
3. Refuse frames smaller than `FRAME_MIN_SIZE` (32 px) on the short side.
4. If `config.MIRROR` is true, flip horizontally (selfie view). Clients send the camera image unmirrored; the server mirrors it so the effects are drawn on what the user will actually see.
5. Call `engine.process(frame, time.perf_counter())`. This runs recognition, the sequence controller and the effects, and returns a result object.
6. Build a `FrameView` (and an `EffectView` if an effect is active) from the result and pass it to `tracker.update(view)`, which returns this frame's list of events.
7. Encode the output frame to JPEG with `OUTPUT_JPEG`.
8. Collect timings: `decode`, the engine's own timings (`hands`, `model`, `yolox`, `seg`, `fx`), `encode` and `total`, in milliseconds.
9. Update the frame counter and the FPS: the instant rate is `1 / (time since the last finished frame)`, smoothed with an exponential moving average (`0.9 * old + 0.1 * new`; the second frame uses the instant rate directly).
10. Build the header with `_frame_state` and return `pack(header, jpeg_bytes)`.

The `EffectView` fields deserve a note. `key` is the engine's activation id (or a negative object id as fallback), so the tracker can tell a new activation from the same one. `clones` is `int(effect.drawn)` for Shadow Clone only. `staged` says whether the effect has a build-up.

_frame_state(): the FrameState header

`_frame_state` reads the stabilizer (the component that waits for a sign to be held before confirming it): `candidate` is the sign it is waiting on (only when different from the latched one), `latched` is the last confirmed sign still being held, and `hold` is 0..1 progress towards confirmation. The fused sign comes from `r.fused` (`name`, `source`, `score` rounded to 3 decimals), or `idle` / `none` / 0 when there is none. The effect object is built when an effect is active.

The full list of FrameState fields:

field	type	meaning
`type`	string	always `"frame"`
`seq`	int	frames processed in this session so far (1, 2, ...)
`client_t`	number or null	the `t` of the client frame this answers (for round-trip latency)
`width`, `height`	int	size of the returned image
`fps`	number	smoothed processing rate of this session
`timings`	object	milliseconds per stage: `decode`, `hands`, `model`, `yolox`, `seg`, `fx`, `encode`, `total` (a key is missing when that stage did not run)
`hands`	int	hands MediaPipe found in this frame
`sign`	object	`{name, source, score}`: this frame's fused decision; `source` is `yolox`, `landmarks`, `landmarks-backup` or `none`
`candidate`	string or null	sign being held but not confirmed yet
`hold`	0..1	how far `candidate` is towards confirmation

field	type	meaning
`latched`	string or null	last confirmed sign while it is still held
`history`	list	last confirmed signs (up to 4, oldest first)
`progress`	object	per jutsu of the current mode: steps done
`time_left`	number or null	free mode: seconds the leading jutsu has left for its next sign; always null in practice mode
`time_left_per_jutsu`	object	free mode: seconds left for every jutsu in progress; `{}` in practice
`ready`	bool	false while the AI is still getting ready (see warm-up)
`cooldown`	number	seconds before another jutsu may fire
`mode`	object	current mode
`effect`	object or null	the active effect (below)
`events`	list	what happened in this frame, in order

The `effect` object:

field	meaning
`name`	effect id: `fire`, `chidori`, `shadow_clone`, `rasengan`
`jutsu`	jutsu id that fired it, or null (started by hand on the desktop)
`age`	seconds since the jutsu fired (`now - activated_at`), build-up included
`stopping`	true while fading out
`clones`	Shadow Clone only: clones visible in the returned frame
`crowded`	Shadow Clone only: the frame is too narrow for 10 clones without overlap
`phase`	`voice`, `charge`, `name` (build-up) or `release` (main effect)
`phase_left`	seconds left in the current build-up stage, null in `release`

class Server and create_app()

`Server` holds what all sessions share: `caps` (Capabilities), `slots` (Slots), `catalog` (built once), and a set of "closing" futures. `track_closing(future)` remembers a session's cleanup job until it finishes. `wait_closed(timeout=10)` waits for all pending cleanups; it is called when the server shuts down, because once Python is exiting MediaPipe can no longer close its models properly. `info()` returns the `ServerInfo` dictionary (name, version, protocol, yolox, landmark_model, recognizable, jutsu_available, current sessions, frame limits, mirror, `sequence_max_gap`, and the activation timing: `charge` per jutsu, `voice_default`, `voice_gap`). Note that `frame.jpeg_quality` is sent as 0..1 (`FRAME_JPEG_QUALITY / 100` = 0.8), the format a browser's canvas encoder expects.

`create_app(max_sessions, classifier)` creates the `Server`, defines a `lifespan` handler whose shutdown part runs `server.wait_closed` in a thread, creates the `FastAPI` app and registers the routes:

route	method	returns
`/api/info`	GET	`server.info()` (`ServerInfo`)
`/api/jutsu`	GET	the catalog: `{signs, jutsu}`

route	method	returns
`/api/credits`	GET	`{"sections": catalog.CREDITS}` (image, sound and model credits, disclaimer)
`/api/voices`	GET	`{"voices": catalog.voice_clips()}` : installed character voice clips per jutsu (all null today)
`/ws`	WebSocket	one session
`/references/...`	GET	static files from `gesture_references/`
`/audio/...`	GET	static files from `assets/audio/`
`/{path}`	GET	the React app (SPA fallback)

The WebSocket handler accepts the connection first, then tries to take a slot. If none is free it sends `{"type": "error", "code": "busy", "fatal": true, "message": "All N sessions are in use. Try again later."}` and closes with code 1013 ("try again later"). Otherwise it runs `Session(ws, server).run()`, which frees the slot when it ends.

```
@app.websocket("/ws")
async def ws_session(ws: WebSocket):
    await ws.accept()
    if not server.slots.acquire():
        await ws.send_text(json.dumps({"type": "error", "code": "busy", "fatal": True,
                                       "message": f"All {server.slots.max} sessions are in use. Try
again later."}))
        await ws.close(code=1013)
        return
    await Session(ws, server).run()
```

`/api/voices` calls `catalog.voice_clips()`, which looks in `assets/audio/voice/<jutsu>/` for `activation.wav` and `name.wav`, reads their length with Python's `wave` module, and returns a URL and `seconds` for each, or `null` when a file is missing or not a readable PCM WAV. The folder is read on every request, so adding a clip does not need a restart.

The catch-all `frontend(path)` route is registered last, so the API and static mounts win. It returns 404 for unknown `/api/...` paths (so a typo in an API call does not silently return the HTML page). If `frontend/dist/index.html` does not exist it returns `BUILD_HELP`. If the path is an existing file inside `dist` it is returned; the check `file.is_relative_to(dist)` after `resolve()` blocks path-traversal tricks such as `../../secret`. Any other path returns `index.html`, so client-side routes of the React app (for example `/practice`) work when the page is reloaded.

main()

`main()` parses the command-line options (`--host`, `--port`, `--max-sessions`, `--ssl-certfile`, `--ssl-keyfile`), checks that the two SSL options are given together and that `--max-sessions` is at least 1, builds the app, prints the URL, and runs it with `uvicorn`. HTTPS matters because browsers only allow camera access on `localhost` or HTTPS; phones on the local network therefore need `--host 0.0.0.0` plus a certificate (the Arabic guide shows how to make a self-signed one with `openssl`).

Relevant config constants:

constant	value	used for
`SERVER_HOST` / `SERVER_PORT`	`127.0.0.1` / 8000	default address
`SERVER_MAX_SESSIONS`	2	simultaneous sessions

constant	value	used for
`FRAME_MAX_WIDTH` / `FRAME_MAX_HEIGHT`	1280 / 1280	bigger frames are scaled down
`FRAME_MIN_SIZE`	32	smaller frames are refused
`FRAME_JPEG_QUALITY`	80	quality the browser should use (sent as 0.8)
`OUTPUT_JPEG_QUALITY` / `OUTPUT_JPEG_444`	90 / True	quality of returned frames
`FRAME_MAX_IN_FLIGHT`	2	unanswered frames a client may have
`FRAME_MAX_BYTES`	4 MiB	largest accepted binary message
`MIRROR`	True	server mirrors frames
`PROTOCOL_VERSION` / `APP_VERSION`	1 / "0.1.0"	sent in `hello` and `/api/info`
`FRONTEND_DIST`, `AUDIO_DIR`, `REFERENCES_DIR`	paths	built app, sounds, reference images

WebSocket messages at a glance

Client to server:

message	form	effect on the server	reply
camera frame	binary: header `{"t": ms}` + JPEG	queued, processed in order	binary frame (FrameState + JPEG) or `skipped`
`{"type": "mode", "mode": "free"}`	text	all four jutsu can fire, 7 s limit between steps	`{"type": "mode", "mode": "free", "jutsu": null}`
`{"type": "mode", "mode": "practice", "jutsu": "<id>"}`	text	only this jutsu, no time limit	`{"type": "mode", "mode": "practice", "jutsu": "<id>"}`
`{"type": "reset"}`	text	forgets every sequence in progress (a running effect keeps running)	none
`{"type": "options", "landmarks": true}`	text	draw or hide hand landmarks on returned frames	none
`{"type": "ping", "t": n}`	text	answered at once	`{"type": "pong", "t": n}`

Server to client:

message	when
`hello`	first message after the engine is ready: protocol, session id, ServerInfo, mode
binary frame	the answer to each processed camera frame
`mode`	confirmation of a mode change
`pong`	answer to `ping`
`skipped`	a frame was dropped (backlog) or was bad; carries `client_t`
`error` `busy`	all sessions in use (fatal, close code 1013)
`error` `bad_frame`	frame too big, malformed header, not a JPEG, undecodable or too small (non-fatal, followed by `skipped`)
`error` `bad_message`	invalid text message (non-fatal)

message	when
`error` `internal`	engine failed to start or session crashed (fatal, close code 1011)

Events inside FrameState

The `events` list is produced by `EventTracker` in `shinobi/events.py` (documented in another part). The browser plays every sound and UI animation from these events.

event	fields	when
`sign`	`sign`	the stabilizer confirmed a sign (once per held sign)
`step`	`jutsu`, `step`, `total`, `sign`	a confirmed sign advanced a jutsu (not sent for the last step)
`wrong_sign`	`jutsu`, `step`, `expected`, `got`	the confirmed sign is not the next one; nothing is reset
`sequence_failed`	`reason` ("timeout"), `jutsu`, `step`, `expected`, `got`	free mode only: more than 7 s without the next sign, so that jutsu started over
`jutsu`	`jutsu`, `effect`, `stages`	the full sequence was validated; the build-up starts; `stages` gives the seconds of `voice`, `charge`, `name`
`effect_phase`	`effect`, `jutsu`, `phase`	the activation entered `charge`, `name` or `release` (each once, in order)
`effect_stopping`	`effect`	the effect started fading out
`effect_end`	`effect`	the effect is gone (also for an effect replaced by a new jutsu)
`clones`	`count`	Shadow Clone: more clones became visible (used to time smoke-poof sounds)

Order inside one frame: `sequence_failed`, `sign`, `wrong_sign`, `step`, `effect_end`, `jutsu`, `effect_phase`, `effect_stopping`, `clones`.

How server.py connects to other files

- `engine.py` does all the AI and effect work; the server only decodes, calls `engine.process`, and encodes.
- `events.py` turns per-frame state into events; the server builds the `FrameView` it needs.
- `catalog.py` provides card texts, credits and voice-clip discovery.
- `config.py` provides every constant.
- The frontend (`frontend/src/...`) implements the other half of the protocol; `docs/web_protocol.md` is the contract both follow.

The effects package: overview

The folder `shinobi/effects/` contains:

file	role
`__init__.py`	the `EFFECTS` factory dictionary and `activation_stages()`
`base.py`	the `Effect` base class (timeline, fade in/out, build-up stages) and `EffectManager`
`fx.py`	shared drawing tools: `add_glow`, `color_ramp`, `Particles`, `random_directions`
`fire.py`	Fire Style (`FireStyle`)
`chidori.py`	Chidori (`Chidori`, `lightning_path`)

file	role
`rasengan.py`	Rasengan (`Rasengan`)
`shadow_clone.py`	Shadow Clone (`ShadowClone` , `plan_row` , `clearance`)

Some vocabulary used in all of them:

- A **frame** is a NumPy array of shape `(height, width, 3)`, type `uint8`, in OpenCV's **BGR** colour order (blue, green, red), so a colour like `(255, 150, 40)` is blue-ish, not red.
- A **layer** is a black image of the same size where the effect is drawn first; it is then added onto the frame.
- **Additive blending** means pixel values are added (light on top of light), clipped at 255. That is how glowing effects look like light.
- `tracking` is a `Tracking` object from `shinobi/tracking.py`: `now` (time), `visible` (hand seen, or lost for less than `HAND_LOST_GRACE` = 0.6 s), `anchor` (smoothed palm centre in pixels), `size` (smoothed hand size in pixels) and `direction` (unit vector from the wrist to the middle knuckle). The values are smoothed with One Euro filters so effects do not jitter.
- `dt` is the time in seconds since the previous frame. All motion is multiplied by `dt`, so effects move at the same real speed whether the server runs at 5 or 30 frames per second.
- `segmentation` is `(small_frame, mask)` from `shinobi/segmentation.py`, only computed when the active effect needs it (Shadow Clone).

shinobi/effects/__init__.py

Purpose

This file is the "menu" of effects. Other code never creates effect objects directly; it asks this file.

EFFECTS

```
EFFECTS = {
    "shadow_clone": lambda: ShadowClone(config.EFFECT_DURATION["shadow_clone"]),
    "rasengan": lambda: Rasengan(config.EFFECT_DURATION["rasengan"]),
    "chidori": lambda: Chidori(config.EFFECT_DURATION["chidori"]),
    "fire": lambda: FireStyle(config.EFFECT_DURATION["fire"]),
}
```

Each value is a function (a `lambda`) that builds a brand-new effect object. That matters: every activation needs fresh state (new particles, new clone history, new timers). Storing ready-made objects would make the second activation start with the leftovers of the first. The durations come from `config.EFFECT_DURATION`: Shadow Clone 10 s, Rasengan 8 s, Chidori 8 s, Fire 6 s (main effect only, without the build-up and the fade-out).

activation_stages(jutsu, audio_dir=None)

Returns the build-up plan of a fired jutsu: `{"voice": s, "charge": s, "name": s}`.

- `voice` and `name`: if the character voice clip `assets/audio/voice/<jutsu>/activation.wav` (or `name.wav`) is installed, its length plus `config.VOICE_GAP` (0.05 s); otherwise `config.VOICE_DEFAULT` (0.4 s). The length is read by `catalog.voice_seconds`.

- `charge`: `config.ACTIVATION_CHARGE[jutsu]` (1.2 s for all four), or `ACTIVATION_CHARGE_DEFAULT` (1.2).

Since no voice clip is installed today, every jutsu currently builds up for 0.4 + 1.2 + 0.4 = 2.0 s before its main effect. The engine calls this in `start_effect(..., staged=True)` and the plan is sent to the browser in the `jutsu` event's `stages`, so sounds and visuals stay in step.

shinobi/effects/base.py

Purpose

Defines what every effect has in common: its timeline (build-up stages, main effect, fade-in, fade-out, end) and the `EffectManager` that runs the one active effect.

The timeline: voice, charge, name, release

A jutsu fired by its hand-sign sequence does not show its main effect immediately. It goes through four phases:

1. `voice`: the character would shout the activation call (if a clip were installed). The chakra starts to gather (charge level 0 to 0.2).
2. `charge`: the chakra charges up (charge level 0.2 to 1.0).
3. `name`: the character would say the jutsu's name. Charge level stays at 1.0.
4. `release`: the main effect runs for its `duration`, then fades out.

During the first three phases the effect's `render_charge()` draws a smaller "gathering chakra" picture. An effect started by hand (desktop keys 1-4, or tests) has no build-up and is in `release` from its first frame.

```
jutsu fires                          release_at                          duration   fade_out
    |-- voice --|---- charge ----|-- name --|======== main effect ========|~~ fade ~~| gone
      0.4 s          1.2 s          0.4 s
```

class Effect

Class attributes (overridden by subclasses):

attribute	default	meaning
`name`	`"base"`	effect id
`needs_segmentation`	False	if True the engine runs the person segmenter for this effect
`follows_hand`	False	if True the effect is drawn at the tracked hand
`fade_in`	0.3 s	opacity rises from 0 to 1 over this time
`fade_out`	0.5 s	opacity falls to 0 over this time after the stop

Instance fields: `duration`, `t0` (start of the main effect, `None` during the build-up), `stop_at` (when the fade-out started), `age` (seconds since release), `activated_at` (when the jutsu fired), `stages` (list of `(phase, seconds)`).

Methods:

- `begin(now, stages=None)`: starts an activation. With a stages dictionary it stores the three stages (negative values become 0); without one it calls `start(now)` directly.
- `stage_plan` (property): `{phase: seconds}`, rounded; used for the `jutsu` event.

- `release_at` (property): `activated_at + sum of stage lengths`.
- `building` (property): true while `t0` is `None` and there are stages.
- `phase(now)`: returns `(phase, seconds_left)` by walking through the stages; `("release", None)` once the build-up is over.
- `charge_level(now)`: 0..1, as described above. During `voice` it goes linearly from 0 to 0.2; then from 0.2 to 1.0 over the `charge` stage; then stays at 1.0.
- `render_charge(frame, tracking, dt, segmentation, level, alpha)`: draws the build-up. The base version returns the frame unchanged; each effect overrides it.
- `start(now)`: sets `t0`.
- `stop(now)`: sets `stop_at` once (calling it again does nothing).
- `min_on_screen` (property): seconds of main effect before a lost hand may end it. With `config.HAND_EFFECT_MIN_SECONDS = None` it equals the whole `duration`, meaning hand effects never end early because MediaPipe lost the hand. Two-hand seals often hide the hand, so this keeps the effect on screen at the hand's last known place.
- `opacity(now)`: the fade envelope. It also updates `age` and starts the fade-out automatically when `age >= duration`.
- `finished(now)`: true once the fade-out is complete.
- `render(...)`: must be implemented by each subclass.

```
def opacity(self, now):
    self.age = now - self.t0
    if self.stop_at is None and self.age >= self.duration:
        self.stop_at = now
    a = min(1.0, self.age / self.fade_in)
    if self.stop_at is not None:
        a = min(a, max(0.0, 1.0 - (now - self.stop_at) / self.fade_out))
    return a
```

class EffectManager

Holds the `factories` dictionary (`EFFECTS`) and `active`, the one running effect (or `None`). Only one effect runs at a time.

- `start(name, now, stages=None)`: builds a new effect with the factory and calls `begin`. Any running effect is simply replaced.
- `stop(now)`: stops the active effect (it then fades out).
- `needs_segmentation` (property): true when the active effect needs the segmenter. The engine checks this before each frame, so the expensive segmentation only runs while Shadow Clone is active.
- `render(frame, tracking, dt, segmentation)`: the per-frame logic:

1. No active effect: return the frame unchanged.
2. Still building up and stopped early: draw the build-up with a fading alpha; remove the effect when the fade reaches 0.
3. Still building up and `now < release_at`: draw the build-up at full alpha.
4. Build-up just ended: call `effect.start(effect.release_at)`. Note it starts at the exact planned time, not at "now", so a slow frame does not shift the timeline.
5. For hand effects: if the hand is not visible, the effect is not already stopping, and the main effect has run at least `min_on_screen`, stop it.
6. If the effect is finished, set `active = None` and return the frame.

7. Otherwise return `effect.render(...)`.

Connections

The engine (`engine.py`) creates `EffectManager(EFFECTS)`, calls `start` when the controller fires a jutsu, calls `render` every frame, and reads `effect.phase(now)`, `stage_plan`, `stop_at`, `activated_at` and (for Shadow Clone) `drawn` and `crowded` to fill the FrameState.

shinobi/effects/fx.py

Purpose

Shared, speed-optimised drawing tools used by the three hand effects (and the particles also by Shadow Clone's smoke).

add_glow(frame, layer, center, extent, strength=1.0, blur=25, opacity=1.0, soften=0.0, backdrop=0.0)

Adds `layer` onto `frame` additively and adds a soft blurred halo around it (a "bloom"). This is what makes a drawn shape look like glowing light. Returns a new frame.

Step by step:

1. Compute a square region of interest (ROI) around `center` of half-size `extent + 2 * blur`, clipped to the frame. If it is smaller than 8 pixels (the effect is off screen) return a copy of the frame.
2. If `soften > 0`, blur the sharp layer a little at half resolution (four times cheaper). This melts separate particles into one shape, used by Fire.
3. Make the halo: shrink the ROI of the layer by 4 (or by 8 if `blur > 40`), blur it there with a sigma of `blur / shrink`, and resize it back. A halo is soft anyway, so blurring a small image looks the same and is many times faster.
4. If `backdrop > 0`, darken the picture under the light first, strongest where the halo is brightest. Reason given in the code: added light cannot show on a white wall (it is already at 255), so in a bright room a little shadow under the effect keeps it vivid.
5. Add the sharp core (`addWeighted` with `opacity`) and the halo (`strength * opacity`). OpenCV's `addWeighted` on `uint8` saturates at 255 instead of wrapping around.

Two performance tricks summarised: only the ROI around the effect is processed, never the full frame; and blurs are done on shrunken copies.

color_ramp(t, stops)

Interpolates colours along a gradient. `t` is an array of values 0..1 (for example particle ages) and `stops` is a list of `(position, (b, g, r))`. It uses `np.interp` per colour channel and returns an `(N, 3)` array. Fire uses it so a particle goes white-hot, yellow, orange, red, then dark smoke as it ages.

class Particles

A particle system stored as a "structure of arrays": one NumPy array per property, one row per particle, with no Python object per particle. That makes updating hundreds of particles a handful of vector operations.

array	shape	meaning
`pos`	(N, 2)	position in pixels (or fractions for Shadow Clone smoke)
`vel`	(N, 2)	velocity per second
`life`	(N,)	seconds lived so far
`max_life`	(N,)	lifespan
`size`	(N,)	radius

- `emit(pos, vel, max_life, size)`: appends `n = len(vel)` new particles, all starting at `pos`.
- `update(dt, gravity=(0, 0), drag=0)`: simple Euler integration: velocity += gravity times dt, velocity is reduced by drag, position += velocity times dt, life += dt. Then all arrays are filtered with a boolean mask `life < max_life`, which removes dead particles in one step.
- `t` (property): normalised age 0 (born) to 1 (dies).
- `draw(layer, colors, sizes=None, positions=None)`: draws each particle as an anti-aliased filled circle. The arrays are converted to Python lists once before the loop, which the comment notes is much faster than converting each value inside the loop. Particles outside the layer are skipped.

random_directions(n, rng, center_angle=None, spread=pi)

Returns `n` unit vectors. With `center_angle`, the angles are within plus or minus `spread` of it (a cone, used for the fire jet); without it, any direction.

shinobi/effects/fire.py

Purpose

Fire Style (Great Fireball): a stream of flame particles blown out of the hand. Effect id `fire`, fired by the jutsu `fire_style`. `follows_hand = True`.

Constants

- `FIRE_RAMP`: five colour stops by particle age: white-hot `(190, 250, 255)` at 0, yellow at 0.15, orange at 0.40, red at 0.70, dark smoke `(10, 10, 30)` at 1.0 (all BGR).
- `EMBER_RAMP`: for the build-up embers: orange far out, white-hot when they reach the gathering point.

class FireStyle

Constructor `FireStyle(duration, rate=650, seed=None)`: a random generator, the emission `rate` (650 flame particles per second), and two particle systems: `flames` (main effect) and `embers` (build-up).

`_origin(tracking, size)` (static): the point where flames come out, `anchor + direction * size * 0.9`, just past the fingers in the direction the hand points. If the direction is unknown, it points up `(0, -1)`.

`render_charge(...)`: the build-up. Embers are spawned on a ring around the origin (distance 1.0 to 1.8 hand sizes, shrinking as the charge rises) with a velocity aimed back at the origin plus a sideways swirl, so they are sucked in and spiral. The emission rate grows with `level` (80 to 400 per second). Two filled circles form a heat glow (dark orange outer, bright yellow inner) whose radius grows from 0.15 to 0.5 hand sizes. Everything goes through `add_glow` with strength, opacity and backdrop rising with `level`.

`render(...)`: the main effect.

1. `self.opacity(now)` updates `age` and starts the fade-out at the end of the duration.
2. If the hand anchor is unknown, return the frame.
3. While not stopping, emit `rate * dt` particles (ramped up over the first 0.3 s) inside a 0.35-radian cone around the hand direction, at 3 to 7 hand sizes per second, from slightly random points near the origin. Lifespan 0.45 to 0.9 s, size 0.09 to 0.2 hand sizes.
4. Update flames with an upward "gravity" of `-3 * size` (hot air rises; in image coordinates negative y is up) and a strong drag of 2.2 (the jet slows down and billows). Leftover embers keep burning out.
5. Colour by age with `FIRE_RAMP`, multiplied by `1 - t^2` so they fade as they cool; size grows with age (`1 + 2t`), so flames expand and merge.
6. Draw flames at half brightness ("dim: overlaps add up to bright"): where many particles overlap, the additive layer becomes bright yellow-white, like the core of a real flame.
7. Compute the bounding box of all flames and call `add_glow` only around it, with `soften` (to melt particles together) and `backdrop=0.35`.

`finished(now)` is overridden: the effect only ends when the fade-out is over **and** every flame has burned out, so the fire never disappears abruptly.

When the effect is stopping, no new particles are emitted, so the fire dies out naturally instead of fading as a block.

shinobi/effects/chidori.py

Purpose

Chidori: crackling lightning bolts around the hand. Effect id `chidori`. `follows_hand = True`.

Constants

`GLOW = (255, 185, 100)` (light electric blue, BGR) and `CORE = (255, 245, 225)` (white-hot, slightly blue).

lightning_path(start, end, rng, detail=5, roughness=0.28)

Generates a jagged lightning polyline with the "midpoint displacement" algorithm:

1. Start with a straight segment from `start` to `end`.
2. For every segment, take its midpoint and push it sideways (along the segment's normal) by a random amount up to `offset`.
3. Insert the midpoints between the old points (the list doubles in size).
4. Halve `offset` and repeat `detail` times.

Big zig-zags come from the first rounds, small jitter from the later rounds, which looks like real lightning. With `detail=5` a bolt has 33 points. The initial offset is `length * roughness`.

```
for _ in range(detail):
    mids = (points[:-1] + points[1:]) / 2
    seg = points[1:] - points[:-1]
    normal = np.stack([-seg[:, 1], seg[:, 0]], axis=1)
    normal /= np.linalg.norm(normal, axis=1, keepdims=True) + 1e-6
    mids += normal * rng.uniform(-offset, offset, (len(mids), 1))
    out = np.empty((len(points) + len(mids), 2), np.float32)
    out[0::2], out[1::2] = points, mids
    points, offset = out, offset / 2
```

class Chidori

Constructor `Chidori(duration, n_bolts=7, seed=None)`: random generator, number of bolts, a list `bolts` of `(polyline, thickness)` relative to (0, 0), the time `next_regen` of the next regeneration, and a `sparks` particle system.

`_regenerate(radius, n_bolts=None, reach=(0.7, 1.8), branches=0.6)`: creates new bolts in random directions with a length of 0.7 to 1.8 times `radius`. With probability `branches` a bolt gets a thinner side branch that forks from a random point in its later two thirds, at 0.8 radians off and 0.4 of its length. Bolts are stored relative to (0, 0) and shifted to the hand when drawn, so a moving hand does not require new bolts.

`_draw_bolts(layer, offset, scale)`: draws every bolt twice: first a wide line in a dimmed glow colour (70 % so overlaps stay below white), then a thin bright core line. Line widths scale with hand size (`scale`, capped at 1.8), so bolts stay bold when the hand is close to the camera.

`render_charge(...)`: the build-up. The radius grows from 0.3 to 0.75 of the full radius with `level`. Every 0.06 s new small crackles are generated: 2 to 7 bolts, shorter reach (0.35 to 1.0) and fewer branches. A small glowing dot at the palm grows. Sparks are emitted on a ring around the hand and flow inward (their velocity is set so they reach the centre at the end of their life). `add_glow` with rising strength, opacity and backdrop.

`render(...)`: the main effect.

1. `alpha = self.opacity(now)`; return if no anchor or invisible.
2. `radius = max(30, hand size * 1.5)`.
3. Every 0.05 s (about 20 times per second) regenerate all bolts. This rapid change is the flicker of electricity.
4. Draw a glow disk at the palm (radius 0.28), then the bolts.
5. Emit about 250 sparks per second in all directions, fast, with downward gravity (400 px/s squared) and drag, coloured from `CORE`, fading with age.
6. `add_glow` with strength 1.5 and `backdrop=0.45`.
7. With probability `0.12 * alpha` per frame, add `(60, 25, 0)` to the whole frame: a brief blue flash over the scene, like a lightning flash lighting the room.

shinobi/effects/rasengan.py

Purpose

Rasengan: a spinning blue energy sphere sitting on the palm. Effect id `rasengan`. `follows_hand = True`. The Rasengan's last sign is the one open palm with fingers spread (`rasengan_palm`), and the sphere is drawn at that palm's centre.

Constants

`BLUE = [255, 150, 40]` and `LIGHT = [255, 225, 170]` (BGR NumPy arrays, so they can be multiplied for darker shades). `CHARGED = 0.45`: the sphere reaches 45 % of its full size by the end of the build-up. `bgr(color)` converts a NumPy colour to a tuple of plain Python floats, because OpenCV drawing functions want that.

class Rasengan

Constructor `Rasengan(duration, n_orbit=70, seed=None)` prepares 70 orbiting sparks. Each has its own orbit radius (0.45 to 1.15 sphere radii), speed (5 to 11 radians per second, random direction), starting phase, tilt (1 = circle, small = flat ellipse) and rotation of its ellipse. This gives the impression of a 3D ball of swirling energy. It also creates a `wind` particle system and a `streaks` array for the build-up (one row per streak: start angle, start radius, age, lifespan, spin).

`_sphere(layer, center, radius)` (static): three filled circles: a soft outer shell (1.05 r, dark blue), a middle (0.55 r, blue) and a bright core (0.28 r, light).

`render_charge(...)`: the build-up.

- A small sphere grows from 0.12 to `CHARGED` (0.45) of the full size with `level`.
- Two spinning arcs (ellipse segments of 220 degrees) rotate at 420 degrees per second in opposite directions.
- Streaks are spawned around the palm (30 to 140 per second) and spiral inward: their radius shrinks with `(1 - t)^1.3` while their angle advances by `spin * t`. The vertical axis is squashed by 0.75 for a perspective look. Each streak is drawn as a short line between its position now and 0.015 s earlier (a tail) plus a bright dot at the head, fading in quickly and brightest half way in.
- `add_glow` with rising strength, opacity and backdrop.

`render(...)`: the main effect, in four drawing layers:

1. **Sphere**: `grow = min(1, age / 0.6) ** 0.5` (a square-root ease: fast at first, then slowing). After a build-up the growth starts from `CHARGED`, so the sphere continues from the charged size instead of jumping back to zero. Radius is `max(25, hand size * 0.95) * grow`.
2. **Spinning arcs**: 8 ellipse arcs with different axes; their angle changes each frame (`now * 420 * direction`), half clockwise and half anticlockwise at 1.4 times the speed. The changing angles create the illusion of rotation.
3. **Orbiting sparks**: positions for all 70 sparks computed in one NumPy step: a point on a tilted ellipse, then rotated by the orbit's rotation angle.
4. **Wind**: about 120 particles per second are emitted on the sphere's rim, moving mostly tangentially (the tangent of direction `(x, y)` is `(-y, x)`) plus a little outward, with drag: wind thrown off a spinning ball.

Finally `add_glow` with strength 1.4, `blur = radius * 0.6` and `backdrop=0.4`.

shinobi/effects/shadow_clone.py

Purpose

Shadow Clone (Kage Bunshin): delayed, cut-out copies of the user appear around them in puffs of smoke. Exactly **10 clones** are shown (11 people on screen with the user), and the count is never reduced to gain

speed. Effect id `shadow_clone`. `needs_segmentation = True` (the engine runs the person segmenter while this effect is active). `fade_out = 0.6`.

The idea: instead of pre-recorded videos, each clone is the user themselves, cut out with the segmentation mask from a frame recorded 0.25 to 1.6 seconds ago. So the clones copy the user's movements with a delay, each a little later than the previous one, and never move in sync.

Where the mask comes from (segmentation.py, briefly)

The engine's `PersonSegmenter.segment(frame)` (in `shinobi/segmentation.py`) shrinks the frame to `config.SEGMENT_WIDTH` (640 px wide), runs the MediaPipe selfie segmenter, and returns `(small_frame, mask)`. The mask is float32 0..1 (1 = person). It is improved three ways: an exponential moving average with the previous mask (`MASK_SMOOTHING` 0.5) removes flicker; a guided filter snaps the mask edge to real image edges (hair, shoulders); and a contrast curve `(mask - 0.3) / 0.4` makes the edge crisp but slightly feathered. The engine runs segmentation on a worker thread in parallel with hand tracking.

Constants

constant	value	meaning
`SMOKE_COLOR`	235	grey level of the smoke
`AURA_COLOR` / `AURA_EDGE`	orange / pale yellow (BGR)	chakra aura of the build-up
`POP_START`	0.3 s	the first clone appears this long after the release
`POP_STEP`	0.12 s	then one more every 0.12 s (the 10th at 1.38 s)
`BOTTOM_FADE`	0.3	raised clones fade over their bottom 30 % instead of a hard cut
`TOP_FADE`	0.06	soft top edge when the user's head touches the top of the image
`EDGE`	3x3 ones	erosion kernel for clone edges
`FIT_STEPS`	1.0 to 0.8	a row may shrink to 80 % when there is not enough room
`SPACING`	0.55	minimum distance between clones of a row, in clone body widths
`SIDE_HYSTERESIS`	0.3	a clone changes side only when its row's share moved clearly
`ROOM_SLACK`	0.25	tolerance before a side counts as too small
`GLIDE`	8.0 per second	how fast a clone glides to its new spot

From `config.py` (one entry per clone, matched by index, front row first):

list	values
`CLONE_COUNT`	10
`CLONE_OFFSETS`	-0.36, 0.36, -0.46, -0.27, 0.27, 0.46, -0.19, 0.19, -0.37, 0.37
`CLONE_SCALES`	0.68, 0.68, 0.56, 0.56, 0.56, 0.56, 0.45, 0.45, 0.45, 0.45
`CLONE_LIFTS`	0.00, 0.00, 0.22, 0.22, 0.22, 0.22, 0.40, 0.40, 0.40, 0.40
`CLONE_DELAYS`	0.25, 0.40, 0.55, 0.70, 0.85, 1.00, 1.15, 1.30, 1.45, 1.60
`CLONE_PARALLEL`	True

So there are three rows: 2 clones in front (68 % size, standing on the bottom edge), 4 in the middle (56 %, raised by 22 % of the frame height) and 4 at the back (45 %, raised by 40 %). Smaller and higher rows look further away. `offset` only gives the left-to-right order inside a row (and the first position before the user is found); the actual positions are planned every frame.

clearance(body, scale, lift) and plan_row(...)

`clearance` is the distance a clone's body centre keeps from the user's: half the user's body width plus half the clone's, reduced for raised clones (their heads are above the user's shoulders, so they can stand closer).

`plan_row(n, center, body, w, scale, lift, phase, left_before=None)` decides where the `n` clones of one row stand, given the user's body centre and width and the frame width `w`. It returns `(xs, clones_on_left, size_factor, crowded)`.

1. For each size factor in `FIT_STEPS` (1.0, 0.95, 0.9, 0.85, 0.8): compute the free room left and right of the user (from the clearance to the frame edge, keeping the clone's whole body inside), and how many clones fit on each side at `SPACING`. Stop at the first size where all `n` fit.
2. If none fits even at 80 %: pack them into what is left beside the user, overlapping, and return `crowded = True`. Clones are never placed behind the user (they would be fully hidden) and **a clone is never dropped**.
3. Otherwise split the clones between the sides in proportion to the room on each side (user in the middle: 5 and 5 overall; user at the right edge: all on the left). To prevent flicker, the previous split (`left_before`) is kept unless the fair share moved by more than 0.5 + `SIDE_HYSTERESIS`, or a side became too small even with `ROOM_SLACK`.
4. Spread the clones evenly over each side; `phase` 0.5 puts them in the middle of their slots, 0 starts at the user. The back row uses phase 0 so rows interleave and back heads are not hidden behind front heads.

class ShadowClone: setup

The constructor checks that the four CLONE lists have equal lengths and that `count` fits, then builds:

- `clones`: a list of dictionaries with `offset`, `scale`, `lift`, `delay`, `appear` (`POP_START + POP_STEP * i`), `shown`, `gone`, `x`, `fit`, `side`.
- `order`: clones sorted by scale, smallest (farthest) first, so nearer clones are drawn later and cover them.
- `rows`: clones grouped by `(scale, lift)`, front row first, each sorted by offset.
- `buffer`: a `deque` of `(time, small_frame, mask, person)` history, oldest first; it keeps `max(delay) + 0.3` = 1.9 s.
- `smoke`: a `Particles` system with positions stored as fractions of the frame (0..1), so it is independent of resolution.
- `anchor`: smoothed `(body centre, body width)` of the real person.
- `drawn`: clones drawn in the last frame (sent as `clones` in FrameState); `crowded`.
- `_worker`: when `parallel` is on, a private one-thread executor. The code comment explains why it is per effect: with several web users each one's clones are built in parallel, and one worker keeps this effect's jobs in order.

measure(mask) and _remember(now, small, mask)

`measure` finds the person in a mask: `left`, `right`, `top` (the box to cut the clone from, arms included) and `center` / `body` computed from the middle 80 % of the mask's horizontal mass (10 % to 90 % of the cumulative column sums). Using mass percentiles means a raised arm hardly moves the body centre. It returns `None` when nobody is there.

`_remember` appends the frame to `buffer`, drops entries older than `keep`, and updates `anchor` smoothly: the centre follows at 15 % per frame, the width only at 6 % ("a moving arm must not reshuffle the crowd").

layout(w, dt) and place(...)

`layout` runs `plan_row` for every row and moves each clone towards its planned x:

- first plan: the clone starts right there;
- same side as before: glide with `ease = 1 - exp(-GLIDE * dt)` (a frame-rate independent smooth approach);
- other side of the user: the clone does not slide through the user; it vanishes and reappears with a smoke puff at the old and the new place.

`place(clone, person, w, h, x=None, fit=None)` computes the clone's target rectangle on the layer: the size is the person's box times the clone's scale and fit; the bottom is at the clone's lifted baseline (`h * (1 - lift)`); the body centre goes to x, but is pushed back inside the frame if it would stick out.

_edge_fade, _poof

`_edge_fade(th, bottom, top)` returns a cached one-column opacity gradient: a linear fade over the bottom 30 % for raised clones (so a floating torso does not end in a hard line) and/or over the top 6 % when the user's head is cut by the image top.

`_poof(clone, w)` emits a burst of smoke particles at the middle of a clone's body (about `90 * scale` particles, spreading and rising slightly). Smoke appears when a clone pops in, disappears or changes sides; it also hides the rough first frames of a clone.

render_charge(): the build-up

Runs on the main thread (the worker only starts with the main effect). While the history buffer is already filling up (so clones can pop in at the release with delayed frames ready):

- Builds a chakra aura around the user's outline at half the segmentation size: the mask is dilated by a radius that grows with `level`, the ring between dilated and original mask is blurred, and it pulses (`0.8 + 0.2 * sin(9 * now)`). The aura darkens slightly (`keep`) and adds orange light (`add`), because pure added light would turn white on a bright wall.
- Emits smoke wisps on both sides of the user, drifting inward and upward. These are the same smoke particles as the clone puffs, so the last wisps simply live on when the clones appear.
- Composites everything with one multiply and one add at full frame size.

render(): the main effect and the parallel worker

```
job = (now, dt, small, mask, self.stop_at)
if self.parallel:
    previous, self._pending = self._pending, self._worker.submit(self._clone_layer, *job)
    if previous is None:
        self.drawn = 0
        return frame
    color, alpha, density, self.drawn = previous.result()
else:
    color, alpha, density, self.drawn = self._clone_layer(*job)
alpha = cv2.multiply(alpha, 1.0 - mask)
```

With `CLONE_PARALLEL` the expensive clone layer for this frame is built on the worker thread while the main thread continues (the engine goes on with the next frame's hand tracking), and the layer built for the previous frame is shown. The clones are already 0.25 to 1.6 s behind on purpose, so one extra frame of delay is invisible. On the very first frame there is no layer yet.

Then:

1. **The real person stays in front**: the clone layer's opacity is multiplied by `1 - mask` using the **current** frame's mask, so the user's own outline never lags even though the clones come from the worker.
2. **Smoke on top**: a uniform grey image is put "over" the layer with the smoke density as opacity.
3. If nothing is visible, return the frame.
4. Resize the colour layer and opacity to full frame size and blend once with `cv2.blendLinear(layer, frame, a, 1 - a)`.

_clone_layer(): building the clones

Runs at segmentation resolution (640 px wide), possibly on the worker thread; it only reads `stop_at` as passed in, because the main thread may stop the effect meanwhile.

1. `_remember` the frame; `layout` the rows if the user was found.
2. For each clone whose `appear` time has come, mark it `shown` and emit a puff. If the effect is stopping, mark shown clones `gone` with a puff. While stopping, clone visibility fades out over 0.25 s.
3. For each clone in far-to-near order: fetch the buffered frame that is at least `delay` seconds old (`_delayed`); crop the person's box from that frame and its mask; resize to the target size; make the mask edge firmer (`a * a`) and erode it by 1 pixel so less of the wall behind the user glows around the clone; apply the edge fades and visibility; clip to the layer; and composite it "over" the layer.
4. Return `(color, alpha, smoke_density, drawn)`.

_over(): Porter-Duff compositing

`_over(color, alpha, region, img, a)` puts `img` with opacity `a` in front of what is already in the layer, in place. The resulting colour is `(img * a + old * old_a * (1 - a)) / (a + old_a * (1 - a))` and the new opacity is `a + old_a * (1 - a)`. The code uses `cv2.blendLinear(img, old, a, old_a * (1 - a))`, which divides by the sum of the two weights, so it computes exactly this formula in one fast OpenCV call.

_smoke_density() and finished()

`_smoke_density(w, h, dt, scale_up=True)` updates the smoke (slight upward drift, drag 1.5), draws every particle as a disc on a half-resolution canvas with a strength that rises fast and fades slowly (`sin(pi/2 *`

`min(4t, 1)) * (1 - t) * 140`) and a size that grows with age, blurs it (sigma 4.5) so puffs merge into a cloud, and returns a 0..1 density map.

`finished(now)` waits for the fade-out **and** for all smoke to disappear.

Performance tricks (summary)

- Everything (clones, smoke, person cut-out) is composed into one layer at segmentation resolution, then blended onto the full frame once, instead of one full-size blend per clone.
- Each clone only touches its own box, not the whole layer.
- Smoke and aura are drawn at half resolution.
- The clone layer is built on a parallel worker thread, one frame ahead.
- Edge fade columns are cached by size.
- If the frame size changes during a Shadow Clone (camera switched, phone rotated), the engine ends the effect because its one-frame-behind layer cannot be blended onto the new size.

Timeline with the default settings

time after the jutsu fires	what is on screen
0 to 2.0 s	build-up: pulsing aura and smoke wisps (voice 0.4, charge 1.2, name 0.4)
2.0 s	release: main effect starts
2.3 s	first clone pops in with smoke
2.3 to 3.38 s	one more clone every 0.12 s until all 10 are visible
12.0 s	duration (10 s) over: clones vanish in smoke, fade-out 0.6 s
after that	the effect ends once the last smoke has cleared

assets/audio/

Purpose and layout

All sounds of the web app. The server exposes this folder read-only at `/audio/...`; the Python code itself does not play any sound (only the voice clips are read, for their length). The frontend defines the sound cues in `frontend/src/audio/cues.ts` and decides when to play them in `frontend/src/audio/eventCues.ts` (sound effects) and `frontend/src/audio/jutsuMusic.ts` (music), always in reaction to server events.

```
assets/audio/
  ASSET_LICENSES.md    every file with source, creator, licence, attribution, commercial use
  SOURCES.md           sources, edits, levels, layering details, credits block
  fire_style/          7 WAV files
  chidori/             6 WAV files
  rasengan/            8 WAV files
  shadow_clone/        7 WAV files
  ui/                  4 WAV files
  jutsu_music/         4 WAV loops (one per jutsu)
  music/               leaf_trail_loop.flac + compose_leaf_trail.py
  voice/               README.md + 4 empty folders (fire_style, chidori, shadow_clone, rasengan)
```

That is 37 audio files in total. All sound effects are 16-bit PCM WAV at 44.1 kHz.

When each sound plays

The activation timeline in the browser follows the server's events: `step` plays the hand-seal sound; `jutsu` plays the last sign's hand-sign sound (and would play the activation voice); `effect_phase charge` plays the charge; `effect_phase name` would play the jutsu name voice; `effect_phase release` stops the charge and plays the release sounds and starts the loop; `effect_stopping` plays the ending; `effect_end` stops the loop.

file	when it plays
`ui/hand_seal.wav`	every confirmed step (`step` event), pitched one semitone higher per step
`ui/sequence_failed.wav`	`sequence_failed` (timeout)
`ui/step_success.wav`	defined as a cue; played as a sound test on the Settings page
`ui/mode_switch.wav`	played by the sound context when audio is unlocked
`fire_style/hand_sign.wav`	Fire Style accent under the hand seal (a flame flick)
`fire_style/prepare_inhale.wav` , `charge_rise.wav`	Fire Style charge phase
`fire_style/ignition.wav` , `flame_burst.wav` , `impact.wav`	Fire Style release
`fire_style/burning_loop.wav`	loop during the fire, faded out when it stops
`chidori/hand_sign.wav`	Chidori accent (an electric chirp)
`chidori/lightning_charge.wav`	Chidori charge
`chidori/activation_zap.wav` , `thunder_impact.wav`	Chidori release
`chidori/crackle_loop.wav`	loop during Chidori
`chidori/impact.wav`	Chidori ending (`effect_stopping`)
`rasengan/hand_sign.wav`	Rasengan accent (a gust)
`rasengan/charge_magic.wav` , `chakra_charge.wav`	Rasengan charge (two layers)
`rasengan/release_whoosh.wav` , `wind_impact.wav`	Rasengan release
`rasengan/energy_loop.wav`	loop during Rasengan (replaces `spin_loop.wav`)
`rasengan/impact.wav`	Rasengan ending (its 6.9 s tail is faded out after about 1 s)
`rasengan/spin_loop.wav`	defined as a cue but replaced by `energy_loop.wav`
`shadow_clone/hand_sign.wav`	Shadow Clone accent (a hand swish)
`shadow_clone/charge_chakra.wav`	Shadow Clone charge
`shadow_clone/duplicate.wav` , `clone_appear.wav`	Shadow Clone release
`shadow_clone/smoke_poof.wav`	each `clones` event (at most 3 poofs per event, at least 110 ms apart) and at the ending
`shadow_clone/clone_disappear.wav`	Shadow Clone ending
`shadow_clone/seal_charge.wav`	defined as a cue, not used by the event mapping

file	when it plays
`jutsu_music/<jutsu>.wav`	music while that jutsu's signs are being made; ducked under the build-up, faded out at the release or when progress resets
`music/leaf_trail_loop.flac`	soft background music of the app

A wrong sign makes no sound at all, by design.

Licences

According to `ASSET_LICENSES.md` (37 files):

licence	files	attribution
CC0 1.0 (public domain)	16	not required (optional)
CC BY 4.0, or a mix of CC0 and CC BY 4.0 layers	16	required
CC BY 3.0 (`smoke_poof.wav` also has a CC BY 4.0 layer)	4	required
project's own work (the background music)	1	none

There are no non-commercial (NC), no-derivatives (ND) or share-alike (SA) audio files. All third-party sounds were downloaded directly from OpenGameArt.org (two are re-uploads of Freesound sounds, whose original pages were also checked). They are generic sound effects: no Naruto or anime audio, no dialogue, no soundtrack. Creators include faxcorp, qubodup, pyranostudios, BMacZero, Julien Matthey, wobbleboxx, dklon, fins, Fupi, Kenney, rubberduck, artisticdude, jaggedstone, leohpaz, ogrebane, zeroisnotnull, Hansjörg Malthaner, Jordan Irwin and iochi-glaucus. On 2026-11-01, 15 effects were rebuilt by layering them with sounds from JC Sounds' "Fantasy SFX Pack Vol 1" (CC BY 4.0, credit "JC Sounds"), Kenney's "Sci-Fi Sounds" (CC0) and synthesised noise and sine layers; their loudness was matched to the files they replaced so the volume settings in `frontend/src/audio/gains.ts` stay correct.

The four jutsu music loops are licensed tracks cut into seamless loops: Rasengan "Blackmoor Ninjas" by Matthew Pablo (CC BY 3.0), Fire Style "Taiko drums" by congusbongus / jobro (CC BY 3.0), Chidori "Revenge" by TAD (CC BY 4.0), Shadow Clone "Samurai" by TAD (CC BY 4.0). None of them is the original Naruto soundtrack.

The background music "Leaf Trail" was composed and synthesised for this project by `music/compose_leaf_trail.py` (Python, numpy/scipy): a minor pentatonic piece at 90 BPM, 64 s, with synthesised koto, bamboo flute, taiko and bass, rendered to loop seamlessly and stored as FLAC.

The required attribution texts are collected in the "Credits block" sections of `SOURCES.md` and `ASSET_LICENSES.md`, and the server returns credits to the app via `GET /api/credits`.

voice/: character voice slots (empty)

The folder `assets/audio/voice/` has one empty sub-folder per jutsu and a `README.md`. **No voice clip is installed, because no licensed voice is available.** The README lists the 8 expected files (`<jutsu>/activation.wav` and `<jutsu>/name.wav` for Naruto, or Sasuke for Chidori), their required format (16-bit PCM WAV, mono, 44.1 or 48 kHz, dry voice, `activation` 0.25 to 2 s, `name` 0.25 to 2.5 s, about -16 LUFS) and a checker, `python tools/check_voices.py`.

It also documents the research: the anime voices (Junko Takeuchi as Naruto, Noriaki Sugiyama as Sasuke) belong to the rights holders (Masashi Kishimoto / Shueisha, TV Tokyo, Studio Pierrot), no legal source offers them for this use, and online "Naruto voice" files are AI clones or rips that may not be used. The

legal routes would be a written licence from the rights holders, or new recordings by the original actors with approval from the character rights holders.

What the code does with this: `catalog.voice_clips()` (behind `GET /api/voices`) reports every clip as `null`, `activation_stages()` uses `VOICE_DEFAULT` (0.4 s) for the `voice` and `name` stages, and the web app skips a `null` clip. Nothing generic, synthetic or imitated is played instead. If licensed clips are added later, the build-up automatically lasts exactly as long as the clips, without code changes.

gesture_references/

Purpose

Illustrations that teach the user each jutsu's hand signs. The server exposes the folder read-only at `/references/...`, and the catalog (`GET /api/jutsu`) points to these images. They are not training data: the models learn from camera recordings, not from these pictures.

Layout

```
gesture_references/
  README.md                          overview (in Egyptian Arabic)
  fire_style/   chidori/   shadow_clone/   rasengan/
      step_1.png ... step_4.png      one card per step
      sequence_reference.png         the four steps in one image
      instructions.md                step-by-step explanation
  _sources/
      signs/                         original line art (PNG) + CREDITS.txt
      signs_bold/                    thickened versions for the web cards + CREDITS.txt
      rasengan_palm_landmarks.json   hand points used to draw the Rasengan palm
```

The cards and sequence images are generated by `tools/build_gesture_references.py`; the bold line art by `tools/build_bold_signs.py`. The server prefers `signs_bold/<sign>.png` for the `art` URL when it exists.

The four sequences

jutsu	steps	status
Fire Style: Great Fireball	Monkey, Boar, Horse, Tiger	official (the last 4 of the 6 official seals)
Chidori	Snake, Dog, Tiger, Monkey	official (the last 4 of the 9-seal version)
Shadow Clone	Ram, Snake, Tiger, Clone seal	custom combination of official seals
Rasengan	Clone seal, Dog, Horse, Rasengan palm	custom (the Rasengan officially has no hand seals)

Each jutsu starts with a different sign so they are not confused at the first step. The Rasengan palm is not a seal: it is one open palm with the fingers spread, facing the camera, the other hand out of view.

Licences

The eight seal drawings (Ram, Snake, Tiger, Dog, Monkey, Boar, Horse, Clone seal) are line art by Schlum on Wikimedia Commons, vectorised from photographs, licensed CC BY-SA 4.0. Because the project cropped and placed them on labelled cards, and thickened them for the bold versions, the derived images are also CC BY-SA 4.0 (attribution and the same licence are required when sharing them). The Rasengan palm drawing is the project's own, made from one recorded frame of MediaPipe hand points (`_sources/rasengan_palm_landmarks.json` records the source file

`data/landmarks/rasengan/me_day_20261030_174647.npz`, frame 247, the aspect ratio and the 21 point coordinates). No screenshot from the anime or manga is used.

Note: section 6 of `gesture_references/README.md` was written on 2026-10-30, before the seal recognition was built, and still says recognition is not trained yet. That part describes the state at that date, not the current code.

How it connects

- `server.py` mounts the folder at `/references` and uses `_sign_art()` to pick the bold or thin art.
- `catalog.py` holds the same card texts (names, Japanese names, cues, tags, provenance) used to draw the cards, so the app and the images show identical text.
- The React app gets the URLs of `step_<n>.png`, `sequence_reference.png` and the sign art from `GET /api/jutsu` and loads them from `/references/...`.

Part 3 — Frontend (React)

This part explains the whole `frontend/` folder: the web app you open in the browser. It is written in **React 19** with **TypeScript**, built by **Vite**, and it talks to the Python backend (FastAPI) that runs at `http://127.0.0.1:8000`. In production the backend also serves the built app (`frontend/dist`), so the browser only ever talks to one address.

Overview: pages and data flow

The pages

The router (`src/App.tsx`) defines these pages. All of them are drawn inside the light "Konoha" frame (village background, NARUTO header, brush navigation).

URL	What it shows	Main file
`/` (Home)	The "dojo": live camera, list of the four jutsu, hand-sign sequence card	`konoha/dojo/DojoView.tsx`
`/live`	Same dojo state, but the camera is big with a narrow side column	`konoha/live/LiveView.tsx`
`/practice`, `/practice/:jutsuId`	Carousel to choose a jutsu, then a practice session (no time limit)	`konoha/practice/PracticeView.tsx`
`/library`, `/library/:jutsuId`	"Jutsu Scrolls": carousel of the four jutsu and the selected jutsu's signs	`konoha/jutsu/JutsuView.tsx`
`/settings`	Camera, sound, music, animations, reset	`konoha/pages/SettingsPage.tsx`
`/about`	Project description and credits (older dark design)	`pages/AboutPage.tsx`
anything else	404 page (older dark design)	`pages/NotFoundPage.tsx`

The data flow, step by step

The most important idea of the frontend is: **the browser never decides by itself that a sign was made**. It only sends camera pictures and shows what the backend answers.

1. `useCamera` (in `src/hooks/useCamera.ts`) asks the browser for the webcam (`getUserMedia`) and gets a `MediaStream`.
2. `useShinobiSession` (in `src/hooks/useShinobiSession.ts`) plays that stream in a hidden `<video>`, draws each picture on a canvas, compresses it to **JPEG** and sends it over a **WebSocket** (`/ws`) as a binary message: `[4-byte length][JSON header {"t": time}][JPEG bytes]`.
3. The backend recognises the hand sign, updates the sequence, draws the effects (fire, lightning, clones...) on the picture and sends back a binary message: `[4-byte length][FrameState JSON][processed JPEG]`.
4. The frontend decodes the reply (`services/socket/framing.ts`), draws the processed JPEG on the visible `<canvas>` and gives the `FrameState` to every "listener".
5. Listeners turn the `FrameState` and its `events` (`step`, `wrong_sign`, `sequence_failed`, `jutsu`, `effect_phase`, `effect_end`...) into UI (the dojo cards, toasts, the activation overlay — `useDojo`,

`useLiveEvents`) and into sound (`useSessionSounds` → the Web Audio engine).

```
camera -> <video> -> canvas -> JPEG -> WebSocket /ws -> backend (AI + effects)
backend -> FrameState + processed JPEG -> canvas on screen
                                       -> events -> cards, toasts, overlay
                                       -> events -> sounds, voices, jutsu music
```

Besides the WebSocket, three REST calls are used: `GET /api/jutsu` (the catalog: signs and jutsu), `GET /api/info` (what the server can recognise), `GET /api/credits` (About page) and `GET /api/voices` (which character voice clips are installed). Static files come from `/references/...` (sign drawings) and `/audio/...` (sounds).

Provider tree

`src/main.tsx` wraps the app in "providers" (React contexts) in this order; inner ones can use outer ones:

```
<StrictMode>
  <BrowserRouter>
    <ThemeProvider>          Naruto / Sasuke colours
      <SettingsProvider>     user preferences (localStorage)
        <SoundProvider>      Web Audio engine (effects, voices, jutsu music)
          <MusicProvider>    background music loop
            <CatalogProvider>  /api/jutsu + /api/info
              <App />
```

Project files at the root of frontend/

frontend/package.json

Describes the npm project `shinobi-ai-web` (version 0.1.0, `"type": "module"` so files use `import` / `export`).

Script	Command	What it does
`npm run dev`	`vite`	Starts the development server on port 5173 with hot reload
`npm run build`	`tsc -b && vite build`	Type-checks everything, then builds the optimised app into `dist/`
`npm run typecheck`	`tsc -b`	Only the TypeScript check
`npm run lint`	`oxlint`	Fast linter (finds common mistakes)
`npm test`	`vitest run`	Runs the unit tests (`*.test.ts`) once
`npm run preview`	`vite preview`	Serves the built `dist/` for a quick check

Main dependencies: `react`, `react-dom`, `react-router-dom` (pages/URLs), `framer-motion` (animations), `lucide-react` (icons) and several `@fontsource` packages (fonts bundled with the app, so no Google Fonts request: Inter, Nunito, Bebas Neue, Caveat Brush, Permanent Marker, Rajdhani, Yuji Syuku). Dev dependencies: `vite`, `@vitejs/plugin-react`, `typescript`, `vitest` (unit tests), `oxlint`, `playwright-core` (drives Microsoft Edge in the e2e tests), type packages.

frontend/vite.config.ts

Configures Vite:

- `plugins: [react()]` enables JSX and fast refresh.

- `cacheDir` can be changed with the `VITE_CACHE_DIR` environment variable, so two dev servers can run side by side.
- `server.port: 5173` with `strictPort: true` (fail instead of silently picking another port).
- `server.proxy`: in development, requests to `/api`, `/references`, `/audio` and the WebSocket `/ws` (`ws: true`) are forwarded to the backend (`SHINOBI_BACKEND`, default `http://127.0.0.1:8000`). Thanks to this the app always uses its own origin, so the same relative URLs work in development and in the built app.
- `test`: Vitest runs `src/**/*.test.ts(x)` in a `node` environment (no browser DOM needed, because the tested code is pure logic).

frontend/index.html

The single HTML page. It sets `lang="en"` and `data-theme="naruto"` on `<html>`, the tab icons (`/konoha-leaf.svg?v=3` and PNG sizes; the `?v=3` forces browsers to drop a cached old icon), viewport, theme colour, description and Open Graph tags, and the title `NARUTO`. A tiny inline script reads `shinobi.theme` from `localStorage` **before the first paint**, so a user who chose Sasuke does not see a flash of Naruto colours. The body only contains `<div id="root">` and loads `/src/main.tsx`.

Other root files

- `tsconfig.json`, `tsconfig.app.json`, `tsconfig.node.json`: TypeScript settings (target ES2023, `jsx: react-jsx`, `noEmit` because Vite does the build, strict "no unused locals/parameters").
- `public/`: files copied as-is to the site root: `favicon.svg`, `favicon.ico`, `favicon-32/48.png`, `apple-touch-icon.png`, `konoha-leaf.svg` and its PNG sizes.
- `README.md`: notes for developers. `dist/` is the build output, `node_modules/` the installed packages.

Entry point and routing

frontend/src/main.tsx

Imports all fonts and the two global style sheets (`konoha/konoha.css`, `styles/global.css`), then renders the provider tree shown above around `<App />` inside `#root`. `StrictMode` makes React run effects twice in development to reveal bugs; that is why every hook in this project cleans up after itself.

frontend/src/App.tsx

Declares the routes with React Router `<Routes>`. A parent route uses `<KonohaShell />` as layout (header, background, footer) and renders the child page in its `<Outlet />`. `/about` and the catch-all `*` (404) are nested in an extra layout `<LegacyFrame />` that puts the older dark pages into a dark panel so they stay readable on the light background. A comment notes that the older dark pages `pages/PracticePage.tsx`, `pages/LibraryPage.tsx` and `pages/SettingsPage.tsx` stay in the repository but are no longer routed: the routes use the Konoha versions.

Types: the contract with the backend

frontend/src/types/protocol.ts

TypeScript types of everything that goes over the WebSocket (protocol version `PROTOCOL_VERSION = 1`). Keys are snake_case exactly as Python sends them.

- `SignId`: `idle`, `ram`, `snake`, `tiger`, `dog`, `monkey`, `boar`, `horse`, `clone_seal`, `rasengan_palm`.
- `JutsuId` / `EffectId`: `fire_style` / `fire`, `chidori`, `shadow_clone`, `rasengan`.
- `ServerInfo` (`/api/info` and the `hello` message): name, version, whether YOLOX and the landmark model are loaded, `recognizable` signs, `jutsu_available`, session counts, `frame` limits (`max_width`, `max_height`, `jpeg_quality`, `max_in_flight`), `mirror`, optional `sequence_max_gap` (seconds allowed between two signs in Live, default 7).
- `Mode`: `{mode:'free'}` (Home/Live, all jutsu) or `{mode:'practice', jutsu}`.
- `EffectState` and `EffectPhase` (`voice` → `charge` → `name` → `release`): the activation timeline of a fired jutsu; for Shadow Clone also `clones` and `crowded`.
- `FrameState`: the JSON header of every processed frame: `seq`, `client_t` (echo of the time the client sent), size, `fps`, `timings`, `hands`, `sign` (the fused sign with `source` and `score`), `candidate` / `hold` / `latched` (the stabiliser), `history`, `progress` (steps done per jutsu), `time_left` and `time_left_per_jutsu`, `ready` (false while the AI warms up), `cooldown`, `mode`, `effect`, and `events`.
- Events: `SignEvent`, `StepEvent` (jutsu, step, total, sign), `SequenceFailedEvent` (`reason: 'timeout' | 'wrong_sign'`), `WrongSignEvent` (expected, got; progress is **kept**), `JutsuEvent` (optional `stages` with the lengths of voice/charge/name), `EffectPhaseEvent`, `EffectStoppingEvent`, `EffectEndEvent`, `ClonesEvent`.
- Client→server messages (`ClientMessage`): `mode`, `reset`, `options` (`landmarks` on/off), `ping`.
- Server text messages: `hello`, `mode`, `pong`, `skipped`, `error` (`busy`, `bad_frame`, `bad_message`, `internal`, with `fatal`).

frontend/src/types/catalog.ts

Types of the REST answers. `SignInfo` (name, Japanese name, number of hands, `official`, how-to `cues`, `art` URL of the line drawing, `recognizer` `'yolox' | 'landmarks'`, `recognizable`). `JutsuStep` (step number, sign, reference `card` URL). `JutsuInfo` (id, title, name, kanji, `official` = original seals vs custom sequence, description, effect, `steps`, `available`, `missing_signs`). `Catalog = { signs, jutsu }`. `Credits` with sections of items (text, url, licence).

Services: talking to the server

frontend/src/services/api.ts

- `ApiError`: an `Error` with an HTTP `status` (or `null` when the server could not be reached at all).
- `getJSON(path, signal)`: `fetch` with `Accept: application/json`. A network failure becomes the friendly message "The SHINOBI AI server cannot be reached. Is it running?"; an abort is re-thrown untouched; a non-OK status becomes `"/api/jutsu answered 502"`.
- `api.info()`, `api.catalog()`, `api.credits()`: the three REST calls.
- `sessionUrl()`: builds `ws://host/ws` (or `wss://` on https) from the current page address.

frontend/src/services/socket/framing.ts (+ framing.test.ts)

Pure functions (no WebSocket, no DOM) for the binary format `[uint32 big-endian N][N bytes UTF-8 JSON][JPEG]`:

```
export function encodeBinaryMessage(header: unknown, payload: ArrayBuffer | Uint8Array): ArrayBuffer {
  const json = encoder.encode(JSON.stringify(header))
  const body = payload instanceof Uint8Array ? payload : new Uint8Array(payload)
  const out = new Uint8Array(4 + json.byteLength + body.byteLength)
  new DataView(out.buffer).setUint32(0, json.byteLength, false) // big-endian length
  out.set(json, 4)
  out.set(body, 4 + json.byteLength)
  return out.buffer
}
```

- `decodeBinaryMessage` does the reverse and throws `FramingError` if the message is too short, the length runs past the end, or the header is not UTF-8 JSON. The payload is a view (no copy).
- `parseFrameState(header)`: validates that the header looks like a `FrameState` (type `frame`, numeric `seq` / `width` / `height`, a `sign`, a `mode`), filters the events (unknown or malformed events are dropped, known ones are checked field by field in `isEvent`) and fills safe defaults (`fps` 0, `ready` true for older servers, `time_left` null...). This protects the page from crashing if the protocol drifts.
- `parseServerText(text)`: parses and validates the text messages; `skipped` may carry `client_t: null`; an `error` without `fatal` counts as non-fatal.
- `fitWithin(w, h, maxW, maxH)`: scales a size down to fit, keeping the aspect ratio, never upscaling.
- `InFlightFrames`: bookkeeping of camera frames sent but not yet answered, keyed by their `t`. Because the server answers in order, `answer(t)` also settles every older frame; `skip(t)` settles one; `dropOldest()` handles a `skipped` without a time; `expire(now, maxAge)` drops frames that never got an answer, so a lost reply can never block the stream.

`framing.test.ts` checks the byte layout, round trips (also non-ASCII JSON and headers longer than 255/65535 bytes), every rejection case, the validation of frames/events/text messages, `fitWithin` and `InFlightFrames`.

frontend/src/services/socket/client.ts (+ client.test.ts)

`ShinobiSocket` is a typed WebSocket client. One instance = one backend session. It knows nothing about the camera or React.

- **States** (`ConnectionState`): `idle`, `connecting`, `connected` (after the server's `hello`), `reconnecting`, `offline`, `busy` (no free session slot), `error` (protocol mismatch or fatal error). `SocketStatus` adds a user message, the number of consecutive `failures` and `retryAt`.
- **Handlers**: `onStatus`, `onHello`, `onFrame(state, jpeg)`, `onSkipped`, `onMode`, `onPong(t, rtt)`, `onServerError`.
- `connect()`, `disconnect()`, `retryNow()` (skip the waiting time: Retry button or the browser coming back online), `send(message)` and `sendFrame(header, jpeg)` (both refuse while not connected).
- **Reconnect with exponential backoff**: `backoffDelay(failures)` = `base × 2^(failures-1)` capped at `maxMs`, multiplied by a random jitter (defaults 600 ms base, 8 s max, ±20 %). After 3 failures the state becomes `offline` (it keeps retrying in the background). A `busy` answer (error `busy` or close code 1013) waits at least 4 s.
- **Hello timeout**: an attempt that gets no `hello` within 8 s is closed.

- **Protocol check**: a `hello` with another protocol version closes the socket (code 4001) and stops retrying with a clear message.
- **Heartbeat / stall detection**: every 4 s it sends a `ping`; if something was sent and nothing came back for 12 s, it closes (code 4000) and reconnects.
- A `generation` counter is incremented for every new WebSocket, so late callbacks from an old socket are ignored.

`client.test.ts` uses a fake WebSocket and fake timers to check: backoff growth and jitter, connecting then `connected` after hello, sending only while connected, delivering frames/text messages, `offline` and retries, `reconnecting` then `offline` then recovery, busy handling (including a bare 1013 close), stopping on a protocol mismatch, fatal internal error, hello timeout, clean `disconnect()`, `retryNow()`, and ping/stall reconnect.

frontend/src/services/socket/index.ts

Re-exports the public pieces of `client.ts` and `framing.ts` so other files can `import { ShinobiSocket, InFlightFrames } from '../services/socket'`.

Hooks: camera and session

frontend/src/hooks/useCamera.ts

React hook around `navigator.mediaDevices`. Returns `status` (`idle`, `requesting`, `active`, `denied`, `unavailable`, `insecure`, `error`), `stream`, a user-facing `error` text, the list of `devices`, `activeDeviceId`, and the functions `start(deviceId?)`, `stop()`, `refreshDevices()`.

- `start` asks for 1280×720 at 30 fps (ideal values). If a saved camera id no longer exists (`OverconstrainedError` / `NotFoundError`) it falls back to the default camera.
- Errors are translated into clear sentences (for example a busy camera: "Close other apps that use it (Zoom, Teams, the Python camera window)").
- A `requestId` counter ignores the result of an older `start()` if the user clicked again meanwhile (race protection), and stops the losing stream.
- If the track ends (camera unplugged), the status becomes `error` with "The camera stopped".
- Device labels are only visible after permission, so the list is refreshed after a successful start and on `devicechange`.

frontend/src/hooks/useShinobiSession.ts

The heart of the live connection. Options: `stream` (from `useCamera`), `mode`, `landmarks`, `enabled`. The session only runs while there is a stream (`active = enabled && stream !== null && !hiddenLong`).

What it returns (`ShinobiSession`): `status`, `connected`, `info`, `sessionId`, the latest `frame` (FrameState), `live` (a processed frame is on the canvas), smoothed `latencyMs`, `serverFps`, `serverMode`, `paused` (tab hidden), and the functions `attachCanvas`, `subscribe(listener)`, `reset()`, `retry()`.

How it works:

- **Sending (`pump`)**: grabs the current video frame, scales it with `fitWithin` to the server's `max_width/max_height`, draws it **not mirrored** (the server mirrors), encodes JPEG with the server's `jpeg_quality` (using `OffscreenCanvas.convertToBlob` when available) and sends it with header `{t:`

`performance.now()}`. It never has more than `max_in_flight` frames waiting for an answer, never sends faster than the camera frame rate (max 30 fps), waits when the socket buffer holds more than 1.5 MB (backpressure), and does nothing while the tab is hidden. `pump` is called again after each answer, each `skipped`, and by a 400 ms safety timer.

- **Receiving (`onFrame`): settles the in-flight frame, updates the latency (exponential smoothing, factor 0.2), calls every subscribed listener** synchronously, before React re-renders**, so no event is ever missed, then stores the frame in state and draws the JPEG.
- **Drawing (`draw`)**: `createImageBitmap` on the JPEG, then `drawImage` on the attached canvas; frames older than the last drawn `seq` are discarded.
- **On `hello`**: reads the frame limits, then sends the current `mode` and the `options` (landmarks).
- **Mode/landmark changes** are sent as soon as they change.
- **Tab visibility**: hidden → sending pauses; hidden for 60 s → the session is closed so it gives its server slot back.
- **Safety rule**: whenever the status is not `connected`, everything (frame, info, latency, live) is cleared, so the UI can never show recognition from a dead connection.

Contexts (app-wide state)

frontend/src/contexts/ThemeContext.tsx

Holds `theme` (`naruto` or `sasuke`), `setTheme`, `toggleTheme`. It writes the theme to `<html data-theme>` (CSS variables change the colours) and saves it under `shinobi.theme`. It never changes recognition.

frontend/src/contexts/SettingsContext.tsx

User preferences saved in `localStorage` (`shinobi.settings`), no database. `Settings`: `masterVolume`, `sfxVolume`, `voiceVolume`, `muted`, `audioEnabled`, `cameraId`, `animations` (`full` / `reduced` / `off`), `showLandmarks`, `musicOn`, `musicVolume`, `jutsuMusic`. `DEFAULT_SETTINGS` gives the defaults (for example music off, music volume 0.3). `sanitize()` checks every loaded value (volumes clamped to 0..1, wrong types replaced), so a corrupted storage can never break the app. `update(patch)` merges and re-sanitises, `reset()` restores defaults. `motion` is the effective animation level: if the operating system asks for reduced motion, `full` becomes `reduced`. The value is written to `<html data-motion>` for CSS.

frontend/src/contexts/SoundContext.tsx

Creates one `SoundEngine` (see the audio section) and keeps it in sync with the settings: effect volume (`outputVolume`), voice volume, jutsu-music volume, `jutsuMusic` on/off, `audioEnabled`. The first pointer or key press anywhere calls `engine.unlock()` (browsers block audio until the user interacts). Switching Naruto/Sasuke plays `ui.mode_switch`. It exposes two hooks: `useSound()` (a simple API: `ready`, `unlock`, `play`, `startLoop`, `stopLoop`, `stopAll`; a silent no-op version is the default outside the provider) and `useSoundEngine()` (the engine itself, used by `useSessionSounds` and the music context). `ready` uses `useSyncExternalStore` to follow the engine's state.

frontend/src/contexts/CatalogContext.tsx

Loads `/api/jutsu` and `/api/info` once in parallel and shares them: `catalog`, `info`, `loading`, `error`, `reload()`, `jutsu(id)`. `reload()` increments an `attempt` counter, which re-runs the loading effect. An

`AbortController` cancels the request when the provider unmounts.

frontend/src/lib/storage.ts

`loadJSON`, `saveJSON`, `removeKey`: `localStorage` helpers that **never throw** (private mode, full quota or bad JSON simply return `null` / do nothing).

The Konoha UI: frame and shared pieces

The `src/konoha/` folder holds the current light design ("parchment" cards over the Konoha village painting).

frontend/src/konoha/konoha.css

Global design tokens of the Konoha look, all prefixed `--k-` so they do not clash with the older dark tokens: fonts (`--k-logo` Permanent Marker, `--k-brush` Caveat Brush, `--k-body` Nunito), paper and ink colours, Naruto orange and Sasuke blue accents, "Ready"/"Training" status colours, radius, shadow and easing.

frontend/src/konoha/KonohaShell.tsx + KonohaShell.module.css

The layout route of every page. It renders a "Skip to content" link, `<KonohaBackdrop />`, `<KonohaHeader />`, the `<main>` with the page `<Outlet />`, and a footer with the fan-project disclaimer and an About link. On every path change it scrolls to the top and moves keyboard focus to `<main>` (good for screen readers). It wraps everything in framer-motion's `MotionConfig`, which turns off framer animations unless the motion level is `full`. The page `div` is keyed by the first path segment so the entrance animation plays once per section.

frontend/src/konoha/shell/KonohaBackdrop.tsx + KonohaBackdrop.module.css

The fixed decorative background: the painting `assets/konoha/background.jpg` (Naruto left, Sasuke right), a wash, an orange light on Naruto's side and a blue one on Sasuke's side, plus `<FallingLeaves />` only when animations are `full`.

frontend/src/konoha/shell/FallingLeaves.tsx

A canvas with 4 to 12 orange leaves (about one per 150 000 px^2) drifting down. Each leaf sprite is drawn once on a small 64 px canvas (almond shape, midrib, veins). Every animation frame moves each leaf (fall, drift, sideways sway, spin, a horizontal "flip" squash) and respawns it at the top when it leaves the screen. It pauses when the tab is hidden and adapts to window resizes and pixel density.

frontend/src/konoha/shell/KonohaHeader.tsx + KonohaHeader.module.css, BrushSwash.tsx, nav.ts

`KonohaHeader` shows the leaf emblem, a divider and the "NARUTO" wordmark on an orange brush swash (all one link to `/`), the navigation from `KONOHA_NAV` (Home, Live, Practice, Jutsu, Settings; About is in the footer) and the `<MusicControl />`. The active link gets an ink underline that slides between links thanks to framer-motion's shared `layoutId`. `BrushSwash.tsx` is the SVG of the orange brush stroke (gradient, a turbulence/displacement filter for rough edges, bristle streaks). `nav.ts` holds the menu items.

frontend/src/konoha/shell/MusicControl.tsx + MusicControl.module.css

The small background-music button in the header: play/pause toggle and a volume popover (closed by a click outside or Escape). On phones only the toggle is shown. It uses `useMusic()` and marks itself with `MUSIC_CONTROL_ATTR` so the "start music on first click" logic does not fight with it.

frontend/src/konoha/shell/LegacyFrame.tsx + LegacyFrame.module.css

A layout route that wraps the older dark pages (About, 404) in a dark translucent panel.

frontend/src/konoha/LeafEmblem.tsx

The Hidden Leaf symbol redrawn as SVG paths (`LEAF_SPIRAL`, `LEAF_OUTER`, `LEAF_STEM`, `LEAF_TIP`) in a 100×100 box, used as logo (the favicon uses the same paths). Props: `size`, `weight` (stroke width), `title` (if given, it becomes an accessible image; otherwise it is decorative).

frontend/src/konoha/BrushButton.tsx + BrushButton.module.css

The button of the new look. `variant="primary"` draws an orange brush-stroke SVG behind the label, `secondary` is a quiet paper button. Optional `icon`. With `to` it renders a router `<Link>` instead of a `<button>`.

frontend/src/konoha/BrushTag.tsx + BrushTag.module.css

A black dry-brush label with a white hand-lettered title (the "LIVE CAMERA", "JUTSU" headers). Prop `as` chooses `h1` / `h2` / `h3`.

frontend/src/konoha/ParchmentCard.tsx + ParchmentCard.module.css

A `<section>` styled as parchment, optionally with a `BrushTag` title (`title`, `icon`) and `actions` on the right of the header. Used by almost every card.

frontend/src/konoha/jutsuLook.ts

`JUTSU_LOOK`: a lucide icon, colour and tint per jutsu. It is not imported anywhere at the moment (the cards now use `JutsuIcon`).

The dojo (Home and Live)

frontend/src/konoha/dojo/useDojo.ts

The state hook shared by Home and Live. It combines everything:

- `useCatalog()`, `useSettings()`, `useCamera()`, `useShinobiSession({stream, mode: free, landmarks})`, `useSessionSounds(session)` and `useLiveEvents(session, catalog)`.
- If the catalog failed because the server was down, it reloads it as soon as a session connects.
- `live` = camera on **and** processed frames on screen **and** `frame.ready !== false` (not while the AI warms up). `frame` is `null` when not live.
- **Which jutsu the sequence card shows (`focusId`)**: the jutsu the user clicked (`selected`), updated by `followFocus` whenever the progress changes. `followFocus` only moves to another jutsu if that one has

strictly more steps, so a wrong sign that happens to start another jutsu never steals the card. An activation (`jutsu` event) moves the card to the jutsu that fired. While a timeout notice is shown, the card is "held" on its jutsu. Fallback: the first available jutsu, else the first one.

- `complete`: for 4 s (`SUCCESS_MS`) after the backend fires the focused jutsu (its progress is back to 0 at that moment).
- `timeLeft`: the countdown of the focused jutsu from `time_left_per_jutsu` (each jutsu has its own clock); older servers only send `time_left` of the leader.
- It subscribes to frames to build the card notes: a `wrong_sign` for the focused jutsu sets `warning` and increments `missKey` (the amber ring); a `step` clears them; a `sequence_failed` with reason `timeout` sets a `notice` built by `timeoutNotice` (kept 6 s); a `jutsu` clears everything. When the focused jutsu changes, its notes are cleared.
- `startOver()` sends `{type:'reset'}` and clears the notes.

It returns all of this for the views (`catalog`, `info`, `camera`, `session`, `events`, `live`, `frame`, `focus`, `done`, `complete`, `timeLeft`, `warning`, `missKey`, `notice`, `startOver`, `select`...). Note the pattern used several times here: "derive state during render" (`if (key !== seenKey) { setSeenKey(key); ... }`), which React allows and which avoids an extra effect.

frontend/src/konoha/dojo/DojoView.tsx + DojoView.module.css

The Home layout: `CameraCard` and `JutsuListCard` side by side on top, `SequenceCard` under them, plus a visually hidden `aria-live` paragraph that announces steps, failures and activations for screen readers. Props: `heading` (hidden `h1`) and `documentTitle`.

frontend/src/konoha/dojo/CameraCard.tsx + CameraCard.module.css

The "LIVE CAMERA" card. Props: `camera`, `session`, `catalog`, `events`, `className`, `size` (`default` or `hero` for the Live page).

- While the AI is live it shows the `<canvas>` (attached with `session.attachCanvas`) with the processed frames: already mirrored, effects drawn by Python.
- Otherwise it shows the local `<video>` preview (mirrored by CSS) dimmed, with a notice: "AI is getting ready..." (connecting, connected-but-no-frame, or warming up) or the connection label (offline, busy...) plus a "Retry now" button. The text explicitly says no recognition is shown until the AI is live.
- `ConnectionChip`: AI state label (or "Paused" when the tab is hidden), latency in ms and server fps (`data-testid="stat-latency"`, `stat-fps`).
- `SignChip`: "AI sees" + the current sign name and score, or "No hands" / "No sign", and a small bar showing the stabiliser's hold progress.
- `StageOverlay` (activation overlay and toasts) is drawn over the canvas when live.
- When the camera is off: "Camera is off", "Waiting for the camera...", or a problem message (blocked, insecure page, not found, error).
- Controls: "Start Camera" (also unlocks audio; becomes "Try Again" after an error), "Stop Camera", and a camera picker when several cameras exist (the choice is saved in settings). A full-screen button uses the Fullscreen API on the video box (`useFullscreen`); full screen is left automatically when the camera stops.
- The box keeps an aspect ratio between 1 and 16/9 based on the frame or preview size. `useNow` re-renders every 500 ms only to update a "Retrying in N s" countdown.

frontend/src/konoha/dojo/JutsuListCard.tsx + JutsuListCard.module.css

The "JUTSU" card: one button per jutsu (in API order) with `JutsuIcon`, the title-cased name and sub-name, and on the right either the live step count `2/4` (from `progress`), a "Ready" pill, or a "Training Required" pill whose reason (from `jutsuAvailability`) is given as tooltip and as hidden description. Clicking calls `onSelect`. While loading it shows skeleton rows; on error, "The jutsu scrolls are sealed" with the start-server hint and a Retry button. `density="compact"` for the Live side column.

frontend/src/konoha/dojo/sequenceSteps.ts (+ sequenceSteps.test.ts)

Pure helpers of the sequence card:

- `stepStates(total, done, live, complete)`: an array of `done` / `current` / `todo`. The next step is `current` only while live; when `complete`, every step is `done`.
- `focusStep(...)`: which step "zooms in": the next one, but only after the backend confirmed at least one sign, while live and not complete.
- `shownDone(...)`: the "N / 4" number.
- `nextCountdownAnchor(prev, timeLeft, now, step, maxExtrapolateS)`: the logic that makes the countdown smooth **and** able to jump back up (see below).
- `signGesture(sign)`: the little CSS motion hint per sign (`rise`, `press`, `twist`, `push`).

`sequenceSteps.test.ts` checks: a new step starts from the backend value, jitter never makes it go up within a step, it **jumps back up when the backend restarts the clock after a wrong attempt**, step states while live/not live/complete, clamping, and when the zoom applies.

frontend/src/konoha/dojo/SequenceCard.tsx + SequenceCard.module.css

The big "FIRE STYLE — HAND SIGN SEQUENCE" card used on Home, Live (`variant="side"`) and Practice (`timed={false}`).

Props: `jutsu`, `catalog`, `info`, `done`, `complete`, `live`, `cameraOn`, `status`, `timeLeft`, `loading`, `variant`, `timed`, `missKey`, `onReset`, `warning`, `notice`.

What it renders:

- Header: `JutsuIcon`, title, "— Hand sign sequence", progress dots with a filling line, and `N / 4` (`data-testid="step-count"`).
- If `timed`: the `Countdown` row and the "Start over" button (the slot is always there but `hidden` when no sequence is in progress, so the layout never jumps).
- Four numbered tiles (`<ol>`): the sign drawing (`SignPicture`, with a hand icon if the image fails), an arrow between tiles, a check mark when done, an amber ring (`data-testid="step-miss"`, re-keyed by `missKey` so its animation replays) on the current tile after a wrong attempt, and "not trained yet" if the server cannot recognise that sign. CSS animations (only with full motion) make the waiting sign "breathe", a confirmed step "pop", and a shimmer run when the jutsu fires.
- Footer, in priority order: offline hint (start camera / AI getting ready / connection label) → "X activated!" → timeout `notice` → `warning` (wrong sign, progress kept) → "Training required" with the reason → "Next: HORSE — first cue".
- `SignCredit`: licence line of the drawings (Schlum / Wikimedia Commons, CC BY-SA 4.0; the palm drawing is the project's own).

The countdown and why it can jump back up. The backend sends `time_left` for the focused jutsu in each frame (about 10–30 times per second). To show a smooth value between frames, `useSmoothCountdown` stores an *anchor* (value, time, step) and every 100 ms displays `anchor.value - elapsed`, but never extrapolates more than `MAX_EXTRAPOLATE_S = 0.4` s past the last frame (the backend's clock also stops when no frame arrives). On every new frame the anchor is recomputed with `nextCountdownAnchor`:

```
if (!prev || prev.step !== step) return { value: timeLeft, at: now, step } // new step: start fresh
const shown = Math.max(0, prev.value - Math.min(maxExtrapolateS, (now - prev.at) / 1000))
if (timeLeft > shown + CLOCK_RESTART_S) return { value: timeLeft, at: now, step } // clock restarted
return { value: Math.min(timeLeft, shown), at: now, step } // otherwise only go down
```

So within one step the number only goes down (network jitter cannot make it flicker upward), **except** when the backend's value is more than `CLOCK_RESTART_S = 1` s above what is shown. That happens in Live after a wrong attempt: the backend gives the same step its full time again (7 s), and the bar refills. A new confirmed step also restarts from the backend value. The bar width is `left / maxGap` (`sequence_max_gap`, default 7) and turns "urgent" at 2 s or less. The countdown is only shown while live, not complete, and after at least one step (`counting`).

Live page

frontend/src/konoha/live/LiveView.tsx + LiveView.module.css

Same `useDojo()` state as Home but a different layout: a large `CameraCard size="hero"` and, beside it on desktop (under it on tablets and phones), a compact `JutsuListCard` and a side `SequenceCard`. Also the hidden `aria-live` announcement.

frontend/src/pages/LiveDemoPage.tsx and frontend/src/pages/HomePage.tsx

Tiny route components: `HomePage` renders `<DojoView heading=... documentTitle={null} />`, `LiveDemoPage` renders `<LiveView heading=... documentTitle="Live" />`.

Jutsu page (library)

frontend/src/konoha/pages/JutsuPage.tsx and frontend/src/konoha/pages/PracticePage.tsx

One-line route wrappers: `KonohaJutsuPage` renders `<JutsuView />`, `KonohaPracticePage` renders `<PracticeView />`.

frontend/src/konoha/jutsu/JutsuView.tsx + JutsuView.module.css

"Jutsu Scrolls": a `BrushTag` heading, a `Carousel` of `JutsuCard`s and, under it, `JutsuDetail` for the selected jutsu. `/library/<id>` preselects that card; an unknown id shows a short notice. Selecting a card updates the URL with `replaceUrl` (no real navigation, so the shell does not scroll to the top). Loading shows skeletons; an error shows the "sealed scrolls" message with Retry. The CSS module is also reused by the Practice page.

frontend/src/konoha/jutsu/pageState.ts

Helpers shared by the Jutsu and Practice pages: `replaceUrl(path)` (uses `history.replaceState`), `useSelectedJutsu(param)` (selection that starts from the URL and follows real navigations), `selectedIndex(list, selected, fallback)`.

frontend/src/konoha/jutsu/sequenceKind.ts

`sequenceKind(official)` returns the label and meaning: "Official seals — The original hand seals, in their original order." or "Custom sequence — Designed for SHINOBI AI: the source has fewer than four seals."

frontend/src/konoha/jutsu/JutsuCard.tsx + JutsuCard.module.css

One carousel card: a big decorative kanji glyph, the index `01` … `04`, the Official/Custom badge, the animated `JutsuIcon`, the name (the heading holds a button stretched over the whole card, `aria-pressed` = selected), the sub-name and kanji, the description (hidden when `compact`), the meaning of the badge, and a footer with "Ready" / "Training Required" and either `extra` (Practice: "Training now") or "4 hand signs".

frontend/src/konoha/jutsu/JutsuDetail.tsx + JutsuDetail.module.css

The "Hand Signs" parchment under the carousel: the selected jutsu's sign tiles (`SignSteps`), provenance (where the sequence comes from), availability on this server, and a "Practice" `BrushButton` linking to `/practice/<id>`. Animated with framer-motion when the selection changes.

frontend/src/konoha/jutsu/SignSteps.tsx + SignSteps.module.css

- `SignArt`: the sign's line drawing (`<img>`), or a hand icon if it fails to load. Also used by `PracticeSession`.
- `SignSteps`: numbered white tiles with drawing, name, Japanese name and cues (optionally limited by `cues`). A soft glow walks over the tiles 1→4 (stops with reduced/off animations).
- `ArtCredit`: the licence line for the drawings.

Carousel

frontend/src/konoha/carousel/Carousel.tsx + Carousel.module.css

A generic horizontal carousel (`Carousel<T>`), used by the Jutsu and Practice pages. The parent owns the selection (`active`, `onActiveChange`); other props: `items`, `getKey`, `getLabel`, `renderItem(item, {index, active})`, `label`, `noun`, `testId`.

- `measure()` reads each slide's `offsetLeft` from the DOM (widths come from CSS container queries) and computes the track offset that brings each slide to the leading (left) position, clamped so the last card ends at the right edge. A `ResizeObserver` re-measures on size changes.
- The track's `x` is a framer-motion value animated with a spring (`stiffness 280, damping 34`); without full motion it jumps instantly.
- Input: left/right arrow buttons, dots, keyboard (←, →, Home, End; focus moves to the card's own button without scrolling), mouse drag and touch swipe. `onDragEnd` projects the release position with

the velocity, picks the nearest snap point, and makes a short flick (more than 36 px or 380 px/s) move at least one card. A click that ends a drag does not select the card under the pointer.
- If Tab focuses an off-screen card, the clipped viewport is kept unscrolled and the track slides instead.

Practice page

frontend/src/konoha/practice/PracticeView.tsx

The `/practice` and `/practice/:jutsuId` page. It reuses the Jutsu page's `Carousel` and `JutsuCard` (`compact`, footer "Training now" on the active card) and its CSS module, then renders `<PracticeSession>` for the selected jutsu. Without a jutsu in the URL it starts on the first available one. Choosing another card updates the URL (`replaceUrl`) and switches the session's practice jutsu while the camera keeps running. If the catalog failed, it retries every 5 s by itself.

frontend/src/konoha/practice/PracticeSession.tsx + PracticeSession.module.css

The practice session of one jutsu. It creates its **own** camera and session in mode `{mode:'practice', jutsu}` (so the backend only counts this jutsu), plus `useSessionSounds` and `useLiveEvents`.

- It subscribes to frames (only frames whose mode is this practice) and keeps: `completedKey` (set on this jutsu's `jutsu` event), `failure` (the last `wrong_sign`, or a `sequence_failed` from older servers) and `missKey`. A `step` clears the failure, except when the wrong sign arrived in the same frame.
- Changing jutsu or losing the connection resets this local state (derived-state pattern with `scopeKey`).
- `done` comes **only** from `frame.progress[jutsu]` (and becomes all steps when completed).
- Layout: `CameraCard`; a "coach" `ParchmentCard` titled "Step N of 4" (or "Complete!") with the current sign shown big (`SignArt`, number, name, Japanese name, cues, a link to the step's reference card), an "AI sees" line ("Right sign: hold it" / "Not this step's sign"), the failure message (amber "not recognized" for `wrong_sign`, red alert for older resets), a "Training Required" note if the server cannot recognise all signs, and the buttons "Start over" / "Practise again" (`session.reset()` and stop the voice) and "Back to Jutsu". Under all that comes the `SequenceCard` with `timed={false}`: Practice has **no time limit** and a wrong sign never resets it.
- framer-motion animates the step change (slide), the completion (scale) and a small shake on a failure.

Settings page

frontend/src/konoha/pages/SettingsPage.tsx

Uses the CSS of `konoha/settings/SettingsPage.module.css`. Sections, each a `ParchmentCard`:

- `CameraSection`: its own `useCamera()` with a mirrored local preview (no AI), a camera selector (browser default, detected cameras, and "Saved camera (not detected yet)" if the saved id is not visible), start/stop buttons, framing tips and a message per camera status.
- `SoundSection`: switches "Enable sounds" and "Mute", sliders Master volume, Sound effects and Character voices (with a hint from `useVoiceManifest`: how many of the 8 voice slots are installed; none are bundled), and "Test sound", which unlocks audio and plays `ui.hand_seal`.
- `MusicSection`: background music on/off (`useMusic().toggle`) with a status line, a "Music volume" slider and the "Jutsu music" switch (music while a jutsu's signs are made).

- `ExperienceSection`: the Animations choice (full / reduced / off, with a note if the operating system forces reduced motion) and "Show hand landmarks".
- `ResetSection`: a two-step confirmation ("Reset settings", then "Yes, reset" or "Cancel") that calls `reset()` and sets the theme back to Naruto.

`KonohaSettingsPage` puts the camera section on top and the others in two columns.

frontend/src/konoha/settings/controls.tsx + controls.module.css, frontend/src/konoha/settings/SettingsPage.module.css

Light form controls: `SettingRow` (icon, label, description, control), `SwitchRow` (a `role="switch"` button with `aria-checked`), `SliderRow` (a range input 0..1 shown as a percentage), `Segmented<T>` (a radio-group style choice) and `SelectField`. `SettingsPage.module.css` holds the page layout (grid, columns, notes, confirmation box).

Background music

frontend/src/konoha/music/player.ts

`MusicPlayer`: the soft background loop (`MUSIC_SRC = /audio/music/leaf_trail_loop.flac`, an original loop) on its own small Web Audio graph. The file is decoded once and played by a looping `AudioBufferSourceNode` (gapless, unlike `<audio loop>`), through a gain for fades (fade-in 2.2 s, fade-out 0.8 s) and a duck gain. `preload()` downloads the file early, `play(level)` starts or follows a new level (it needs a user gesture first), `pause()` fades out and keeps the position, `setDuck(level, seconds)` lowers it, `subscribe` / `isPlaying` support `useSyncExternalStore`, `dispose()` closes everything.

frontend/src/konoha/music/MusicContext.tsx

`MusicProvider` and `useMusic()`. The music plays only when it is wanted (`musicOn`), sounds are enabled, not muted, volume above 0, the tab is visible, **and** the user has interacted with the page (browser rule). If the music was on last time, it starts with the first click or key press of the visit (inside the gesture, which Safari requires), except clicks on the music controls themselves. It follows the sound engine: while a jutsu's own sequence music plays it fades to silence (0.35 s); under a character voice it ducks to `VOICE_DUCK_MUSIC` (0.3); then it comes back (0.8 s). API: `on`, `playing`, `available`, `waiting`, `volume`, `toggle`, `setOn`, `setVolume`.

Audio engine (sound effects, voices, jutsu music)

All jutsu sounds follow the backend's events only. The design separates **pure planners** (input: a `FrameState`; output: a list of commands; easy to unit-test) from **players** (Web Audio).

frontend/src/audio/cues.ts

`CUES`: the list of sound-effect files (served by the backend under `/audio/...`) with a `loop` flag, and the type `CueId`. Groups: `ui.*` (hand_seal, step_success, sequence_failed, mode_switch) and one group per jutsu: `fire_style.*`, `chidori.*`, `shadow_clone.*`, `rasengan.*` (hand-sign accents, charges, activation sounds, impacts, and loops such as `fire_style.burning_loop`, `chidori.crackle_loop`, `rasengan.energy_loop`).

frontend/src/audio/gains.ts

- `CUE_GAIN`: a fixed gain per cue, because the files differ a lot in loudness (from -38 to -9 dBFS). Each cue is brought to the level of its role: the hand seal about -14 dBFS (clearly heard), accents -20, UI -24, charges -21, activations -13 (the loudest moment), loops -23, endings -17. The comments list the measured levels.
- `outputVolume`, `musicOutputVolume`, `voiceOutputVolume`: master × (sfx, music or voice), 0 when muted, clamped to 0..1.
- Ducking constants: during a voice the effects go to `VOICE_DUCK_SFX = 0.4` (about -8 dB) and the music to `VOICE_DUCK_MUSIC = 0.3` (about -10 dB), with attack 0.02 s and release 0.18 s.

frontend/src/audio/engine.ts

`SoundEngine`, one per app (created in `SoundContext`). The audio graph ("buses"):

```
effect one-shot/loop -> voice gain (cue gain x volume x fades)
   -> output gain (user sfx volume) -> duck (under a voice) -> limiter -> speakers
character voice clip -> clip gain -> voice gain (user voice volume) -> limiter
jutsu music (JutsuMusicPlayer) -> music gain (user music volume) -> music duck -> limiter
```

- `unlock()` creates or resumes the `AudioContext` (it must follow a user gesture), builds the graph above with a `DynamicsCompressor` used as a limiter (threshold -6 dB, ratio 12), then decodes all cues (`load`) and the installed voice clips (`loadVoices`, from `/api/voices`).
- `play(cue, options)`: a one-shot with optional `delayMs`, `fadeInMs`, `fadeOutAfterMs`/`fadeOutMs`, `rate` (pitch) and a `filter`. A cue that is not decoded yet is **skipped** (a late sound would not match the picture). At most 32 sounds at once.
- `startLoop(cue)` (one loop per cue), `stop(cue, fadeMs)`, `stopAll(fadeMs)`.
- `run(commands)`: executes the `SoundCommand`s produced by `eventCues.ts`.
- `playVoice(plan)` / `stopVoice()`: plays the character voice clips at their planned times, ducks the effects and the music until the end of the plan, and notifies `subscribeSpeech` listeners (`isSpeaking()`).
- `setVolume`, `setVoiceVolume`, `setMusicVolume`, `setEnabled`, `voiceDuration(id)`, `musicBus()`, `isReady()`/`subscribe` (for `useSyncExternalStore`), `dispose()`.
- `music`: the `JutsuMusicPlayer` attached to the music bus.

frontend/src/audio/eventCues.ts (+ eventCues.test.ts)

The pure planner of sound effects: `soundsForFrame(state, frame, nowMs)` returns the new `CueState` and a list of `SoundCommand`s (`play`, `loop`, `stop`).

Backend event	Sound
`step`	the hand seal `ui.hand_seal`, one semitone higher at each step (`SEAL_RISE`), with the jutsu's own accent under it (`HAND_SIGN_ACCENT`, volume 0.55); once per frame (the most advanced jutsu gives the accent), never twice for the same step
`wrong_sign`	nothing (a wrong gesture is silent)
`sequence_failed` (timeout)	`ui.sequence_failed` (soft)
`jutsu` with `stages`	the hand-seal sound of the 4th sign; the rest follows the phases
`effect_phase` charge	`CHARGE[jutsu]` (for example Fire: inhale, then a rising charge with fade-in)

Backend event	Sound
`effect_phase` release	the charge fades out (250 ms); `RELEASE[effect]` (main sounds, impact, loop)
`jutsu` without stages (older server)	`RELEASE` at once
`clones`	`shadow_clone.smoke_poof` for new clones, at most 3 per event, at least 110 ms apart
`effect_stopping`	`ENDING[effect]` (impacts, fades, clones disappearing)
`effect_end`	that effect's loop stops

`CueState` remembers the running `activation` (charged / released), `lastStep` (to avoid double sounds), the clone count and the next allowed poof time. The tests check the hand-sign rules, the full activation timeline (confirmation, charge, name, release), stopping during the build-up, and that every cue has a gain.

frontend/src/audio/voiceCues.ts (+ voiceCues.test.ts)

Pure planner of the character voices (Naruto or Sasuke, see `voices.ts`). **Only** a backend `jutsu` event speaks: never a step, a wrong sign or a timeout. With the activation timeline, the `activation` clip plays at once and the `name` clip waits for `effect_phase` name; without it both play back to back (80 ms gap). If the effect ends before its name phase, there is no name clip. A mode change stops the voice. The same activation twice (same `seq`, or the same jutsu within 1 s) is ignored. Missing clips are listed in `missing` and never replaced by another sound. `voicePlan()` lays the clips out in time. The tests cover all these rules plus the parsing of the voice manifest.

frontend/src/audio/voices.ts

Voice clip ids (`<jutsu>.activation`, `<jutsu>.name`), the speaker per jutsu (`chidori` = Sasuke, the others Naruto), `parseVoiceManifest` (accepts only URLs under `/audio/voice/<jutsu>/`) and `fetchVoiceManifest` (`GET /api/voices`; an old server or no network means no voices).

frontend/src/audio/useVoiceManifest.ts

A hook that loads the voice manifest once, for the Settings page (`null` while loading).

frontend/src/audio/jutsuMusic.ts (+ jutsuMusic.test.ts)

Pure planner of the music played while the user makes a jutsu's signs (one licensed loop per jutsu in `TRACKS`, each with a level-matching gain; not the Naruto soundtrack). `musicForFrame(state, frame)` returns `MusicCommand`s:

- The **leading** jutsu (most steps, at least 1, from `leader()`; on a tie the one already playing keeps the music) starts its track with a 1.5 s fade-in; another jutsu taking the lead with **more** steps crossfades (0.8 s).
- Every step: a short `pulse` dip so the hand seal stays clear.
- `jutsu` event: duck to 25 % (`DUCK_ACTIVATION`) under the build-up; `effect_phase` release: fade out over 1.5 s.
- Progress back to 0 without a `jutsu` (timeout, Start over, another mode): fade out over 1 s. A wrong sign keeps the music going.
- No new track starts while an activation runs.

`jutsuMusic.test.ts` tests the planner, the player (with a fake AudioContext) and that the engine's three volumes are independent.

frontend/src/audio/jutsuMusicPlayer.ts

`JutsuMusicPlayer` executes the music commands on the engine's context: `loop source -> track gain (fades, track level) -> activation duck -> step dip -> music bus`. One track at a time: a new one crossfades from the old one once decoded; going back to a track that is still fading out brings that same copy back (never two copies). Only the two most recent tracks stay decoded (they are large). `prefetch()` downloads the files at the first live frame; a track asked for before audio is allowed starts as soon as it is. `setEnabled` follows the "Jutsu music" setting; also `stop(fadeMs)` and `dispose()`.

frontend/src/audio/useSessionSounds.ts

The hook that connects a session to the audio. It subscribes to frames and, for each frame, runs `voiceForFrame` first (so the voice ducks the effects before this frame's activation sounds), then `soundsForFrame` and `engine.run`, then `musicForFrame` and `engine.music.run`. In development it logs once per missing voice clip. A watchdog stops the jutsu music if no frame has arrived for 2 s (camera stopped). On disconnect or unmount it resets every planner state and stops everything: **no sound without the backend**. It returns `stopVoice()` (used by Practice's Start over).

frontend/src/audio/voicePlayback.test.ts

Tests the `SoundEngine` with a fake Web Audio context: character voices (scheduling, replacing, stopping, ducking the effects), the hand-sign sound and the charge, and the background music ducked under a voice.

Live overlay components (src/components/live)

frontend/src/components/live/useLiveEvents.ts

Turns backend events into overlay state for `StageOverlay`. It returns `activation` (`{key, jutsu, effect, phase}` with phase `reveal`, then `active`, then `ending`), `toasts`, `dismissToast` and `announcement` (text for screen readers).

- The overlay appears **only** on a `jutsu` event. The big reveal lasts the planned build-up (`(voice + charge + name) × 1000 + 800 ms`), or `REVEAL_MS = 2200` ms on older servers, and leaves early at `effect_phase` release so it never covers the effect. A stop or end arriving during the reveal is remembered (`pending`) and applied after it.
- After the reveal it becomes a compact "active" banner, then `ending`, and it disappears on `effect_end`.
- Toasts (at most 3, 3.4 s each): a gentle note on `wrong_sign`, a reset notice on `sequence_failed`.
- When the connection drops, the activation and the toasts are cleared.

frontend/src/components/live/labels.ts (+ labels.test.ts)

Texts and small derivations that only format what the backend sent: `signName`, `signInfo`, `SOURCE_LABEL`, `failureTitle` / `failureDetail`, `wrongSignTitle` / `wrongSignDetail` ("HORSE not recognized — that looked like DOG"; "Your 2 completed steps are kept. Try HORSE again (step 3 of 4)."), `titleCase`, `followFocus` and `leadingJutsu` (see `useDojo`), `timeoutNotice` ("Time ran out — sequence reset."), `jutsuAvailability`

(available or not, and why: landmark model not trained, YOLOX not loaded...), `connectionText` (label, tone and detail with "Retrying in N s"), `formatMs`, `formatFps`. The tests cover the wrong-sign texts, the `followFocus` tie rules and the timeout notice.

frontend/src/components/live/StageOverlay.tsx + StageOverlay.module.css

Drawn over the live video (used by `CameraCard`): the `ActivationOverlay` (the reveal with a light band near the top, the `JutsuIcon`, the jutsu title and kanji, `data-testid="jutsu-overlay"`; then the small `jutsu-banner` saying "active" or "fading"), the `FailureToasts` (`failure-toast`) and, for Shadow Clone, a "Not enough room for all 10 clones" notice when `frame.effect.crowded` is true.

Older live components (kept; only used by the unrouted pages/PracticePage.tsx)

- `frontend/src/components/live/CameraStage.tsx` + `CameraStage.module.css`: the older camera stage (processed frames in a "chakra frame", local preview while not live, start/stop, camera picker, landmarks).
- `frontend/src/components/live/RecognitionPanel.tsx` + `RecognitionPanel.module.css`: what the AI sees (fused sign with source and confidence, hold progress, confirmed signs, hands, connection numbers).
- `frontend/src/components/live/SequenceTrack.tsx` + `SequenceTrack.module.css`: the older four-step track with a countdown.
- `frontend/src/components/live/ConnectionPill.tsx` + `ConnectionPill.module.css`: a status pill (used by CameraStage and RecognitionPanel).
- `frontend/src/components/live/AvailabilityNotice.tsx` + `AvailabilityNotice.module.css`: which jutsu this server can activate and why not.
- `frontend/src/components/live/JutsuProgressList.tsx` + `JutsuProgressList.module.css`: an older list of the four jutsu with their progress; not imported anywhere now.

Art components (src/components/art)

frontend/src/components/art/JutsuIcon.tsx + JutsuIcon.module.css

The jutsu icon used everywhere (jutsu list, sequence header, carousels, activation overlay). Props: `jutsu`, `size` (by default it fills its parent), `animated`, `className`. It is a round window with an SVG energy ring (a glow, a band, ticks spinning one way and arcs spinning the other) coloured by `--ring` / `--ring-hot` from `jutsuVisual()`. Inside: the `ShadowCloneIcon` for Shadow Clone, otherwise a `JutsuMotif` (flames, lightning or a sphere). The animations are CSS and respect the motion setting.

frontend/src/components/art/ShadowCloneIcon.tsx + ShadowCloneIcon.module.css

The approved Shadow Clone art cut into layers (`assets/konoha/shadow-clone/bg.webp`, `clone-left.webp`, `clone-right.webp`, `front.webp`), plus an orange aura, two smoke puffs (five circles each) and sparks. CSS animations move only `transform` and `opacity` (cheap for the browser): the clones grow out from behind Naruto in smoke, hold, then vanish. A `useLayoutEffect` sets `startTime = 0` on all of its animations, so every copy on the page shows the same moment of the loop.

frontend/src/components/art/JutsuMotif.tsx + JutsuMotif.module.css

Small original SVG motifs (`MotifKind`: `fire`, `lightning`, `sphere`, `smoke`) with the props `kind`, `color`, `color2`, `size`, `animated`.

frontend/src/components/art/jutsuVisuals.ts

`JUTSU_VISUALS`: per jutsu the colour (a CSS variable), a second colour, the motif, one decorative kanji (火, 雷, 影, 螺) and the ring colours. `jutsuVisual(id)` returns it, with a fallback for unknown ids.

Other art files (used only by the older AppShell, or not at all)

- `ChakraBackground.tsx` + `ChakraBackground.module.css`: the old dark animated backdrop (gradients, fog, `ParticleField`, vignette, grain).
- `ParticleField.tsx`: a canvas of embers (Naruto) or electric motes with faint lightning (Sasuke); it pauses when hidden or off-screen.
- `particles.ts` (+ `particles.test.ts`): the pure particle simulation (`particleCount`, `spawnParticle`, `stepParticle`, `particleOpacity`, `lightningBolt`, `seededRandom`); the tests check counts, spawning, rising embers, lifetime, fading and deterministic bolts.
- `HeroArt.tsx` + `HeroArt.module.css`: an old home-page artwork (a chakra seal circle); not imported now.
- `ShinobiLogo.tsx` + `ShinobiLogo.module.css`: the old "SHINOBI AI" mark (four blades around a core).
- `ThemeShift.tsx` + `ThemeShift.module.css`: a wave of light when the theme changes.

Older layout and UI kit (src/components/layout, src/components/ui)

frontend/src/components/layout/useDocumentTitle.ts

Used by every page: sets the tab title to `"<title> · NARUTO"`, or just `NARUTO`.

frontend/src/components/layout/AppShell.tsx, MobileMenu.tsx, ServerStatus.tsx, ThemeToggle.tsx, nav.ts (+ their .module.css files)

The previous dark app frame: `AppShell` (animated background, top bar, mobile drawer, footer), `MobileMenu` (a drawer with a kanji per item), `ServerStatus` (server reachability indicator), `ThemeToggle` (the Naruto / Sasuke switch) and `nav.ts` (`NAV_ITEMS` with labels, icons and kanji). `AppShell` is no longer used by the router (it was replaced by `KonohaShell`), so these files are currently inactive.

frontend/src/components/ui/ (index.ts and each component with its CSS module)

A small dark UI kit, re-exported by `index.ts`. It is still used by the About and 404 pages and by the unrouted legacy pages.

File	Purpose
`Button.tsx`	Button or router link, variants and sizes, optional icon
`Panel.tsx`	Dark card with title, icon, actions, optional glow
`Badge.tsx`	Small coloured label (tone, icon, dot)

File	Purpose
`Slider.tsx`, `Switch.tsx`, `SegmentedControl.tsx`, `Select.tsx`	Form controls
`SectionHeading.tsx`	Eyebrow, title and text heading
`Skeleton.tsx`	Grey loading placeholder
`StatusMessage.tsx`	`StatusMessage`, `ErrorState` and `EmptyState` boxes
`Lightbox.tsx`	Full-screen image viewer with previous / next
`SignArt.tsx`	Tinted sign drawing (old look)
`SequenceBadge.tsx`	"Official" / "Custom" badge
`Reveal.tsx`	Fade-and-rise animation on scroll
`ArtCredit.tsx`	Licence line of the drawings
`serverError.ts`	`describeServerError` (turns "answered 502" into a sentence) and `START_SERVER_HINT`

Older pages (src/pages)

- `frontend/src/pages/AboutPage.tsx` + `AboutPage.module.css` (routed at `/about`): what SHINOBI AI is, how the recognition works, the technologies, and the credits loaded from `GET /api/credits` (loading, error or ok state), with the disclaimer section shown separately.
- `frontend/src/pages/NotFoundPage.tsx` + `NotFoundPage.module.css` (routed as `*`): "This path leads nowhere", the unknown path, and buttons back home and to the library.
- `frontend/src/pages/PracticePage.tsx` + `PracticePage.module.css`, `LibraryPage.tsx` + `LibraryPage.module.css`, `SettingsPage.tsx` + `SettingsPage.module.css`: the previous dark versions of Practice, Library and Settings. They follow the same rules (backend-driven progress) but are **no longer routed**.
- `HomePage.tsx`, `LiveDemoPage.tsx`: described above (they render the Konoha views).

Styles and assets

- `frontend/src/styles/tokens.css`: design tokens of the older dark look (fonts, fluid type scale, colours per theme via `[data-theme]`, and the jutsu colours `--jutsu-fire`, `--jutsu-chidori`... which `jutsuVisuals.ts` still uses).
- `frontend/src/styles/global.css`: imports `tokens.css`, resets (`box-sizing`, margins), base page styles and the `.visually-hidden` class (text only for screen readers, used all over the app).
- `frontend/src/assets/konoha/background.jpg`: the village painting.
  `frontend/src/assets/konoha/shadow-clone/*.webp`: the Shadow Clone icon layers. Vite bundles them and gives them hashed URLs.
- CSS modules (`*.module.css`): each component imports `styles from './X.module.css'`; Vite renames the classes so they never clash between components.

Unit tests: how to run them

`npm test` (Vitest) runs every `src/**/*.test.ts` file in Node: `framing.test.ts`, `client.test.ts`, `labels.test.ts`, `sequenceSteps.test.ts`, `particles.test.ts`, `eventCues.test.ts`, `voiceCues.test.ts`, `jutsuMusic.test.ts` and `voicePlayback.test.ts`. They are fast because they test pure logic or use fakes (a fake WebSocket, fake timers, a fake AudioContext).

End-to-end tests (frontend/e2e)

These are **real** tests: a real Microsoft Edge (driven by `playwright-core`), the real backend and the real AI. Edge's fake camera plays a video built from the user's own sign recordings, so nothing is faked in the recognition: the backend must recognise every sign by itself. Each script prints `PASS` / `FAIL` lines and a JSON summary, and exits with code 1 if anything failed.

frontend/e2e/make_fixtures.py

Builds the fake-camera videos. Run it with the project's Python:

```
.venv/Scripts/python.exe frontend/e2e/make_fixtures.py --out <folder outside the repo>
.venv/Scripts/python.exe frontend/e2e/make_fixtures.py --out <folder> --only fire_style chidori
```

- The files contain the user's face, so the script **refuses** to write inside the repository.
- `FIXTURES` lists each fixture as segments `(sign or "idle", seconds)`. Examples: `fire_style` = idle, monkey, boar, horse, tiger, idle; `chidori`; `wrong_order` (monkey, boar, then dog instead of horse); `idle`; `practice_retry`; `practice_wait` (a 32 s pause); `live_timeout` (an 8.5 s pause, longer than the 7 s limit); `live_retry`, `live_tie`, `live_repeat`, `live_wrong_keep`, `live_retry_slow`.
- For each sign it reads the first `me_night_*.mp4` recording in `data/videos/<sign>/`. If the YOLOX seal detector can be loaded, `best_window` picks the part of the recording where YOLOX recognises that sign in the most frames (score at least `SEAL_MIN_SCORE`), so the test checks the pipeline, not a bad clip. Without YOLOX it takes the middle of the recording.
- Frames are **un-mirrored** (`cv2.flip`) because the recordings were saved mirrored and the server mirrors again; they are resized to 1280×720, converted to I420 and written as a `.y4m` file (a `YUV4MPEG2` header with the recordings' frame rate). Edge loops it as its camera.

frontend/e2e/lib.mjs

Shared helpers. Environment variables: `SHINOBI_WEB` (app URL, default `http://localhost:5173`), `SHINOBI_FIXTURES` (required: the folder with the `.y4m` files), `SHINOBI_E2E_OUT` (screenshots; default: the fixtures folder).

- `launchEdge(fixture, {headless, profile})`: starts Edge (`channel: 'msedge'`) with a persistent profile, camera permission and the flags `--use-fake-device-for-media-stream`, `--use-file-for-fake-video-capture=<file>`, autoplay allowed, plus flags that speed up a busy Windows machine.
- `recordSocket(page)`: listens to the page's `/ws` WebSocket and records every text message, every frame header (progress, time left, fps, ready) and every event, plus what the page sent.
- `openPage(context, route, viewport)`: opens the page, collects console errors and waits with `waitForRendering` until the page paints normally (10 animation frames within 1.5 s), because a fresh headless Edge can be frozen for a while.

- `startAndWaitLive(page)`: clicks `start-camera` and waits for `camera-stage` with `data-live="true"`.
- `recordActivations(context)`: an init script with a `MutationObserver` that records exactly when the activation overlay or banner appears and disappears, and every step count the sequence card shows (a polling loop could miss a 2 s reveal). `readMarks(page)` reads it back.
- `noHorizontalScroll`, `readStats`, `median`, `sleep`, `check` (records a PASS or FAIL), `summary`, `snap` (a screenshot with a long timeout; a failure is only logged).

The test scripts

Script	How to run	What it checks
`live-jutsu.mjs`	`node e2e/live-jutsu.mjs fire_style` (or `chidori`)	On `/live`: processed frames appear; the card shows steps 1, 2, 3; the backend sends `step` events and the `jutsu` event; the overlay follows that event within 3 s and collapses into the "active" banner; no horizontal scroll at 1440×900; no console errors
`live-wrong-order.mjs`	`node e2e/live-wrong-order.mjs`	With `wrong_order`: the backend sends `wrong_sign` (expected horse, got dog) **without** resetting (progress stays at 2); the UI shows "HORSE not recognized — that looked like DOG"; no `jutsu` event and no overlay
`practice.mjs`	`node e2e/practice.mjs fire_style`	`/practice/<jutsu>` starts at "Step 1 of 4", sends mode practice, shows steps 1 to 4 in order, the completion appears after the backend's `jutsu` event, only that jutsu's steps are reported, "Practise again" sends `reset`
`sequence-rules.mjs`	`node e2e/sequence-rules.mjs [case ...]`	The sequence rules case by case: `practice_retry` (wrong attempts are warnings, steps kept, amber ring, no countdown), `practice_wait` (a 30 s pause changes nothing), `live_timeout` (reset about 7 s after the last step, notice "Time ran out"), `live_wrong_keep`, `live_retry` (each wrong attempt gives the full 7 s again and **the card's countdown goes back up**, and only then), `live_retry_slow`, `live_tie` (the card stays on Rasengan), `live_repeat` (repeating the last sign does not restart the clock), `live_start_over`
`reconnect.mjs`	`node e2e/reconnect.mjs` (env `SHINOBI_PORT`, default 8000)	Kills the backend process on that port while live; checks the UI goes "reconnecting", then "offline", with no recognition shown; restarts the server (`python -m shinobi.server --port ...`); checks automatic recovery with a new session and the mode sent again
`two-browsers.mjs`	`node e2e/two-browsers.mjs`	Two Edge instances (Fire Style and Chidori recordings) get two different sessions, and each gets only its own jutsu
`responsive.mjs`	`node e2e/responsive.mjs live` (or `idle`)	Screenshots of `/live` and the practice pages at 1440×900, 1280×800, 1180×820, 820×1180, 390×844 and 360×740, and no horizontal scrolling at each size
`audio-timeline.mjs`	`node e2e/audio-timeline.mjs [fixture ...]`	Patches `fetch`, `decodeAudioData`, `AudioBufferSourceNode.start` and `WebSocket` inside the page to log every sound started and every backend event; checks that every sound decoded, one tone plus accent per confirmed sign, wrong signs are silent, no sound plays without its event, the release sounds played, and the order confirmation, charge, name, release

Script	How to run	What it checks
`music-flow.mjs`	`node e2e/music-flow.mjs [case ...]`	Records the Web Audio graph and the gain automations: a track starts only while its jutsu leads, with a 1.5 s fade-in or a 0.8 s crossfade; the right track plays at completion; duck to 25 % at the confirmation; a 1.5 s fade-out at the release; one seal per sign; `wrong_keep` keeps the music going; `camera_stop` fades it out

Running the e2e tests (summary)

1. Build the fixtures once with `make_fixtures.py --out <scratch folder>`.
2. Start the backend (`python -m shinobi.server`) and the dev server (`npm run dev` in `frontend/`).
3. Set `SHINOBI_FIXTURES` to the fixtures folder (and optionally `SHINOBI_WEB` and `SHINOBI_E2E_OUT`).
4. Run a script with Node from `frontend/`, for example `node e2e/live-jutsu.mjs fire_style`. Microsoft Edge must be installed.

Note: `frontend/e2e/__pycache__/` only holds Python's compiled cache of `make_fixtures.py` and can be ignored.

Part 4 — Data, training, evaluation, tools and tests

This part explains everything around the "brain" of SHINOBI AI: how hand-sign data is recorded, where it is stored, how it is split into training, validation and test data, how the landmark classifier is trained, how the whole recognition pipeline is measured, what every helper tool in `tools/` does, what every test module checks, and where the project stands today. It ends with a list of exact commands to run every part of the project on Windows.

The other parts of this documentation explain the app itself (camera, MediaPipe hands, features, fusion, controller, effects, server and web frontend). Here those modules appear only as building blocks that the data and evaluation scripts call.

The big picture in one page

The app recognises a hand sign in two ways at once:

- **YOLOX** (`models/yolox_nano.onnx`) is a pretrained hand-seal detector. It was trained by someone else on other people's photos and is never trained by this project. It "owns" the seven official seals: ram, snake, tiger, dog, monkey, boar, horse.
- **The landmark classifier** (`models/gesture_model.joblib`) is trained by this project with `train.py`. It reads the 3D hand landmarks that MediaPipe finds and outputs a probability for each of the ten gestures in `config.GESTURES`. In the app it decides only the two custom gestures: `clone_seal` and `rasengan_palm` (`config.LANDMARK_SIGNS`).

`shinobi/fusion.py` (SignFusion) combines both answers per frame, and `shinobi/controller.py` (ActionController) turns a stream of signs into jutsu activations.

The data and machine-learning workflow is:

1. Record sessions with the webcam (`collect_data.py`, or the guided tools `tools/record_training_session.py` and `tools/record_test_session.py`). Each session is one `.npz` file of raw hand landmarks plus one small `.mp4` reference video.
2. Give every recording exactly one set (train, validation or test) in `data/splits.json`. This is done by recording, never by frame.
3. Train the landmark classifier on the TRAIN recordings only (`train.py`). The trained model remembers which recordings it learned from.
4. Measure the whole pipeline (landmarks, YOLOX and the fused app decision) on recordings it never learned from (`evaluate.py run`), and write a report to `reports/<date_time>/`.
5. Optional deeper studies: condition-grouped cross-validation (`tools/grouped_cv.py`) and the extra measurements of `tools/evaluate_pipeline.py`.

Three words you must understand first

- **Recording (session):** one 20-second take of one gesture. Its id is its file name without `.npz`, for example `me_day_20261101_162853` = person `me`, lighting `day`, date 2026-11-01, time 16:28:53.

- **Leakage:** neighbouring frames of one recording are almost identical. If some frames of a recording are used for training and other frames of the same recording for testing, the model is "tested" on near-copies of what it memorised, and the score looks much better than reality. That is why the unit of every split in this project is a **whole recording**.
- **Train / validation / test:** TRAIN recordings teach the model. VALIDATION recordings are used to choose between models and to tune settings. TEST recordings are touched only once, at the end, to report an honest number. Nothing is ever tuned on the test set.

data/ — the data layout

All data lives in `data/` (the path is `config.DATA_DIR`). Nothing in `data/` is ever moved, renamed or deleted by the scripts; the only file the tools write in `data/` is `data/splits.json` (and new recordings, written by the recorders).

path	what it holds
`data/landmarks/<label>/<id>.npz`	raw hand landmarks of one single-sign recording
`data/videos/<label>/<id>.mp4`	the reference video of the same recording (480p, at most 15 fps)
`data/sequences/<jutsu>/<id>.npz`	guided four-sign takes from `record_sequences.py` (this folder does not exist yet: no takes were recorded)
`data/videos/sequences/<jutsu>/<id>.mp4`	the videos of those takes
`data/_discarded/`	two old recordings (two open palms recorded under a wrong name), kept only as "never trained" negative examples
`data/splits.json`	which set each recording belongs to
`data/evaluation_exclusions.json`	test recordings left out of the scores because their content does not match their label

data/landmarks/<label>/*.npz

Today there are 13 label folders: `boar`, `chidori`, `clone_seal`, `dog`, `fire`, `horse`, `idle`, `monkey`, `ram`, `rasengan`, `rasengan_palm`, `snake`, `tiger` (77 files in total).

- `rasengan` is the **old name** of `rasengan_palm`. `config.LEGACY_LABELS = {"rasengan": "rasengan_palm"}` translates it everywhere, so these four files are trained as `rasengan_palm`.
- `chidori` (claw hand) and `fire` (OK sign) are labels of an older design. They are no longer signs, so `splits.py` lists them as "ignored" and they are never trained or scored. `tools/evaluate_pipeline.py` reuses them as never-trained negative clips.

An `.npz` file is a compressed NumPy archive. A file with N frames contains:

key	shape	type	meaning
`image_lm`	(N, 2, 21, 3)	float32	per frame, per hand slot (2), per landmark (21): x, y in 0..1 of the picture, plus MediaPipe's relative z
`world_lm`	(N, 2, 21, 3)	float32	the same 21 points in metres, in MediaPipe's 3D "world" coordinates
`present`	(N, 2)	float32	1.0 if a hand was found in that slot, else 0.0 (that slot's landmarks are then zeros)
`timestamps`	(N,)	float	seconds since the recording started

key	shape	type	meaning
`meta`	()	JSON string	`label`, `session` (the id), `person`, `lighting`, `aspect` (width / height of the camera picture); the guided recorders add more, e.g. `camera`, `camera_note`, `planned_set`, `recording_session`, `block`, `note`

Example from a real file (`ram/me_day_20261101_162853.npz`): `image_lm` (350, 2, 21, 3), `present` (350, 2), meta `{"camera": "A", "planned_set": "train", "recording_session": 1, "block": 1, "label": "ram", "person": "me", "lighting": "day", "aspect": 1.777...}`.

Why raw landmarks and not features? The features (joint angles, normalised fingertip positions, two-hand geometry; `shinobi/features.py`) may change. Because the files keep the raw landmarks, `train.py` can recompute features from old recordings at any time. The conversion from MediaPipe hands to these arrays is `shinobi.features.hands_to_arrays`, the same function the live app uses, so training data and live data have exactly the same format.

Guided sequence takes (`data/sequences/...`, from `record_sequences.py`) have the same keys plus three per-frame label arrays: `frame_phase` (int8, index into `lead_in / hold / transition / tail`), `frame_step` (int8, step number during a hold, else -1) and `frame_sign` (int16, index into `config.GESTURES`, `idle` for hands-down, -1 while changing hands).

data/videos/

One `.mp4` per recording, same id and same label folder as its `.npz`. It is resized to 480 pixels high and written at most 15 times per second (in practice about 10 frames per second). The videos have two jobs:

- `evaluate.py` runs YOLOX on them, because YOLOX needs pictures and the `.npz` holds only landmarks.
- People can watch them to check that a recording really shows its sign (the diagnosis did exactly this).

A video frame is written on the same loop pass as a landmark frame whenever at least 1/15 s passed since the last video frame. `evaluate.py` replays that rule on the saved timestamps to know which landmark frame belongs to which video frame (see `saved_video_frames` below).

data/splits.json

This file decides which recordings may be used for what. Format `shinobi-splits-v1`:

```
{
  "format": "shinobi-splits-v1",
  "about": "...",
  "quotas": {"single": {"train": 3, "validation": 1, "test": 1},
             "take":   {"train": 0, "validation": 1, "test": 1}},
  "updated": "2026-10-31T02:03:19",
  "sessions": {
    "me_night_20261030_221934": {"set": "train", "kind": "single", "group": "idle",
       "label": "idle", "path": "landmarks/idle/me_night_20261030_221934.npz",
       "person": "me", "lighting": "night", "date": "2026-10-30", "time": "22:19:34",
       "assigned_by": "auto", "assigned_on": "2026-10-31T02:03:19"}, ...
  },
  "ignored": {"me_day_20261030_175555": {"path": "...", "label": "chidori", "reason": "..."}, ...}
}
```

- The key of each entry is the recording id, so a recording keeps its set even if its file is moved.
- `set` is `train`, `validation` or `test`. `kind` is `single` (one sign held) or `take` (a guided sequence).

- `assigned_by` is `auto` (chosen by `splits.py`'s rules) or `manual` (put there by hand, by `--assign`, `tools/make_test_set.py` or `tools/assign_recordings.py`).
- Entries are **sticky**: once a recording has a set, nothing moves it automatically.

Today the file holds 68 entries: **train 43** (16 auto from 2026-10-30, 27 manual from the guided session of 2026-11-01), **validation 3** (auto: two idle and one old-name `rasengan` palm, all from 2026-10-30), **test 22** (manual, the independent test session of 2026-11-01 from 14:04). Eight recordings (four `chidori`, four `fire`) are listed as ignored. One recording on disk has no set at all: the rejected snake `me_day_20261101_140757` (0% of frames with hands). Because it has no set, `train.py` and `evaluate.py` never use it.

Never run `python evaluate.py splits` on this project's real data. That command assigns every recording that has no set yet, automatically, using the quotas. It would, for example, give the rejected snake recording a set (possibly train or test), and it rewrites the file. All current assignments were made on purpose, by hand and by id, with a dry run first. New recordings must be added with `tools/assign_recordings.py` (train / validation) or `tools/make_test_set.py` (test), never with `evaluate.py splits`. The command itself is documented below because it exists in the code.

data/evaluation_exclusions.json

Format `shinobi-exclusions-v1`. It lists recordings that stay in their set (nothing is deleted or moved) but are **left out of the scored metrics** because their content does not match their label. Every entry has `set`, `label`, `reason`, `evidence` (a picture in the diagnosis folder), `decided_on` and `decided_by` (the user, on 2026-11-01, after the diagnosis). The five excluded recordings are all in the test set:

recording	label	why it is excluded
`me_day_20261101_140631`	ram	partial: Ram only in the first ~9 s, then no hands (10–14 s) and a hand on the mouth (14–20 s); recorded by the old recorder version that ran ahead of the user
`me_day_20261101_140826`	tiger	no Tiger at any time (phone in hand, touching the face); same old recorder version
`me_day_20261101_141753`	idle	meant to be an OK sign, but shows open palms with spread fingers, i.e. the Rasengan palm
`me_day_20261101_141718`	rasengan_palm	two open palms in most of the recording (two hands in 83% of frames); the agreed palm is ONE open palm, fingers spread
`me_day_20261101_142407`	rasengan_palm	the same two-palm problem

`evaluate.py run` reads this file by default. It still evaluates the excluded recordings one by one, but every aggregate number is given twice: "valid recordings only" (the result) and "all recordings" (for comparison only). `tools/assign_recordings.py` refuses to put any excluded id into train or validation.

models/ — every file

file	what it is	made by
`gesture_model.joblib`	the landmark classifier bundle (see below)	`train.py`
`training_report.txt`	the text report `train.py` printed when it made the installed model	`train.py`

file	what it is	made by
`confusion_validation.png`	confusion matrix of the final model on the validation recordings	`train.py`
`hand_landmarker.task`	MediaPipe's hand-landmark model (downloaded)	Google MediaPipe
`selfie_segmenter.tflite`	MediaPipe's person segmentation model, used by the Shadow Clone effect	Google MediaPipe
`yolox_nano.onnx`	the pretrained YOLOX-nano hand-seal detector, run with onnxruntime on the CPU	third party
`THIRD_PARTY_YOLOX.md`	where the YOLOX model comes from and how it was checked	project notes
`YOLOX_NARUTO_HandSignDetection_LICENSE.txt`	the licence of the YOLOX model	third party

`gesture_model.joblib` is a Python dictionary saved with `joblib`. When trained with `data/splits.json` it contains:

- `model`: the fitted scikit-learn model (a RandomForestClassifier, or a StandardScaler + MLPClassifier pipeline);
- `model_name` (`random_forest` or `mlp`), `labels` (the gesture names in the order of the model's classes), `feature_version` (must match `shinobi.features.FEATURE_VERSION`);
- `selection` (how the model type was chosen: `validation`, `cv` or `default`), `selection_macro_f1`, `cv_macro_f1`, `validation_macro_f1`;
- `splits_file`, `train_sessions`, `validation_sessions` (the exact ids it learned from and was chosen with), `held_out_person`, `created`.

The lists `train_sessions` and `validation_sessions` are the leakage guard: `evaluate.py` and `tools/make_test_set.py` read them and refuse to score the model on a recording it learned from. The installed model today: Random Forest, created 2026-10-31T14:36:03, 16 train sessions, `selection: default`, validation macro-F1 1.000 (but validation had only idle and palm frames).

requirements.txt

The Python dependencies, pinned for Python 3.12:

package	why
`mediapipe==1.2.0`	hand landmarks and person segmentation; it installs `opencv-contrib-python` itself
`opencv-contrib-python==5.0.0.93`	camera, drawing, video files (do NOT also install `opencv-python`: both provide `cv2` and break each other)
`numpy==2.5.4`	arrays everywhere
`scikit-learn==1.9.1`	Random Forest, MLP, metrics, StratifiedGroupKFold
`onnxruntime==1.31.1`	runs the YOLOX detector on the CPU
`fastapi==0.162.0`, `uvicorn==0.54.0`, `websockets==17.4`	the web server for the React app
`httpx==0.28.1`	only for the tests (FastAPI's TestClient)
`joblib`, `matplotlib`, `pytest`	model files, report plots, tests

Note: `tools/check_voices.py` and `tools/build_bold_signs.py` import `scipy`, and `tools/build_gesture_references.py` imports `PIL` (Pillow). They are not pinned here; they come in as dependencies of the other packages or must be installed separately if missing.

README.md

The short English front page of the project. It explains the pipeline in one diagram (camera → MediaPipe hands → features (206) → classifier → controller → effect), the main ideas (per-frame classifier plus a rule-based controller instead of an LSTM; cross-validation split by recording; mirror and jitter augmentation only inside training folds), a quick start for Windows with Python 3.12 (create `.venv`, install requirements, run `shinobi.main`, `collect_data.py`, `train.py`, `pytest`), and points to the full Arabic documentation in `docs/`.

collect_data.py

Purpose: the original, simple recorder. It opens the webcam, lets you choose a gesture with a number key, and records one 20-second session per SPACE press.

Run: `python collect_data.py --person sara --lighting day`. Options: `--person` (who records; used later to tell people apart), `--lighting` (e.g. day, night, lamp), `--seconds` (default `config.RECORD_SECONDS` = 20), `--no-video`.

Keys: 0–9 choose the gesture (index in `config.GESTURES`), SPACE starts a recording, Q or Esc quits.

Main functions:

- `count_sessions()` counts the `.npz` files per gesture, including files under old names (`LEGACY_LABELS`), for the on-screen list.
- `load_sign_pictures(height=220)` loads the reference drawing of each sign from `gesture_references/_sources/signs/<label>.png` and scales it for the corner of the screen (`idle` has no picture). `paste_picture(view, picture)` draws it top-right on a white card. Both are reused by the guided recorders.
- `save_session(label, args, frames, video_frames, aspect)` builds the id `<person>_<lighting>_<YYYYMMDD_HHMMSS>`, writes `data/landmarks/<label>/<id>.npz` with `np.savez_compressed`, writes the video with `cv2.VideoWriter` (mp4v, 15 fps) and prints how many frames had hands.
- `main()` is a small state machine: `idle` → `countdown` (`config.COUNTDOWN_SECONDS` = 3) → `recording` → back to `idle`. During recording every new camera frame is passed to `HandTracker.detect`, converted with `hands_to_arrays`, and appended with its timestamp; a 480p video frame is kept at most every 1/15 s.

Key details:

- Keys are read at the top of every loop pass, before waiting for a frame, so a key pressed while the camera is slow is not lost.
- The countdown is not saved. Recording time 0 is the first recorded frame. That is why `evaluate.py` skips only the first 0.5 s.
- The screen tells you to "move, rotate, change distance" while recording, so one session covers some variety.

record_sequences.py

Purpose: record (or just practise) guided four-sign **jutsu sequences**, including deliberately wrong ones, so the controller can later be tested on real sign sequences.

Run: `python record_sequences.py --person me --lighting day`. Options: `--hold`, `--transition`, `--lead` (seconds, defaults `config.SEQ_HOLD_SECONDS`, `SEQ_TRANSITION_SECONDS`, `SEQ_LEAD_SECONDS`, all 1.5), `--no-video`.

Keys: 1–4 choose the jutsu; TAB changes the kind of take (`correct`, `wrong_order`, `incomplete`, `wrong_sign`); R draws new random signs for a negative take; SPACE records; P practises (nothing saved); `[` and `]` slow down or speed up; C cancels a running take; Q quits.

Main pieces:

- `Segment` (dataclass): `phase` (`lead_in`, `hold`, `transition`, `tail`), `sign`, `step`, `start`, `end`.
- `build_timeline(signs, hold, transition, lead)`: hands down → sign 1 → change → sign 2 → … → last sign → hands down. `segment_at(timeline, t)` returns the segment running at time t (None at the end).
- `fires(signs)` runs the app's real `SequenceMatcher` on a list of signs and returns the jutsu that would fire.
- `make_signs(kind, jutsu_key, rng)`: for `correct` the jutsu's sequence; for the negative kinds it shuffles the order, cuts the sequence after 2 or 3 steps, or replaces one sign, and **redraws up to 500 times until `fires()` returns nothing**. So a negative take can never accidentally complete a jutsu.
- `frame_labels(segments)` builds the per-frame `frame_phase`, `frame_step`, `frame_sign` arrays; `save_take(path, frames, meta)` writes them with the landmarks.
- `take_quality(frames, signs)`: share of hold frames with any hand, and with both hands for two-hand signs.
- Drawing helpers `draw_strip` (cards of the signs along the bottom; done ones faded, the current one with a green border), `draw_timeline` (coloured bar of the phases with a moving cursor), `banner`, `prompt_text`.

The saved meta (`format: shinobi-sequence-v1`) stores the whole timeline, the signs, the kind and `expected_activation` (the jutsu for a correct take, `None` for negatives). The docstring warns that people react 0.3–0.8 s after a prompt, so code using these labels should trust the second half of each hold window more. The video is written while recording (no memory build-up), and a cancelled or interrupted take deletes its partial video.

No takes have been recorded yet (`data/sequences/` does not exist), so sequence-level evaluation currently uses spliced single-sign recordings instead (`tools/evaluate_pipeline.py sequences`).

splits.py

Purpose: the library behind `data/splits.json`. It scans recordings, reads and writes the file safely, assigns new recordings to sets with leakage-aware rules, and prints a status table. It never writes, moves or deletes a recording; it writes only `data/splits.json`.

Constants: `SETS = ("train", "validation", "test")`, `FILL_ORDER = ("train", "test", "validation")`, `FORMAT = "shinobi-splits-v1"`, `QUOTAS` (single: 3 train / 1 validation / 1 test per sign; take: 0 / 1 / 1 per jutsu + kind, because `train.py` does not train on takes).

`Recording` (dataclass): `id`, `kind` (`single` / `take`), `group` (the sign after `LEGACY_LABELS`, or `<jutsu>/<kind>`), `label`, `path`, `person`, `lighting`, `date`, `time`, `signs`. Its property `condition` is `(date, lighting)`: two recordings with the same condition were made on the same day in the same light.

Reading:

- `read_meta(path)` opens only the small `meta` entry (the frames are not decompressed).
- `_when(stem, meta)` takes date and time from the `_YYYYMMDD_HHMMSS` ending of the id (or from `created` for takes).
- `scan(data_dir)` lists every recording in `landmarks/*/*.npz` and `sequences/*/*.npz`. A broken file becomes a note, not a crash. A label that is no longer a sign goes to `ignored`. If the label stored in the file differs from its folder, the file's label wins (like `train.py`) and a note is printed. Two files with the same id raise `SplitsError`, because the id is the key.

The file:

- `load(path)` returns None if the file is missing; it rejects invalid JSON and a key that appears twice (`_no_duplicate_keys`), and `check()` verifies the format and that every `set` is valid.
- `save(splits, path)` checks, writes to `splits.json.tmp`, then `os.replace` swaps it in: a crash can never leave half a file.
- `set_of(splits)` returns `{id: set}`.

Assigning (only recordings without a set):

- `conditions(sessions, by_id)` collects, per (kind, group), the conditions already in each set.
- `train_conditions(...)` gives the day + lighting conditions a TEST recording must avoid: those of train recordings of the same group, and for a take also those of the train sessions of its signs.
- `choose_set(rec, conds, quota)` applies the rules. Train is allowed only if the recording's condition is not used by a test recording of that group; test is allowed only if the condition is known and not used by train. Then the minimum quotas are filled in the order train, test, validation; after that the set furthest below its proportion gets it.

```
allowed = {
    "train": c is None or c not in set(mine["test"]),       # never train on a test day + light
    "test": c is not None and c not in train_conditions(rec.kind, rec.group, rec.signs, conds),
    "validation": True,
}
n = {s: len(mine[s]) for s in SETS}
for s in FILL_ORDER:                                        # minimum quotas: train, test, validation
    if allowed[s] and n[s] < quota[s]:
        return s
```

- `update(existing, recordings, ignored, quotas)` copies every existing entry unchanged ("hand edits win"), assigns the new ones oldest first, sorts the entries and returns `(new splits, ids assigned now)`.
- `assign(splits, recordings, rid, set_name)` puts one recording into a set by hand (`assigned_by: manual`) and returns the previous set.

Checking and printing:

- `review(splits, recordings)` returns rows and warnings per group: missing quotas, a test recording that shares day + lighting with training data ("test numbers will look better than reality"), recordings without a known day or lighting, entries whose file is gone.
- `table(...)` builds the text that `python evaluate.py splits` prints.

Why "test from another day or lighting"? A test recording made in the same sitting as the training recordings measures memory of that sitting, not the ability to work in new conditions.

train.py

Purpose: train the landmark classifier and save it as `models/gesture_model.joblib`, together with `models/training_report.txt` and a confusion matrix PNG.

Run: `python train.py`. Options: `--data` (default `data/landmarks`), `--out` (default `models/gesture_model.joblib`), `--step` (use every n-th frame, default 2), `--test-person NAME` (also hold out one person completely), `--splits` (default: `splits.json` next to the data folder).

If `data/splits.json` exists (it does), `train_with_splits` is used. Without it, the older "all sessions" mode runs.

Loading frames

```
def load_sessions(data_dir, step, only=None):
    for path in sorted(Path(data_dir).glob("*/*.npz")):
        if only is not None and path.stem not in only:
            continue                                  # other files are not even opened
        d = np.load(path)
        meta = json.loads(str(d["meta"]))
        meta["label"] = config.LEGACY_LABELS.get(meta["label"], meta["label"])
        if meta["label"] not in config.GESTURES:
            continue                                  # e.g. old "chidori" / "fire"
        keep = d["present"].any(axis=1)               # frames where a hand was found
        idx = np.flatnonzero(keep)[::step]            # every step-th frame
```

Only frames with at least one hand are used, and only every second one (neighbours are near-duplicates and would only slow training down). With `only=...` the test files are never even opened.

Features and augmentation

`build(sessions, rng)` computes, for every frame, three feature vectors with `shinobi.features.extract`:

- the **original** frame;
- a **mirrored** copy (`features.mirror`: left and right swapped), so a sign learned with the right hand on top also works mirrored;
- a **jittered** copy: Gaussian noise of 0.004 on the picture coordinates and 0.002 m on the world coordinates, only on hands that are present, so the model tolerates MediaPipe's small tremble.

It also returns the label index, the session id (the group for cross-validation) and the person. `augmented(idx, ...)` stacks original + mirrored + jittered rows of the chosen frames. Augmentation is applied **only to the training part** of any split; the frames that are scored are always originals (`features_only`). Otherwise a mirrored copy of a test frame could sit in training.

The two candidate models

`make_models()` returns:

- `random_forest`: `RandomForestClassifier(n_estimators=300, min_samples_leaf=2, class_weight="balanced", n_jobs=-1, random_state=0)`. `class_weight="balanced"` makes rarer gestures count as much as frequent ones.

- `mlp`: `StandardScaler` + `MLPClassifier(hidden_layer_sizes=(128, 64), alpha=1e-3, max_iter=400, early_stopping=True, random_state=0)`. The scaler is needed because neural networks train badly on unscaled features.

train_with_splits: how the model is chosen and trained

1. Read `data/splits.json` with `splits.load` (a broken file stops everything: "Nothing was trained."). `sessions_by_set` sorts the files into train, validation, test, ignored and unassigned **by file name only**. Test files are counted but never loaded; unassigned files trigger a warning and are not used.
2. Load TRAIN and VALIDATION sessions. With `--test-person`, that person's sessions are removed from both and kept apart.
3. Refuse to train without `idle` in train: idle teaches the model what NOT to react to.
4. Choose the model type (three cases, explained in the table below).
5. The final model is fitted on TRAIN only (with augmentation). Validation stays an honest check and is reported (`validation_block` writes a classification report and `confusion_validation.png`).
6. Save the bundle with the lists of train and validation session ids, and write `training_report.txt`.

How step 4 chooses:

condition	what happens	`selection`
validation contains every gesture that train has	both models are fitted on train; the higher validation macro-F1 wins	`validation`
otherwise, every gesture has at least 2 train sessions	`StratifiedGroupKFold` cross-validation by session, on train only	`cv`
otherwise	a warning is printed and the default (Random Forest) is used	`default`

The original model (2026-10-31) was made in the third case: validation had only idle and palm, and most gestures had one train session, so the Random Forest was used by default. The current production model (the MLP, 2026-11-01) was trained with the new `--model-type mlp` option, which trains only that model.

Macro-F1 is the average of the F1 scores of the gestures, each gesture weighing the same. It is used because plain accuracy would be dominated by gestures with many frames (idle, palm).

The mode without splits

`main()` without a splits file builds all sessions, requires at least 2 sessions per gesture, runs `StratifiedGroupKFold` with `groups = session`, keeps out-of-fold predictions, saves a confusion PNG per model, picks the best macro-F1 and fits it on everything (minus `--test-person`). This is the older behaviour, kept for completeness.

Connections

`train.py` imports `splits` and `shinobi.features`. `tools/grouped_cv.py` and `tools/evaluate_pipeline.py` reuse `load_sessions`, `build`, `make_models`, `augmented` and `save_confusion`, so every study trains exactly like `train.py`. The app loads the bundle through `shinobi/classifier.py` (GestureClassifier).

evaluate.py

Purpose: measure how well SHINOBI AI recognises signs and jutsu on recordings it never learned from, using the app's own code (nothing re-implemented), and write a readable report.

Two subcommands:

- `python evaluate.py splits` assigns sets to new recordings (`cmd_splits`). Do not run it on this project's data (see `data/splits.json` above).
- `python evaluate.py run` evaluates recordings and writes `reports/<YYYY-MM-DD_HHMMSS>/summary.md`, `metrics.json` and `confusion_*.png` (`cmd_run`).

Options of `run`: `--sets` (default `test`; or `validation`, `train`, a comma list, `all`), `--sessions` (only these ids), `--recognizers` (default `landmarks,yolox,fused`), `--model` (or `none`), `--exclusions` (default `data/evaluation_exclusions.json`, or `none`), `--skip-start` (default 0.5 s), `--yolox-delay` (default 0.06 s), `--out`, `--title`, `--note`.

The three recognisers

- **landmarks:** the GestureClassifier alone on the recorded landmarks, only on frames where a hand was found.
- **yolox:** YOLOX alone on the saved video frames, through the app's own `SealWorker`. A frame's answer is the best detection of a seal that reaches `SEAL_MIN_SCORE` (0.70), else `idle` (`yolox_sign`). For gestures YOLOX has no class for (idle, clone_seal, rasengan_palm) the right answer is `idle` (`yolox_truth`).
- **fused:** the whole recording replayed in time through `SignFusion` and `ActionController`, with the same calls as `shinobi/engine.py`. This is what the app would actually decide.

Main functions

- `load_recording`, `video_file`, `read_video`: read one recording and its video.
- `saved_video_frames(timestamps, fps=15)` replays the recorders' rule ("write a video frame when at least 1/15 s passed") on the timestamps; `align_video` uses it and falls back to spreading frames evenly in time if the counts differ (reported as "approximate").
- `hands_from_arrays(...)` rebuilds the `Hand` objects the tracker would have returned, in pixels of the video frame (handedness and score are not stored and not needed).
- `SealRunner` wraps the real `SealWorker`: it submits one frame and waits for that exact result, so the replay, not the wall clock, decides the timing.
- `seal_schedule(frame_times, rate_hz=SEAL_RATE_HZ, delay=0.06)` simulates when the app's YOLOX thread would have run: at most `SEAL_RATE_HZ` times per second on the newest frame, each result usable 0.06 s after its frame (the measured median run time).
- `landmark_probs(...)` runs the classifier on every frame with a hand and measures the time per frame.
- `replay(times, hands, probs, lm_labels, seal_runs)` feeds every frame through `fusion.update(...)` and `controller.update(...)` and records the fused sign, its source, confirmed signs, fired jutsu and timings.
- `frame_metrics(truth, pred, labels)` computes accuracy, balanced accuracy (mean recall), macro-F1 and per-class precision / recall / F1. Precision is None for a class never predicted; macro averages use only classes present in the truth.

- `confusion`, `latency` (n, mean, median, p90, max), `take_outcome` (`success`, `missed`, `wrong jutsu`, `fired twice`, or for negative takes `ok` / `false activation`).

The leakage guard

```
def model_status(path):
    ...
    bundle = joblib.load(path)
    if "train_sessions" not in bundle:
        info["reason"] = ("it was trained without data/splits.json, so it may have learned from ANY "
                          "recording, test ones included. Re-train ... and evaluate again.")
        return info, None
```

A model without the list of its training recordings is not used at all. In `evaluate_recording`, if a recording is in the model's `train_sessions`, the landmark and fused numbers are skipped for it. `cmd_run` also warns if splits.json was edited after training (a train recording now in test, or a validation recording now in test).

What evaluate_recording does for one recording

1. Load landmarks, timestamps, aspect. Frames before `--skip-start` (0.5 s) are not scored for single-sign sessions: the hands are already in place at time 0 but fusion needs `SEAL_CONFIRM` YOLOX results before it can name a seal.
2. Landmarks: predict every frame with a hand and add (truth, prediction) to the store.
3. YOLOX: read the video, align it, run YOLOX on every video frame, score the frames after the skip.
4. Fused: replay with the YOLOX results on the app's schedule; record which signs the controller confirmed, when the right sign was first confirmed, wrong confirmations, and any jutsu fired (holding one sign must never fire a jutsu).
5. For takes: the outcome (did the right jutsu fire exactly once, nothing on negatives) and the latency after the last sign.

Aggregation and report

`frame_level_results` computes metrics and confusion per set and recogniser. Excluded recordings are evaluated into a separate store, so `frame_level` is "valid recordings only" and `frame_level_all_recordings` includes them. `aggregate_takes` gives per-jutsu activation rates and false activations; `aggregate_streams` counts jutsu fired during single-sign sessions (should be 0). `_caveats` adds honest limits automatically: signs without a test recording, signs whose only test recordings are excluded, test sharing day + light with train, signs with only one evaluated recording ("smoke test"), all recordings from one or two conditions, and that YOLOX sees the 480p mp4 rather than the live 720p frame.

`write_report` writes `metrics.json` and `summary.md` (sections: what was measured, results at a glance, details per set and recogniser with per-sign tables and confusion matrices, every recording, jutsu level, warnings, how to read the report) and the confusion PNGs (`save_confusion_png`: one blue colour ramp, colour = share of the row, the right answer outlined). A report folder is never overwritten (a `_1`, `_2` suffix is added).

tools/recording_review.py

Purpose: the "six-frame review" shown after every guided recording, so the user decides whether to keep it. Nothing is saved before the answer.

- `pick(n_video, shots=6)`: six video frame indices spread from 5% to 95% of the recording.
- `thumbnail(frame, image_lm, present, width)`: a smaller video frame with the hands MediaPipe found drawn on top (green for slot 0, magenta for slot 1).
- `sheet(video_frames, video_idx, frames, title, lines, picture, size)`: the review screen: title and notes at the top, the reference drawing on the right, six thumbnails in two rows of three with their time stamps, and the key help at the bottom.
- `ask(window, screen)`: waits for a key: ENTER or K → `keep`, R → `redo`, Q or Esc → `quit`; closing the window counts as `quit`.

Why show MediaPipe's hands and not the AI's guess? The decision to keep a recording must not depend on the model that will be trained or tested with it. The review exists because two flawed test recordings (ram, tiger) were saved by an older recorder that only checked that "a hand was seen".

tools/record_training_session.py

Purpose: guided recording of TRAINING and VALIDATION data in up to three sessions (68 recordings planned), with camera-position blocks and the six-frame review.

Run: `python tools/record_training_session.py --session 1 --plan` (print the plan, no camera), `--session 1` (record), `--resume reports/training_sessions/<log>.json` (continue a stopped session). Other options: `--person`, `--lighting` (required for session 3), `--seconds`.

The plan:

- `WEAK = ["ram", "snake", "tiger", "dog", "clone_seal", "rasengan_palm"]` are recorded in every block; `STRONG = ["monkey", "boar", "horse"]` only in some blocks; every block ends with one `idle` recording with its own idle variant (OK sign, fists and claws, pointing and touching the face, phone use, relaxed hands, half-formed seals ...; never a spread open palm).
- `CAMERAS` describes six camera positions: `A` (face level, in front), `A-far` (sitting back), `A-side` (turned 15–20°), `B` (low, looking up), `B-near`, `C` (30° to the side).
- `SESSIONS`: session 1 = train, day light, blocks A, B, A-far; session 2 = train, evening lamp, blocks A, C, B-near; session 3 = validation, another light, blocks A-side, B. In total 51 train + 17 validation = 68.
- `EXTRA_HINTS`: clone_seal must keep both hands inside the picture; rasengan_palm is "ONE hand only", alternating RIGHT (odd block) and LEFT (even block).

`plan(session)` returns the numbered steps. `needed_hands(label)` returns the minimum share of frames with hands: 0.6 for signs, 0.3 for idle.

The flow in `main()`:

1. Create or resume the log `reports/training_sessions/session<N>_<start>.json` (`saved`, `redone`, `not_saved`, `finished`).
2. Before each block, `setup()` shows where to put the camera and the light, and waits for SPACE.
3. Before each recording, a 6-second countdown that runs **only while hands are visible** (SPACE pauses). This fixes the old problem of a recorder running ahead of the user.
4. Record 20 s (landmarks + 15 fps video, remembering which landmark frame each video frame belongs to).
5. If hands were seen in too few frames, it is never kept and is recorded again.
6. Otherwise the six-frame review: keep → `save_new(...)` from `record_test_session.py` (never overwrites, extra meta such as camera, planned set, session, block); redo → record again; quit → stop.

Nothing is put into a set here. After a session, `tools/assign_recordings.py` assigns the kept recordings by id. Only session 1 was recorded (27 recordings on 2026-11-01, 16:28–17:49); the user cancelled sessions 2 and 3.

tools/record_test_session.py

Purpose: guided recording of an INDEPENDENT test session where the user only performs the gestures.

Run: `python tools/record_test_session.py --person me --lighting day` (`--resume LOG` continues a stopped session).

- `plan()`: two rounds of idle, the 9 signs, idle, so 22 recordings: every sign twice and four different idle variants (hands down, OK sign, half-formed signs, random movements).
- `min_hands(label, idle_index)`: 0.6 for signs, 0.3 for idle 2–4, 0 for the first idle (hands down).
- `save_new(label, person, lighting, frames, video_frames, aspect, extra)`: the same file format and names as `collect_data.py`, but it raises an error if the file already exists, and opens the `.npz` with mode `"xb"` (fails if the file appeared in the meantime). Extra meta entries are allowed but cannot replace the standard ones. `big(...)` draws large text. Both are reused by the training recorder.
- Before the first step a 10-second "step in front of the camera" countdown; then, per step, the hands-only countdown, 20 s of recording, the hands check, and the six-frame review.
- `--resume` re-checks earlier saved steps: a saved recording with too few hands is marked `rejected` in the log (so `make_test_set.py --exclude-from` can leave it out) and is recorded again.

The log goes to `reports/test_sessions/<start>.json`. The existing log `20261101_140445.json` belongs to the 22-recording test session of 2026-11-01. The first steps of that session were recorded by an older version without the review step; that is where the flawed ram and tiger recordings come from.

tools/assign_recordings.py

Purpose: put the recordings KEPT in a training session into train or validation, by id, by hand, with a safe dry run first.

Run: `python tools/assign_recordings.py --log reports/training_sessions/session1_<start>.json` (dry run) and the same with `--apply` (write). `--log` may be repeated.

`check_plan(logs, splits, recordings, data_dir, excluded)` builds the list of (id, label, set) and refuses a recording if:

- it is listed twice;
- its planned set is not train or validation (this tool can never put anything into test);
- it already has any set (existing entries never move; the test set is never touched);
- it is in `data/evaluation_exclusions.json`;
- its landmark file is missing, the label on disk differs from the log, or its video is missing.

If anything is refused, nothing is written. Otherwise the dry run prints the plan and the resulting counts per gesture (train / validation / test). With `--apply` it assigns with `splits.assign` (manual), saves, reloads, and checks that **every earlier entry is byte-for-byte unchanged** and every new one has its planned set; if not, it stops with "CHECK FAILED … Restore data/splits.json from the backup."

This is how the 27 session-1 recordings went into train (41 → 68 entries; train 16 → 43).

tools/make_test_set.py

Purpose: turn the recordings of a new session into the TEST set and evaluate them.

Run: `python tools/make_test_set.py --since "2026-11-01 12:00"`. Options: `--dry-run`, `--no-eval`, `--min-per-sign N` (coverage warning, default 2), `--exclude ID,ID`, `--exclude-from LOG` (leave out the ids a test-session log marked `rejected`).

Steps:

1. `scan` all recordings and keep those made at or after `--since` (`when(rec)` from the id's date and time).
2. `trained_on()` reads the installed model's `train_sessions` and `validation_sessions`; any such recording is REFUSED (it can never be test data).
3. Drop excluded ids (they stay unassigned, so `train.py` also ignores them).
4. Assign the rest to `test` by hand (`manual`), so the quotas can never send them to training, and save.
5. Print each recording with its share of frames with hands (`hands_share`) and a coverage table per gesture.
6. Unless `--dry-run` or `--no-eval`, start `evaluate.py run --sets test --sessions <those ids> --title "Independent test set: ..."` as a subprocess.

This made the 22-recording test set and the report `reports/2026-11-01_144049/`.

tools/grouped_cv.py

Purpose: condition-grouped cross-validation of the landmark model on the TRAIN set only (never validation, never test). It estimates how the model does under a camera setup it has not seen.

Run: `python tools/grouped_cv.py` (options `--step`, `--out`, `--model`).

Groups (whole recording conditions):

group	which TRAIN recordings
G1 oct30	every train recording made on 2026-10-30 (old setup)
G2 A	2026-11-01, camera at face level (meta `camera` = A)
G3 B	2026-11-01, camera low, looking up
G4 A-far	2026-11-01, face level, sitting back

`group_of(meta, date)` maps a recording to its group and stops if one has no group. For each model of `train.make_models()`, four folds: train on three groups (with `train.augmented`, so mirror + jitter on the training part only) and predict probabilities on the fourth group's original frames. Predictions are pooled out of fold.

`score(...)` reports for each model: pooled macro-F1 and accuracy, per-class precision / recall / F1, the confusion matrix, macro-F1 and recall per group, accuracy per recording, and what the app's landmark path would accept. `landmark_gate(probs, labels, hands)` re-applies the app's rules: the top class must be in `LANDMARK_SIGNS`, have `p >= LANDMARK_MIN_PROB` (0.60) and a margin over the runner-up `>= LANDMARK_MIN_MARGIN` (0.30), and for the palm the open-palm check `open_palm(hands, PALM_MIN_THUMB_GAP)` must pass. From that it reports the share of clone_seal / palm frames accepted, and false triggers (idle or seal frames accepted as palm or clone).

Baseline: the installed model scored on G2–G4, the groups it never saw (it stops if the installed model was trained on a Nov-1 recording). Output: `reports/<time>_grouped_cv/metrics.json` and `confusion_<model>.png`. Nothing is trained for real or installed: the models exist only in memory.

tools/evaluate_pipeline.py

Purpose: extra measurements for `reports/ai_evaluation/`, on top of `evaluate.py`, all with the app's own code. It was written when there was no test set yet (16 train, 3 validation, 0 test). Each part writes `reports/ai_evaluation/parts/<part>.json`.

Run: `python tools/evaluate_pipeline.py <part>` with part one of:

- `negatives --cache DIR`: never-trained clips of hands making no sign of the app (claw = old `chidori`, OK sign = old `fire`, two open palms in `data/_discarded`) plus the held-out idle recordings. Reports what each recogniser says per frame, which signs the app confirms, and how many jutsu fire in Live and Practice mode. `rule_of_three` reports events per hour and, when none happened, the 95% upper bound 3 / exposure hours.
- `loro`: leave-one-recording-out for the landmark classifier: `train.py`'s Random Forest refitted in memory without each recording in turn. A sign with only one recording cannot be right when its recording is held out, so those rows are shown but left out of the metrics.
- `sequences --cache DIR`: "spliced" performances (windows cut from single-sign recordings, in jutsu order, with 0.4–1.2 s idle gaps) through fusion + controller: activation rate, wrong jutsu, latency; all four jutsu back to back; long random sign streams to count unintended activations. Needed because no guided takes exist.
- `montecarlo`: the `SequenceMatcher` alone fed a random sign every 1.8–3.6 s for 200 simulated hours (Live and Practice), compared with the previous matching rule.
- `fps --url ws://127.0.0.1:8003/ws --fixtures DIR`: streams `.y4m` camera fixtures to the real server over the WebSocket like a browser and records the per-stage timings.
- `figures`: PNG charts (`confusion_landmarks_loro.png`, `fps_stages.png`).
- `summary`: collects everything into `reports/ai_evaluation/metrics.json`.

Key helpers: `Clip` and `split_clips` / `negative_clips` / `load_clip` (YOLOX results are cached as pickles in `--cache`), `Stream` (a timeline built from clip windows), `controller_for(mode, jutsu)` (Live = every jutsu with `SEQUENCE_MAX_GAP`; Practice = one jutsu, no time limit), `run_stream` (like `evaluate.replay`). The script **refuses a cache folder inside the repository**, because the cached clips show the user.

tools/check_voices.py

Purpose: check the 8 character voice clips in `assets/audio/voice/` (`<jutsu>/activation.wav` and `<jutsu>/name.wav` for fire_style, chidori, shadow_clone, rasengan) before they are used.

Run: `python tools/check_voices.py`. Exit code 0 only when all 8 are present and within spec.

For each clip `check(path, part)` reports: missing, or 16-bit PCM WAV, mono, 44.1 or 48 kHz; length 0.25–2.0 s (activation) or 0.25–2.5 s (name); at most 30 ms of silence before the voice (signal above −40 dBFS); peak at most −1 dBFS; loudness −16 ± 2 LUFS. `k_weighting(rate)` designs the two ITU-R BS.1770 pre-filters for any sample rate and `integrated_lufs(x, rate)` computes gated integrated loudness (400 ms blocks, 75% overlap, absolute gate −70 LUFS, relative gate −10 LU). Other files in the folder are listed as "not used".

Why it matters: the server reads the clip lengths to time the activation (voice → charge → name → release). The docstring also states that only licensed recordings belong there.

tools/build_bold_signs.py

Purpose: make the bold hand-sign drawings the web app shows in its sequence cards.

Run: `python tools/build_bold_signs.py`. Reads `gesture_references/_sources/signs/*.png` (thin black line art, Schlum, Wikimedia Commons, CC BY-SA 4.0) and writes `gesture_references/_sources/signs_bold/` (same names and sizes, transparent background) plus a `CREDITS.txt`. The originals are not changed.

- `stroke_width(gray)` measures the median stroke width with a distance transform (twice the distance on the stroke centre lines), at 3× resolution.
- `display_scale(w, h)` estimates how much a drawing is shrunk in the tiles (geometric mean over two tile box sizes).
- `bolden(gray, factor, min_width)` widens every stroke in proportion to its own width around its centre line (thin creases get a minimum width), keeps the grey nail fills slightly darker, and makes the white background transparent.

The factor is chosen per drawing so outlines are about 1.6 CSS pixels wide on screen, so all drawings look equally bold side by side.

tools/build_gesture_references.py

Purpose: build the reference cards in `gesture_references/`: for every jutsu folder `step_1.png` … `step_4.png` (one card per sign) and `sequence_reference.png` (all four steps).

Run: `python tools/build_gesture_references.py`.

- The step order comes from `config.JUTSU` (the same table the app uses); all texts (sign names, Japanese names, cues, titles, tags, credits) come from `shinobi/catalog.py`, the same texts the web app gets from `/api/jutsu`.
- `draw_palm(size)` renders the custom Rasengan palm from one recorded MediaPipe hand pose (`gesture_references/_sources/rasengan_palm_landmarks.json`), because this pose has no published drawing: finger "capsules", palm polygon and a forearm stub, drawn twice (dark grown shapes, then white shapes) so only a clean outline remains. `sign_art` uses it for the palm and the Wikimedia drawings for the seals.
- `step_card(...)` draws a 1280×1000 card: coloured header with jutsu title, step counter and progress dots; sign name and Japanese name; badges (OFFICIAL SEAL / CUSTOM POSE, BOTH HANDS / ONE HAND); the drawing; HOW TO / WHERE / HOLD / NEXT instructions; and the credit line.
- `sequence_sheet(...)` draws a 2500×1000 sheet with four step panels, arrows, an "activates" box and the note that order matters.
- Helpers: `font`, `bold`, `regular`, `japanese` (Windows fonts Segoe UI and Yu Gothic), `wrap` (greedy word wrap), `badge`, `fit`.

`tests/test_catalog.py` renders these cards into a temporary folder and checks they are byte-identical to the committed ones.

reports/ — what each folder is

folder	what it is
`reports/2026-10-31_024126/`	the pilot run of `evaluate.py` (validation + train, no landmark model, no test recordings). Proves the tool works; not a result.
`reports/ai_evaluation/`	the AI evaluation written before any test set existed: `README.md` (the report, with evidence grades), `metrics.json`, `parts/*.json` from `tools/evaluate_pipeline.py`, two `evaluate.py` runs (`evaluate_py/` with the installed model, `evaluate_py_yolox_path/` without it), `confusion_landmarks_loro.png`, `fps_stages.png`
`reports/test_sessions/20261101_140445.json`	log of the independent test recording session (22 recordings)
`reports/2026-11-01_144049/`	**the 2026-11-01 evaluation**: `evaluate.py run` on the 22 new test recordings (made by `tools/make_test_set.py`), with the installed Random Forest; `summary.md`, `metrics.json`, confusion PNGs for landmarks, yolox and fused. This run was made before the exclusions existed, so it includes all 22.
`reports/2026-11-01_diagnosis/`	**the diagnosis** of why that evaluation scored low: `report.md`, `scripts/` (`diagnose.py`, `sheets.py`, `ablation.py`) and `evidence/` (contact sheets of each sign, dense frame strips of the first ram and tiger, `diagnosis.json`, `ablation.json`, `timeline.json`). Diagnostic only; nothing was tuned on the test set.
`reports/training_sessions/session1_20261101_155047.json`	log of guided training session 1 (27 kept recordings)
`reports/2026-11-01_session1_plan/`	**the session 1 plan**: inspection of the 27 new recordings, the dry-run assignment, and the revised training plan (grouped cross-validation, model choice, leakage safeguards)
`reports/2026-11-01_191152_grouped_cv/`	**grouped CV + phase 3 comparison**: `summary.md` (Phase 2: condition-grouped cross-validation), `phase3_comparison.md` (Random Forest vs MLP), `metrics.json` from `tools/grouped_cv.py`, and confusion PNGs for the RF, the MLP and the installed model on G2–G4

tests/ — what each test module checks

All Python tests run with `pytest`. Tests that write files use pytest's `tmp_path` (a fresh temporary folder), so they never touch the real `data/` or `models/`. Some tests read real recordings from `data/videos/` (read

only). Tests that need the trained landmark model are skipped if `models/gesture_model.joblib` is missing. `tests/__init__.py` is empty; it only makes `tests` a package so modules can import each other (for example `from tests.test_splits import write_session`).

tests/test_core.py

The core building blocks, with synthetic hands:

- features: size of the feature vector and a missing second hand; features ignore the position and scale of the world coordinates; mirroring twice gives the original;
- GestureStabilizer: needs a hold and fires once; ignores single-frame spikes; smoothing is per second, not per frame (and equals the old per-frame EMA at its reference rate); re-arms after release;
- SequenceMatcher and ActionController: the full sequence is preferred and times out; cooldown; a single gesture is not a sequence prefix that fires;
- every effect renders without crashing (one test per effect via `parametrize`);
- `train.py` end to end on synthetic data in a temporary folder: the old label `rasengan` is trained as `rasengan_palm`, the old `fire` is skipped, and the resulting classifier has labels `idle, tiger, rasengan_palm`;
- the classifier's "fast forest" (walking the trees itself for one frame) gives exactly sklearn's probabilities.

tests/test_splits.py

`splits.py`, `python evaluate.py splits` and `train.py` with splits, all in temporary folders. A helper `write_session` creates a small recording in the `collect_data.py` format. It checks: every recording is in exactly one set and legacy labels are mapped; a recording listed twice or with an unknown set is refused; two files with the same id are refused; assignments are sticky and hand edits win; only new recordings get a set; test comes from another day or lighting and never shares it with train; the same lighting on another day counts as different; the fill order is train, test, validation; takes avoid the day and light their signs were trained on; the review warns about missing sets and a test from the training day; the splits command writes only the splits file; and `train.py` uses train sessions only and never opens test ones.

tests/test_evaluate.py

`evaluate.py`: frame metrics on a known example; perfect and empty predictions; a confusion matrix with different rows and columns; the latency summary; take outcomes; that `saved_video_frames` follows the recorders' rule; hands rebuilt in video pixels; YOLOX-alone skips unused classes and weak scores; the seal schedule is rate-limited and takes the newest frame; the replay fires a jutsu once like the app; a model trained without splits is not used; a full run writes a report and nothing into `data/`; the exclusion record is checked; excluded recordings are evaluated but scored apart; exclusions outside the run or switched off change nothing; a broken exclusion record stops the run.

tests/test_recording_tools.py

The guided recorders' helpers: the training plan has the approved counts (6 train / 2 validation for each weak sign and idle, 3 / 1 for each strong sign, 68 in total); validation is a session of its own and every block has a known camera; the palm is always "ONE hand only" and idle never asks for an open palm; `--plan`

needs no camera (the Camera is replaced by a function that fails the test if called); session 3 needs a light tag; the review picks six spread frames, draws the hands at the camera size, maps keys to keep / redo / quit, and a closed window keeps nothing; `save_new` keeps extra meta but never overwrites; `assign_recordings` dry run writes nothing, `--apply` keeps old entries unchanged, and every refusal case (already in test, excluded, no video, wrong label, planned set test, no file, listed twice) writes nothing.

tests/test_sequences.py

The four-step jutsu sequences: every jutsu has four known signs and its own first sign; each sequence fires only its own jutsu; reference pictures exist for every step; no other order of the same signs fires; incomplete sequences do not fire; a pause longer than the maximum gap resets the chain; a wrong sign keeps the progress and an activation resets everything; the last sign of one jutsu does not start another; conflicting configs (same sign twice in a row, a jutsu hidden inside another) are refused; held signs fire exactly once, also after a dropout; repeated first or last signs are harmless; Practice mode has no time limit and keeps steps after many wrong attempts; Free (Live) mode follows every jutsu at once, each with its own clock; plus `record_sequences.py`'s timeline layout, generated negative takes that never fire, and a saved take with landmarks and frame labels.

tests/test_fusion.py

The YOLOX detector and SignFusion: the detector returns nothing on an empty frame and finds each seal in its reference drawing; non-maximum suppression keeps the best box; the background worker reports latency; a YOLOX seal needs two results in a row on a hand; a box away from the hands is rejected; without any hand a higher score is needed; stale results are ignored; seals no jutsu uses are ignored; the landmark model owns clone_seal and palm and never overrides YOLOX seals; the palm needs an open hand, not an OK sign (and an OK sign at the last step never fires Rasengan); an unsure landmark sign is no sign; conflicts are won by the higher score and counted; Chidori from YOLOX fires once; Rasengan mixes both models and fires once; a wrong order fires nothing.

tests/test_engine.py

`shinobi/engine.py` and `shinobi/main.py` driven without a window, using frames of a real idle recording from `data/videos/idle/`: any frame size (landscape, portrait) with Shadow Clone running; a size change in the middle of a Shadow Clone; the result has what the HUD and the server need; Practice mode reset and activation numbers; the Free-mode time limit and timeout; no sign counts until the engine is ready; landmarks are drawn only when asked; and `main.py` runs headless with a fake camera, where key 1 starts the Shadow Clone.

tests/test_events.py

`shinobi/events.py`, which turns controller and effect states into web protocol events: nothing happens, nothing is sent; sign then step events; the last sign is a jutsu event; wrong-sign rules in Free and Practice mode; repeats only get a sign event; timeouts reported once for the jutsu furthest along (never in Practice); reset and mode change forget the sequence silently; effect lifecycle (stopping then end, replacement); the clones event only when more clones are visible than ever; and checks against the real

ActionController (Live mistakes keep the progress, steps just under the limit fire, Practice resets nothing after 30 s or 10 minutes, steady YOLOX results fire Chidori once).

tests/test_activation.py

The activation stages of a fired jutsu (`effects/base.py`): voice → charge → name → release, timed by the installed voice clips. Without clips the build-up is short; with clips the stages last as long as the WAV files; the voice listing is WAV only with lengths; every effect builds up then releases on time; effects started by hand (keyboard) have no build-up and report no stages; the charge level rises; Shadow Clone pops only after the name call; stopping during the build-up fades away without a release; hand effects survive a lost hand for their whole duration, with a configured minimum; each stage is reported once and in order even after a slow frame.

tests/test_shadow_clone.py

The Shadow Clone with exactly 10 clones: one entry per clone, balanced, all delays different; wrong clone lists are refused, not shortened; all clones are drawn from live frames and the person stays in front; every clone body stays inside the frame and all ten heads are visible wherever the user stands; clones go where the room is; a walking user moves clones across in smoke without flicker; too little room keeps every clone and says so; the parallel layer is one frame behind and the user stays in front; a head cut by the top edge gets a soft edge; clones pop in and vanish with smoke; the sequence starts the effect with all clones; the segmentation mask has no NaN on a plain bright wall.

tests/test_catalog.py

`shinobi/catalog.py`, the single copy of the card and web texts: every sign and jutsu has its texts; the kanji is the one in the instructions; credit items have exactly four keys; and the reference cards rendered from the catalog (into a temporary folder) are byte-identical to the committed ones.

tests/test_server.py

`shinobi/server.py`: REST endpoints (`/api/info`, the catalog follows config and every picture is served, credits and every sound are served, voice clip listing, the frontend fallback and build help); the binary frame pack / unpack round trip; the WebSocket protocol (hello, ping, mode, options, reset; frame round trip with mirroring and scaling; too many frames in flight are skipped in order; bad frames are skipped; "busy" beyond the maximum sessions and the slot freed after disconnect; a crash closes only that session). The end-to-end tests stream the user's real recordings through the WebSocket in real time and check that the real pipeline fires Fire Style, Chidori in Practice, nothing for a wrong order, a retry after a wrong sign, only the session doing the seals gets the jutsu, Shadow Clone and Rasengan (these two need the landmark model, otherwise skipped), and Shadow Clone rendering through the WebSocket.

Current state of the model and evaluation

This section summarises the report files honestly. All data is from one person ("me"), one webcam and one room.

Dataset counts

set	recordings	where they come from
train	43	16 from 2026-10-30 (one recording per seal, monkey 2, idle 4, palm 3; mostly night) + 27 from guided session 1 on 2026-11-01 (daylight, cameras A, B, A-far, all reviewed with six frames)
validation	3	2 idle + 1 palm, from 2026-10-30; no seal at all, so it cannot choose a model for 8 of the 10 gestures
test	22	the independent test session of 2026-11-01 (14:04–14:24): every sign twice, 4 idle; **5 of them excluded** from the scores, so 17 valid

Ignored: 8 old-label recordings. Unassigned: 1 rejected snake recording. Sessions 2 and 3 of the training plan were cancelled by the user, so no new recordings will be made for now.

The 2026-11-01 evaluation (reports/2026-11-01_144049)

With the installed Random Forest, on all 22 test recordings (before exclusions): landmarks accuracy 62.7% (macro-F1 0.615), YOLOX alone 69.3% (0.604), fused app decision 49.4% (0.462). Weak spots were snake, tiger, ram, dog (YOLOX), clone_seal (rejected by thresholds) and rasengan_palm (called idle).

Diagnosis findings (reports/2026-11-01_diagnosis/report.md)

1. Three test recordings do not show their label (tiger 140826: no Tiger; ram 140631: Ram only for ~9 s; idle 141753: open palms instead of an OK sign). The first two came from the old recorder version that ran ahead of the user.
2. Both test palm recordings were performed with two hands, while the definition and all training palms use one hand. The classifier reads two palms as idle. (These two plus the three above are the 5 exclusions.)
3. The camera setup changed: training (Oct 30) at face level; test (Nov 1) low and tilted up, with bright window light. YOLOX's Ram / Tiger / Snake / Dog scores collapse there, even though YOLOX was never trained on project data, so the conditions changed, not only the model.
4. Fusion listens to one model per sign. On correctly performed test recordings the landmark model was right on Ram 91%, Tiger 75%, Dog 95–100%, where YOLOX was at 0–27%; the correct landmark answer is thrown away.
5. Clone Seal is rejected mainly by `LANDMARK_MIN_PROB = 0.60` (top class in 95% of frames but p ≈ 0.47), with the top hand cut off at the frame edge and only one training recording.
6. Too little data to choose anything fairly: one training recording per sign, validation with idle and palm only.

Recommendations included the six-frame review in the recorder (done), more varied training data (session 1 recorded), retraining on train only, comparing fusion variants on validation (never on test), and one final evaluation on the unchanged holdout.

Session 1 plan (reports/2026-11-01_session1_plan/report.md)

The 27 new recordings had hands in 100% of frames, one-hand palms and a real OK sign. Even before training on them, they showed seal failures on YOLOX's side (dog at a low camera 6%, boar at a low camera 0%, snake sitting back 0%), which a better landmark model alone cannot fix without a fusion change. The dry run of `assign_recordings.py` assigned 27, refused 0, and kept all 41 old entries identical; it was then applied (train 16 → 43). The plan: choose the model by condition-grouped cross-validation on

TRAIN (because validation has no seals), keep thresholds and fusion unchanged, back up the installed model before any replacement, and report the final holdout numbers as "best available, not fully independent" (the holdout was recorded on the same day and in the same room as session 1, and the palm has no valid test recording at all).

Grouped cross-validation and Phase 3 comparison

From `reports/2026-11-01_191152_grouped_cv/summary.md` and `phase3_comparison.md` (43 TRAIN recordings, 7621 frames, groups G1–G4, cross-validation, not a test):

	Random Forest	MLP
macro-F1, pooled	0.856	0.889
accuracy, pooled	0.838	0.871
macro-F1 on G2–G4 (installed model there: 0.639)	0.877	0.901
fold macro-F1 mean ± SD	0.840 ± 0.051	0.859 ± 0.098
worst fold	0.789 (G1)	0.775 (G4)
inference per frame, median	0.41 ms	1.20 ms
clone_seal frames accepted by the app's rules	99.9%	100%
rasengan_palm frames accepted	59%	82%
idle frames wrongly accepted as palm	0.18% (2)	0.9% (12)

Reading: training on more conditions helps a lot on unseen conditions (0.64 → 0.88–0.90 on G2–G4). The MLP is better on average and fixes Boar at the low camera and the old-style palms; the RF is more stable and has fewer idle frames accepted as palm. Both fail Snake when sitting back (G4) and old-setup Tiger (G1); idle precision is low (about 0.6) because unsure sign frames fall back to idle. Clone Seal is solved for the landmark path in these conditions. Most class-by-group cells are a single recording, so numbers swing a lot.

The Phase 3 recommendation is the MLP, but **only after** (a) your approval, (b) a small explicit option in `train.py` to train that model (today `train.py` would choose by its own recording-level cross-validation), and (c) a replay of the retrained MLP on the idle recordings through fusion + stabilizer to confirm it never confirms a false Rasengan palm; otherwise the RF is the safe fallback. A backup of the current model exists (`SHINOBI-AI_backup_20261101_190821_phase2_model`).

Where it stands today

- **Current production model: the MLP** (`models/gesture_model.joblib`, md5 `ed330ca7…`), trained on the 43 TRAIN recordings with `train.py --model-type mlp` and installed on 2026-11-01 with the owner's approval. Report: `reports/model_experiment_20261101/deployment.md` (test set: app-level macro-F1 0.515 → 0.622, Clone Seal 6% → 100%, 0 false activations).
- The previous Random Forest (md5 `be848524…`) is kept in `models/previous_production_20261101/` and in the Desktop backup `SHINOBI-AI_backup_20261101_224739_before_model_experiment` for rollback.
- YOLOX, fusion and thresholds are unchanged.
- The final evaluation should be run once, after everything is frozen, on the 17 valid holdout recordings, with results shown with and without the 5 exclusions. Following the project's evaluation rules, that old holdout is now a diagnostic holdout (it was inspected during the diagnosis); a truly final number needs

a NEW test set recorded in new conditions. Tuning is done on validation or train cross-validation only, never on test.

How to run everything

All commands are for Windows PowerShell from the project folder `C:\Users\user\Desktop\SHINOBI-AI`, using the project's virtual environment.

Setup

```
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
cd frontend; npm install; cd ..
```

Run the app

```
.venv\Scripts\python.exe -m shinobi.server                  # web server on http://localhost:8000
.venv\Scripts\python.exe -m shinobi.server --port 8765      # another port
.venv\Scripts\python.exe -m shinobi.main                    # the OpenCV window version (keys 1-4, 0,
Q)
```

Frontend development and build

```
.venv\Scripts\python.exe -m shinobi.server                  # terminal 1: backend on 8000
cd frontend; npm run dev                                    # terminal 2: Vite on
http://localhost:5173
cd frontend; npm run build                                  # production build into frontend\dist
cd frontend; npm run typecheck                              # TypeScript check only
cd frontend; npm run lint                                   # oxlint
```

After `npm run build`, the server alone serves the app on http://localhost:8000.

Tests

```
.venv\Scripts\python.exe -m pytest -q                       # all Python tests
.venv\Scripts\python.exe -m pytest -q tests\test_splits.py  # one module
.venv\Scripts\python.exe -m pytest -v -k cooldown           # tests whose name contains "cooldown"
cd frontend; npm test                                       # vitest (frontend unit tests)
```

Browser end-to-end tests (real Edge with a fake camera, real backend, Vite dev server). The fixtures contain your face, so write them OUTSIDE the repository:

```
.venv\Scripts\python.exe frontend\e2e\make_fixtures.py --out C:\path\outside\repo\fixtures
.venv\Scripts\python.exe -m shinobi.server                  # terminal 1
cd frontend; npm run dev                                    # terminal 2
cd frontend; $env:SHINOBI_FIXTURES = "C:\path\outside\repo\fixtures"; node e2e/live-jutsu.mjs
fire_style
```

Other e2e scripts in `frontend/e2e/`: `live-jutsu.mjs chidori`, `live-wrong-order.mjs`, `practice.mjs`, `sequence-rules.mjs`, `two-browsers.mjs`, `reconnect.mjs`, `responsive.mjs`, `music-flow.mjs`, `audio-timeline.mjs`.

Recording data

```
.venv\Scripts\python.exe collect_data.py --person me --lighting day
.venv\Scripts\python.exe record_sequences.py --person me --lighting day
.venv\Scripts\python.exe tools\record_training_session.py --session 1 --plan
.venv\Scripts\python.exe tools\record_training_session.py --session 1
.venv\Scripts\python.exe tools\record_training_session.py --session 1 --resume
reports\training_sessions\<log>.json
.venv\Scripts\python.exe tools\record_test_session.py --person me --lighting day
```

Assigning recordings to sets (by id, dry run first)

```
.venv\Scripts\python.exe tools\assign_recordings.py --log reports\training_sessions\<log>.json
.venv\Scripts\python.exe tools\assign_recordings.py --log reports\training_sessions\<log>.json --apply
.venv\Scripts\python.exe tools\make_test_set.py --since "YYYY-MM-DD HH:MM" --dry-run
.venv\Scripts\python.exe tools\make_test_set.py --since "YYYY-MM-DD HH:MM" --exclude-from
reports\test_sessions\<log>.json
```

Do NOT run `python evaluate.py splits` on the real data: it assigns unassigned recordings automatically.

Training (only with your approval: it replaces the installed model)

```
.venv\Scripts\python.exe train.py
.venv\Scripts\python.exe train.py --out C:\path\outside\repo\candidate.joblib   # train without
replacing the installed model
.venv\Scripts\python.exe tools\grouped_cv.py                                    # condition-grouped CV,
nothing installed
```

Evaluation

```
.venv\Scripts\python.exe evaluate.py run                      # TEST set, exclusions applied ->
reports\<time>\
.venv\Scripts\python.exe evaluate.py run --sets validation    # numbers for tuning, NOT a final
result
.venv\Scripts\python.exe evaluate.py run --sessions ID1,ID2 --recognizers fused
.venv\Scripts\python.exe evaluate.py run --model none         # YOLOX + hands only
.venv\Scripts\python.exe tools\evaluate_pipeline.py loro
.venv\Scripts\python.exe tools\evaluate_pipeline.py negatives --cache C:\path\outside\repo\cache
```

Assets

```
.venv\Scripts\python.exe tools\check_voices.py
.venv\Scripts\python.exe tools\build_bold_signs.py
.venv\Scripts\python.exe tools\build_gesture_references.py
```